

Oracle HCM Fusion Fast Formula - Introduction

Introduction

Recently, I have been doing a lot of work with Oracle Fusion Fast Formula for HCM Compensation. So, I thought why don't I create a blog to share the knowledge I have gained?!

Well, in short, Fast formula is an Oracle HCM payroll engine and other products like Compensation (COMP), Benefit (BEN), Talents (OTL), Absences (ABS) and Payroll (PAY) use the engine. These products have provided places to attach formula, whenever there is a need of a complex/custom logic. The consultants add their own logic via a set of statements and subprograms which is the Fast Formula.

Language of Fast Formula

The Fast Formula uses its own language similar to C and PL/SQL. If you are familiar with either one of these languages, you will not find it difficult to understand the syntax of the fast formula.

Basic Components of Fast Formula

Before start writing any fast formula, you need to understand the following.

- 1) Formula types
- 2) Context
- 3) Database items (DBI)
- 4) Input values
- 5) Return Variables/Values.

Formula types:

The Formula types are created by products like Benefit (BEN), Compensation (COMP), and Payroll (PAY).-This is giving a name to a group of formulas. The name will help in identifying which product the formula is being used and where it is used.

Some examples are given below:

- **Compensation Person Selection:** This formula type is from Compensation (COMP) and is used to validate the person selection in Compensation and Total Compensation (TCS) process.
- **Participation and Rate Eligibility:** This formula type is from Benefit (BEN) and is used in COMP, BEN, TCS and ABS to determine the eligibility of a person.
- **Total Compensation Item:** This formula type is from Total Compensation (TCS) and is used to create or calculate a value for total compensation items.

Context:

Context is a very important part of Fast Formula. This is defined for a Formula Type. Different Formula Types support different set of Context. You can find the contexts that are supported by a formula type from the documentation of individual products like COMP, BEN etc.

The value to the context is set by the application when the fast formula is executed.

For example, the Compensation (COMP) sets the value to context like PERSON_ID, ASSIGNMENT_ID etc while executing the Fast Formula.

The context behaves like a value to the bind variable of a SQL.

For example, let us take the SQL statement,

“Select Full_name from PERSON_TABLE where person_id = :P_ID”

In this SQL, P_ID is a bind variable and the value for the bind variable is provided by the Context, PERSON_ID. The value to the context PERSON_ID is set by the application while executing the Fast Formula. In Fusion, Fast Formula developer can also set the value using method CHANGE_CONTEXTS provided the Fast Formula type supports the Context.

Related methods for the context are:

- GET_CONTEXT : To find the value from the context
- CONTEXT_IS_SET: To find whether a context is set
- CHANGE_CONTEXTS: to set one or more contexts

Database Items (DBI):

DBI is similar to a single data or a value from a column or a set of values (Array). In Fusion, you cannot write a meaningful fast formula without using DBI. DBI provides values from the database tables. The source of the DBI is called **Route**. Route is similar to a “*from* and *where*” clause of a SQL statement.

For example of Route: “FROM PERSON_TABLE WHERE PERSON_ID = :PID”.

For this Route, there could be many DBIs like Full_Name, First_Name, Last_Name, Date_of_Birth etc...

DBI	= Select clause	= Select person_id, full_name, date_of_birth
Route	= From clause and Where clause	= FROM Person_TABLE where person_id = :P_ID
Context	= Value for binding variables	= PERSON_ID = 000001

The number of DBI (Database items) supported by a Formula type depends on the context provided by the Formula type.

For example, if a Formula type supports context PERSON_ID and EFFECTIVE_DATE, the formula type can use all the DBI from person level. If it also supports ASSIGNMENT_ID and EFFECTIVE_DATE, it can use the DBI from Employment (Assignment) too.

Input Values:

Input values are additional values/information that are passed by the individual applications to the Fast Formula to assist the Formula developers while executing the Fast Formula. This information can be collected from individual application’s documentation.

For example,

- **Plan Start Date** and **Plan End Date** are input values of Compensation Plan Setup.
- **Period Start Date** and **End Date** are input values of Total Compensation Statement.

Return Variables/Values:

Most of the Formula types do not care about the return variables. There are some exceptions where some Fast Formulas expect to return more than one value. Therefore, the variable has to be set as per the documentation. In these cases, the formula will not work as expected if the return variables are not correct. In some other cases, even if there is one return value, it needs to be a specific return variable.

For example,

- Total Compensation Item Fast Formula expects the return variable, "COMPENSATION_DATES,VALUES, ASSIGNMENTS,LEGALEMPLOYERS."
- Benefit Eligible fast formula expects the return variable, ELIGIBLE.

In some cases, the formula type expects certain values. For example, Benefit Eligible and Compensation Person Selection Fast Formula types expect either a 'Y' or 'N'.

Mostly, the dates are returned in "YYYY/MM/DD" format.

The Fast Formula is created in Payroll Administration Area (You may navigate from other products too). This is a two part entry. In the first screen, you enter the basic details (Header) like Name, Type, Description, Legislative Data Group (LDG) and Effective Date of Fast Formula. And in the second part, you can code the Formula.

Name: Name is a unique and a required column.

We can create a fast formula with or without the Legislative Data Group (LDG). We will discuss the importance of LDG later in a different section.

If you provide a LDG, then the uniqueness is validated within the LDG while saving the formula.

If you have not provided a LDG, it will be considered as a global Fast Formula and the name has to be unique within the LDG and global Fast Formulas.

Type: Type is a required column. Once the name is entered, you can select the Formula Type. In our example, we use the Benefit Eligibility type, "Participation and Rate Eligibility.

For example, in one case, the user has created a Fast Formula in Compensation Work Bench's Formula type, 'Compensation Default and Override Rule' and wanted to attach the formula in Total Compensation Items and found that the formula is not available. The user should know the type and where it is used.

Description: It is an optional column.

Legislative Data Group: You do not need to select a LDG. It is an optional column but some formula types need the LDG. This mandatory attribute is defined in Formula Type of the Fast Formula. If the formula type defines the LDG attribute as required, then you will not be able to save the Formula without selecting LDG. For example, Formula Type, “**Absence**” does not allow to save without LDG. In our example, we are creating Global Formula and therefore we are not selecting the LDG for Fast Formula.

Effective Start date: This decides the availability of the Fast Formula for the products to select in their setups. One suggestion is to specify a date as early as possible.

For example, in one case, a formula in Total Compensation Item was created on current date (1-1-2016) and attached to the setup and the setup was processed for the period of 2015 (Jan 1 to Dec 31, 2015). The processed resulted in an error as the Formula is not compiled. The process was looking for a compiled formula and it was not available but the user is trying to compile the Formula again and again, still it is not working. Therefore, you need to understand the importance of Effective Start Date.

In the second part of the Fast Formula, you can code the logic for the fast formula.

```
ELIGIBLE = 'Y'
```

```
Return ELIGIBLE
```

In our example, we have provided a simple Fast Formula to explain the steps.

Save & Submit

Once the code is written for the fast formula, you can save the formula and submit the formula. The difference between saving and submitting is that the saved formula can be edited immediately but submitted formula cannot be edited immediately.

Compile

Once you submit the formula, you will need to compile the formula. Once you press the compile button, you will see a popup to confirm the compilation (Sometimes the popup does not appear, you may need to click the compile button again) Compilation will initiate an ESS process internally. Once the compilation has started the “Compile Status” icon will change from “X” to a

Clock icon. To see the status of the compilation, you need to press the refresh button. Once the compilation is complete, the “Compile Status” icon will show the result.

In case of an error, the error message will be displayed below the Formula editor window. In some releases, you can see the log from the ESS monitor page. (You can use any monitor page. For example, Compensation Work Bench monitor page, Compensation -> Run Batch Process -> Monitor Process)

Search and Edit the Fast Formula:

To Edit or Delete the Formula you need to search the formula first. If the Formula is created for a LDG, then you must provide the LDG when searching the formula though it is not a required column. If the Formula is Global (i.e. no LDG was specified while creating the formula), you must not provide the LDG information.

The edit feature provides four options (Correct, Update, Delete and End date) to edit (Since we are not discussing the date tracking, I do not want to get into details about these four options). You can use the “**Correct**” to edit the Fast formula. Once you change a Formula, the Compiled Status automatically changes to invalid. You need to Save/Commit and compile the formula again.

In our previous blog, we mentioned that the Benefit Eligibility formula expects the return variable “ELIGIBLE”. You can change the variable to something else. For example

RETURNVAL = ‘Y’

RETURN RETURNVAL

Still the formula compiles fine because the Formula engine does not impose this rule and the rule is validated by the Benefit, Eligibility. But the benefit eligibility will not work as expected.

This concludes the description of how to create, edit and compile a Fast Formula.

Hope this gives you some basic information about the Fast Formula entry steps.

If you have any comments or suggestions, please feel free to provide your comments below.

You Tube Tutorial: First Formula

*/******

Name: TCS_PERSON_SELECTION

type: Compensation Person Selection

Date: today

Dev: Tilak

Requirement: Select the Person who are

Female, age is 50 or more

and having 1 or more year experience in the org

******/*

/ Declare DBI */*

DEFAULT FOR PER_PER_SEX is 'X'

DEFAULT FOR PER_PER_PERSON_AGE is -1

DEFAULT FOR PER_PERSON_ENTERPRISE_HIRE_DATE is '1900/01/01' (date)

/ Input Value */*

INPUTS ARE CMP_IV_PERIOD_START_DATE (date), CMP_IV_PERIOD_END_DATE (date),
CMP_IV_STMT_ID, CMP_IV_PERIOD_ID

l_temp = ESS_LOG_WRITE('Entering TCS_PERSON_SELECTION ')

RETAVL = 'N'

l_person = get_Context(PERSON_ID, -1)

l_date = get_context(EFFECTIVE_DATE, '1900/01/01' (date))

l_temp = ESS_LOG_WRITE('l_person ' || to_char(l_person))

l_temp = ESS_LOG_WRITE('l_date ' || to_char(l_date))

l_gender = PER_PER_SEX

l_age = PER_PER_PERSON_AGE

```
l_hire_date = PER_PERSON_ENTERPRISE_HIRE_DATE
```

```
l_exp_months = MONTHS_BETWEEN(l_date , l_hire_date)
```

```
l_temp = ESS_LOG_WRITE('l_gender ' || l_gender )
```

```
l_temp = ESS_LOG_WRITE('l_age ' || to_char(l_age) )
```

```
l_temp = ESS_LOG_WRITE('l_hire_date ' || to_char(l_hire_date) )
```

```
l_temp = ESS_LOG_WRITE('l_exp_months ' || to_char(l_exp_months) )
```

```
/* Condition */
```

```
if l_gender = 'F' then (
```

```
    if l_age >= 30 then (
```

```
        if l_exp_months >= 12 then(
```

```
            RETAVL = 'Y'
```

```
        )
```

```
    )
```

```
)
```



```
l_temp = ESS_LOG_WRITE('Returning TCS_PERSON_SELECTION ' || RETAVL)
```

Return RETAVL

You Tube Tutorial: Second Fast Formula

```
/******
```

Name: TCS_PERSON_SELECTION

type: Compensation Person Selection

Date: today

Dev: Tilak

Requirement: Select the Person who are

Female, age is 50 or more

and having 1 or more year experience in the org

```
*****/
```

```
/* Declare DBI */
```

DEFAULT FOR PER_PER_SEX is 'X'

DEFAULT FOR PER_PER_PERSON_AGE is -1

DEFAULT FOR PER_PERSON_ENTERPRISE_HIRE_DATE is '1900/01/01' (date)

```
/* Input Value */
```

DEFAULT FOR CMP_IV_PERIOD_START_DATE is '1900/01/01' (date)

*INPUTS ARE CMP_IV_PERIOD_START_DATE (date), CMP_IV_PERIOD_END_DATE (date), CMP_IV_STMT_ID,
CMP_IV_PERIOD_ID*

```
l_temp = ESS_LOG_WRITE('Entering TCS_PERSON_SELECTION ')
```

RETAVL = 'N'

l_person = get_Context(PERSON_ID, -1)

l_date = get_context(EFFECTIVE_DATE, '1900/01/01' (date))

l_start_date = CMP_IV_PERIOD_START_DATE

l_temp = ESS_LOG_WRITE('l_person ' || to_char(*l_person*))

l_temp = ESS_LOG_WRITE('l_date ' || to_char(*l_date*))

l_temp = ESS_LOG_WRITE('l_start_date ' || to_char(*l_start_date*))

l_gender = PER_PER_SEX

l_age = PER_PER_PERSON_AGE

l_hire_date = PER_PERSON_ENTERPRISE_HIRE_DATE

l_exp_months = MONTHS_BETWEEN(*l_date* , *l_hire_date*)

l_temp = ESS_LOG_WRITE('l_gender ' || *l_gender*)

l_temp = ESS_LOG_WRITE('l_age ' || to_char(*l_age*))

l_temp = ESS_LOG_WRITE('l_hire_date ' || to_char(*l_hire_date*))

l_temp = ESS_LOG_WRITE('l_exp_months ' || to_char(*l_exp_months*))

IF CONTEXT_IS_SET(EFFECTIVE_DATE) THEN (

l_temp = ESS_LOG_WRITE(' Eff Date context Set ')

IF CMP_IV_PERIOD_START_DATE WAS NOT DEFAULTED Then (

l_temp = ESS_LOG_WRITE(' CMP_IV_PERIOD_START_DATE not defaulted ')

if *l_start_date* != *l_date* Then (

change_contexts(EFFECTIVE_DATE = *l_start_date*)(

l_age = PER_PER_PERSON_AGE

l_temp = ESS_LOG_WRITE('l_age CC' || to_char(*l_age*))

)

)else (

l_temp = ESS_LOG_WRITE(' start date not same as effective date ')

```

    )
)
)

/* Condition */

if l_gender = 'F' AND l_age >= 30 AND l_exp_months >= 12 then(

    RETAVL = 'Y'

)

l_temp = ESS_LOG_WRITE('Returning TCS_PERSON_SELECTION ' || RETAVL)

Return RETAVL

```

YouTube Tutorial: TOTAL Compensation Item Fast Formula 1:

```

/*

Name: TCS_ITEM_FF

Type: Total Compensation Item

Logic: Return annual Salary as of EO Period

*/

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS 0

DEFAULT FOR PER_ASG_LEGAL_ENTITY_ID IS 0

DEFAULT FOR CMP_ASSIGNMENT_SALARY_CURRENCY_CODE IS 'XXX'

INPUTS ARE CMP_IV_PERIOD_START_DATE (DATE) ,CMP_IV_PERIOD_END_DATE (DATE)

l_data = SET_LOG('Entering TCS_ITEM_FF')

L_START_DATE    = CMP_IV_PERIOD_START_DATE

L_END_DATE      = CMP_IV_PERIOD_END_DATE

l_hr_assign_id = get_context(HR_ASSIGNMENT_ID, -1)

L_DATA = SET_LOG( 'Start Date  :' + TO_CHAR( L_START_DATE,'YYYY/MM/DD'))

L_DATA = SET_LOG( 'END Date   :' + TO_CHAR( L_END_DATE,'YYYY/MM/DD'))

L_DATA = SET_LOG( 'Asg ID    :' + TO_CHAR(l_hr_assign_id))

COMPENSATION_DATES = TO_CHAR( L_END_DATE,'YYYY/MM/DD')

```

```

VALUES          = ''

UNIT            = CMP_ASSIGNMENT_SALARY_CURRENCY_CODE

LEGALEMPLOYERS  = TO_CHAR(PER_ASG_LEGAL_ENTITY_ID )

ASSIGNMENTS     = TO_CHAR(l_hr_assign_id)

CHANGE_CONTEXTS(EFFECTIVE_DATE = L_START_DATE)

(

  L_DATA = SET_LOG( 'ANN SALARY :' + TO_CHAR( CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT))

  VALUES = TO_CHAR( CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT)

)

l_data = SET_LOG('Leaving TCS_ITEM_FF')

RETURN  COMPENSATION_DATES ,VALUES, LEGALEMPLOYERS, ASSIGNMENTS, UNIT

-----*****-----

```

Fast Formula – Compensation- A Real Life Scenario Example

In previous sections, we went over the basics of the Fast Formula, input values and return values of Fast Formula of **Compensation** product.

In this session, we will create our first real life Fast Formula for Compensation Formula Type, Compensation Default and Override (for CWB) and Total Compensation Item (for TCS).

To write such a Fast Formula, we need to have a real life use case. Therefore, we have taken a hypothetical case which very closely resembles a real life scenario and we will write a Fast Formula for Compensation Default and Override (CWB) and Total Compensation Item (TCS) with a little difference.

Use Case:

- We need to calculate the salary of an employee.
- The salary frequency is Monthly.
- We need to calculate the salary as of the first date of every month.
- The salary needs to be calculated for every month within the Plan/Statement Periods.
- We do not need to calculate the salary for the month if the first date of the month is not part of the period. For example, we do not calculate for January 2015 if the period start date is 10-January-2015.
- We do need to calculate the salary for the month if the start date of the month is before the period end date. For example, we will calculate for December 2015 if the period end date is 20-December-2015.

- If there is FTE (Full Time Equivalent Factor), we need to consider the FTE. For example, FTE .5 when a person works for 4 hours a day and the standard working hours are 8 per day.
- The salary could be defined in different frequencies. For example, Monthly, Yearly or Daily. But the salary period is always defined as monthly. Therefore in this example, we will use the Annual Salary for calculation.
- **Compensation Default and Override (for CWB):** For CWB, in this example, we need to sum the amounts for the Plan Period and return one value. In case the salary amount has different currencies for the Plan Period, we do not calculate the sum but return -1.
- **Total Compensation Item (for TCS):** For TCS, In this example, we need to return the Salary value for every month. Since the value is returning on a monthly basis, we do not worry about the currency in Fast Formula.

The following is our analysis before we start writing the formula:

1. We need the start and end date of the Period (Input Values)
2. We need to ascertain the start date of the period. If the start date is not the first of the month, we need to calculate the salary from the first date of the next month.
3. We need to find the DBI for Annual Salary, Currency and FTE.
4. From the Annual salary, we need to derive the monthly salary.

Once we have the above information, we do the following:

- We loop through the dates from the start date to the end date of the period.
- We calculate the First date by using Add_Months Function.
- Within the loop, we set the effective date context to the first date of the month.
- We get the salary from Annual Salary DBI and divide the amount by 12 to get the monthly salary.
- If the FTE DBI value is not 1, then we multiply the salary amount with the FTE to derive the monthly average salary.
- For CWB, we calculate the sum of the monthly values. For TCS, we append the monthly values with a delimiter ','.

CWB FORMULA:

/*

Author : Tilak

Type : Compensation Default and Override

NAME : MY_FIRST_CWB_DEFAULT_FF

Remarks: This is a first simple formula to explain the functionality of a Fast Formula. This does not have an advanced feature.

Requirement: Calculate the sum of the salary as of the first date of the month within the period.

DBI used: CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
CMP_ASSIGNMENT_SALARY_CURRENCY_CODE

```

        PER_ASG_FTE_VALUE

    */

/*
Declare the DBI
*/
DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS 0
DEFAULT FOR PER_ASG_FTE_VALUE IS 1
DEFAULT FOR CMP_ASSIGNMENT_SALARY_CURRENCY_CODE IS 'XYZ'

/*
Declare the Input Values
*/
INPUTS ARE CMP_IV_PLAN_START_DATE (text), CMP_IV_PLAN_END_DATE (text)


L_hr_assign_id = get_context(HR_ASSIGNMENT_ID, -1)
L_DATA = ESS_LOG_WRITE( 'BEGIN MY_FIRST_CWB_DEFAULT_FF' )
L_DATA = ESS_LOG_WRITE( 'Start Date :' + CMP_IV_PLAN_START_DATE)
L_DATA = ESS_LOG_WRITE( 'End Date  :' + CMP_IV_PLAN_END_DATE)
L_DATA = ESS_LOG_WRITE( 'Asg ID   :' + TO_CHAR(L_hr_assign_id))


L_PL_START_DATE  = TO_DATE(CMP_IV_PLAN_START_DATE, 'YYYY/MM/DD')
L_PL_END_DATE    = TO_DATE(CMP_IV_PLAN_END_DATE, 'YYYY/MM/DD')
L_VALUE          = 0
L_CURR           = ''


/*
Determine the First date
*/
IF (L_PL_START_DATE <> ROUND(L_PL_START_DATE, 'MM')) THEN
(
    L_PL_START_DATE = TRUNC(ADD_MONTHS(L_PL_START_DATE, 1), 'MM')
    L_DATA = ESS_LOG_WRITE( 'New Start Date   :' + TO_CHAR( L_PL_START_DATE, 'YYYY/MM/DD'))
    L_DATA = ESS_LOG_WRITE( 'New End Date     :' + TO_CHAR( L_PL_END_DATE, 'YYYY/MM/DD'))
)

WHILE (L_PL_START_DATE <= L_PL_END_DATE)
LOOP(
    L_DATA = ESS_LOG_WRITE( 'New Start Date in Loop   :' + TO_CHAR( L_PL_START_DATE, 'YYYY/MM/DD'))
    CHANGE_CONTEXTS(EFFECTIVE_DATE = L_PL_START_DATE)
    (
        L_DATA = ESS_LOG_WRITE( 'Salary       :' + TO_CHAR( CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT ))
        IF (CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED ) THEN
        (
            L_AMOUNT = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
            L_DATA = ESS_LOG_WRITE( 'Annual Salary   :' + TO_CHAR(L_AMOUNT))
            L_DATA = ESS_LOG_WRITE( 'FTE           :' + TO_CHAR(PER_ASG_FTE_VALUE))
        )
    )
)

```

```

L_DATA = ESS_LOG_WRITE( 'CURRENCY    :' + CMP_ASSIGNMENT_SALARY_CURRENCY_CODE)

IF (PER_ASG_FTE_VALUE WAS NOT DEFAULTED) THEN
(
  L_AMOUNT = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT * PER_ASG_FTE_VALUE
)
IF (L_CURR = ' ') THEN
(
  L_CURR = CMP_ASSIGNMENT_SALARY_CURRENCY_CODE
)
IF ( L_CURR = CMP_ASSIGNMENT_SALARY_CURRENCY_CODE) THEN
(
  L_VALUE = L_VALUE + (L_AMOUNT/12)
  L_DATA = ESS_LOG_WRITE( 'CALC VALUE    :' + TO_CHAR(L_VALUE))
)ELSE
(
  L_DATA = ESS_LOG_WRITE( 'CURRENCY MIS MATCH    :')
  L_VALUE    = -1
Exit
)

)

)

L_PL_START_DATE = ADD_MONTHS(L_PL_START_DATE, 1)
)

L_DATA = ESS_LOG_WRITE( 'END MY_FIRST_CWB_DEFAULT_FF' )
RETURN L_VALUE

```

Explanation of the Fast Formula:

Once we determine the DBI, we need to declare and assign a default value to the DBI. Since the Fast Formula does not support null values, we need to define the default value for the DBI in case it returns a null value to avoid any unknown issues.

```
/*
```

```
  Declare the inputs and DBI
```

```
*/
```

```
  DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS 0
```

We can also check the DBI value is null or not null using the syntax 'WAS DEFAULTED' and 'WAS NOT DEFAULTED'.

Once the DBI is declared, we need to declare the Input Values.

We need to understand the type of input values passed. Some developers also default the Input Values. The product documentation provides the information about the input values and its types. Since this Fast Formula is attached to the formula type, *Compensation Default and Override*, we can see the input values passed to this one from [section 3](#) under Input Values. We declare the CWB dates as text but in TCS we declare them as date.

```
INPUTS ARE CMP_IV_PLAN_START_DATE (text), CMP_IV_PLAN_END_DATE (text)
```

For this example, we have explained the main points of the business logic below.

- We determine if the start date of the plan is the start of the month and if not, we change the start date to the beginning of next month.

```
/*  
Determine the Start Date  
*/  
IF (L_PL_START_DATE <> ROUND(L_PL_START_DATE, 'MM')) THEN  
(  
..  
..  
..)  
)
```

- We go through the loop for each month

```
WHILE (L_PL_START_DATE <= L_PL_END_DATE)  
LOOP (  
  L_DATA = ESS_LOG_WRITE( 'New Start Date in Loop   ' + TO_CHAR( L_PL_START_DATE, 'YYYY/MM/DD'))
```

- We change the context to the beginning of the month using CHANGE_CONTEXTS. When we change the context, the DBI values are extracted as of the new context value. The change context takes effect only within the brackets.

```
CHANGE_CONTEXTS(EFFECTIVE_DATE = L_PL_START_DATE)  
(  
  .....  
  .....  
  .....  
)
```

And we get the salary amount using the DBI, *CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT*.

```
CHANGE_CONTEXTS(EFFECTIVE_DATE = L_PL_START_DATE)  
(
```



```

L_DATA = ESS_LOG_WRITE( 'Salary    :' + TO_CHAR( CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT ))
IF (CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED) THEN
(
  L_AMOUNT = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
  L_DATA = ESS_LOG_WRITE( 'Annual Salary    :' + TO_CHAR(L_AMOUNT))
  L_DATA = ESS_LOG_WRITE( 'FTE    :' + TO_CHAR(PER_ASG_FTE_VALUE))
  L_DATA = ESS_LOG_WRITE( 'CURRENCY    :' + CMP_ASSIGNMENT_SALARY_CURRENCY_CODE)

```

- We check if the DBI, *PER_ASG_FTE_VALUE* has a null value (*WAS NOT DEFAULTED*) and if not null, we use this value to calculate the salary.

```

IF (PER_ASG_FTE_VALUE WAS NOT DEFAULTED) THEN
(
  L_AMOUNT = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT * PER_ASG_FTE_VALUE
)

```

- We check if the currency of the monthly salary amounts is same. If so, we calculate the sum of these values. Otherwise, we set the value to -1 and exit the loop.

```

IF (L_CURR = ' ') THEN
(
  L_CURR = CMP_ASSIGNMENT_SALARY_CURRENCY_CODE
)
IF ( L_CURR = CMP_ASSIGNMENT_SALARY_CURRENCY_CODE) THEN
(
  L_VALUE = L_VALUE + (L_AMOUNT/12)
  L_DATA = ESS_LOG_WRITE( 'CALC VALUE    :' + TO_CHAR(L_VALUE))
)ELSE
(
  L_DATA = ESS_LOG_WRITE( 'CURRENCY MIS MATCH    :')
  L_VALUE    = -1
  Exit
)
)
)

```

- We set the start date to the next month and continue the loop till the period end date.

```

L_PL_START_DATE = ADD_MONTHS(L_PL_START_DATE, 1)
)

```

The most important part is logging (debugging) the Fast Formula. You should try to add as much debug data as needed in the log. Once you feel the Fast Formula is working fine, you can comment them out. There are different logging mechanisms available: *ESS_LOG_WRITE*, *HRTRACE* etc. If the Formula is processed in ESS, we recommend using *ESS_LOG_WRITE()*.

```

L_DATA = ESS_LOG_WRITE( 'BEGIN MY_FIRST_CWB_DEFAULT_FF' )

```

The final part is the **return value**. As we explained in the previous section, you must know what the Fast Formula is expected to return. From [section 3](#), we know it can be any variable for Compensation Default and Override Formula type.

RETURN L_VALUE

Once the CWB Fast Formula has been compiled successfully, you can attach the Fast Formula to the Attribute Column property.

You can process the CWB Plan in Compensation -> Run Batch Process -> Start Workforce Compensation Cycle. Once the process is completed successfully, you can see the log messages in Compensation -> Run Batch Process -> Monitor process -> Log and output (Navigation may vary from release to release). By checking the log, you can easily debug the Fast Formula.

The sample of log message from this process is given below:

TCS Formula:

```
/*  
  Author : Tilak  
  Type  : Total Compensation Item  
  NAME  : MY_FIRST_TCS_ITEM_FF  
  Remarks: This is a first simple formula to explain the functionality of Formula. This does not have an advanced feature.
```

Requirement: Calculate the Salary as of first date of the months within the period and sum the salary.

```
DBI used:  CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT  
          CMP_ASSIGNMENT_SALARY_CURRENCY_CODE  
          PER_ASG_FTE_VALUE
```

```
*/
```

```
/*Declare the inputs and DBI */  
DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS 0  
DEFAULT FOR PER_ASG_FTE_VALUE IS 1  
DEFAULT FOR CMP_ASSIGNMENT_SALARY_CURRENCY_CODE IS 'XYZ'
```

INPUTS ARE CMP_IV_PERIOD_START_DATE (DATE) ,CMP_IV_PERIOD_END_DATE (DATE)

L_DATA = ESS_LOG_WRITE('BEGIN MY_FIRST_TCS_ITEM_FF')

L_PL_START_DATE = CMP_IV_PERIOD_START_DATE

L_PL_END_DATE = CMP_IV_PERIOD_END_DATE

L_hr_assign_id = get_context(HR_ASSIGNMENT_ID, -1)

L_DATA = ESS_LOG_WRITE('Start Date : ' + TO_CHAR(L_PL_START_DATE, 'YYYY/MM/DD'))

L_DATA = ESS_LOG_WRITE('END Date : ' + TO_CHAR(L_PL_END_DATE, 'YYYY/MM/DD'))

L_DATA = ESS_LOG_WRITE('Asg ID : ' + TO_CHAR(L_hr_assign_id))

L_DATA = ESS_LOG_WRITE(' start of Date Conversion')

COMPENSATION_DATES = ''

VALUES = ''

L_CURR = ''

L_DATA = ESS_LOG_WRITE(' Determine the First date')

/*Determine the First date*/

IF (L_PL_START_DATE <> ROUND(L_PL_START_DATE, 'MM')) THEN

(

L_PL_START_DATE = TRUNC(ADD_MONTHS(L_PL_START_DATE, 1), 'MM')

L_DATA = ESS_LOG_WRITE('New Start Date : ' + TO_CHAR(L_PL_START_DATE, 'YYYY/MM/DD'))

)

L_DATA = ESS_LOG_WRITE(' start of Date Loop')

WHILE (L_PL_START_DATE <= L_PL_END_DATE)

LOOP(

L_DATA = ESS_LOG_WRITE('New Start Date in loop : ' + TO_CHAR(L_PL_START_DATE, 'YYYY/MM/DD'))

CHANGE_CONTEXTS(EFFECTIVE_DATE = L_PL_START_DATE)

(

L_DATA = ESS_LOG_WRITE('ANN SALARY : ' + TO_CHAR(CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT))

IF (CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED) THEN

(

L_AMOUNT = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT

```

L_DATA = ESS_LOG_WRITE( 'Annual Salary   :' + TO_CHAR(L_AMOUNT))
L_DATA = ESS_LOG_WRITE( 'FTE       :' + TO_CHAR(PER_ASG_FTE_VALUE))
L_DATA = ESS_LOG_WRITE( 'CURRENCY    :' + CMP_ASSIGNMENT_SALARY_CURRENCY_CODE)


IF (PER_ASG_FTE_VALUE WAS NOT DEFAULTED) THEN
(
  L_AMOUNT = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT * PER_ASG_FTE_VALUE
)
IF (I_CURR = ' ') THEN
(
  L_CURR = CMP_ASSIGNMENT_SALARY_CURRENCY_CODE
)
IF ( L_CURR = CMP_ASSIGNMENT_SALARY_CURRENCY_CODE) THEN
(
  COMPENSATION_DATES = COMPENSATION_DATES + to_char(L_PL_START_DATE, 'YYYY/MM/DD') + ';'
  VALUES           = VALUES + TO_CHAR(L_AMOUNT/12) + ';'
  L_DATA = ESS_LOG_WRITE( 'CALC VALUE   :' + TO_CHAR(L_AMOUNT))
)ELSE
(
  L_DATA = ESS_LOG_WRITE( 'CURRENCY MIS MATCH   :')
)

)

)

L_PL_START_DATE = ADD_MONTHS(L_PL_START_DATE, 1)
)

L_DATA = ESS_LOG_WRITE( 'END MY_FIRST_TCS_ITEM_FF' )
RETURN COMPENSATION_DATES,VALUES

```

Total Compensation Item Fast Formula is added to the Total Compensation Item, when the Item type is Formula.

We can see the input values passed to this Fast Formula from [section 3](#), under heading, **Input Values**. We declare the TCS period dates as **Date** because the value to the **Input Value** is passed in canonical format.

```

INPUTS ARE CMP_IV_PERIOD_START_DATE (DATE),CMP_IV_PERIOD_END_DATE (DATE)

```

Also, you can notice from the TCS Fast Formula that the return variables are defined as “COMPENSATION_DATES” and “VALUES” and the values are added with delimiter, ‘;’ and the date values are added in YYYY/MM/DD format.

```

IF ( L_CURR = CMP_ASSIGNMENT_SALARY_CURRENCY_CODE) THEN
(
  COMPENSATION_DATES = COMPENSATION_DATES + to_char(L_PL_START_DATE, 'YYYY/MM/DD') + ';'
  VALUES           = VALUES + TO_CHAR(L_AMOUNT/12) + ';'
  L_DATA = ESS_LOG_WRITE( 'CALC VALUE   :' + TO_CHAR(L_AMOUNT))
)ELSE
...

...

RETURN COMPENSATION_DATES,VALUES

```

You may not be able to see the logs in the TCS Monitor page. Instead, you can see the logs of the TCS process from the CWB Monitor Page.

The sample of TCS log message from this process is given below:

Hope this has helped you with an understanding of a simple Fast Formula. We will go over more complex issues in the next section.

TCS Eligibility Fast Formula

Compensation You Tube Tutorial

/*

Name: TCS_ELIG_FROMULA

Type: Participation and Rate Eligibility

*/

DEFAULT FOR PER_ASG_REL_ORIGINAL_DATE_OF_HIRE is '2000/01/01' (date)

DEFAULT FOR GLOBAL_PAY_INTERFACE_EXTRACTION_DATE is '2000/01/01'

l_temp = ESS_LOG_WRITE('Entering TCS_ELIG_FROMULA')

ELIGIBLE = 'N'

personid = get_Context(PERSON_ID, -1)

assignmentid = get_context(HR_ASSIGNMENT_ID , -1)

l_temp = ESS_LOG_WRITE('Person ' + to_char(personid))

l_temp = ESS_LOG_WRITE('assignmentid ' + to_char(assignmentid))

today = TO_DATE(GLOBAL_PAY_INTERFACE_EXTRACTION_DATE, 'YYYY/MM/DD')

doj = PER_ASG_REL_ORIGINAL_DATE_OF_HIRE

```

l_temp = ESS_LOG_WRITE('Today ' + to_char(Today) )
l_temp = ESS_LOG_WRITE('hire date ' + to_char(doj) )
daydiff = DAYS_BETWEEN (today , doj)
l_temp = ESS_LOG_WRITE('daydiff ' + to_char(daydiff) )
if daydiff > ( 365 * 5) then (
    ELIGIBLE = 'Y'
)
l_temp = ESS_LOG_WRITE('Leaving TCS_ELIG_FORMULA : ' + ELIGIBLE)
RETURN ELIGIBLE

```

Parent and Child Formula

Fast Formula – Parent and Child Formula (Formula Calling Formula)

In the previous [section](#), we went over on how to create a sample Fast Formula that can be used in a real life scenario.

In this session, we will create a Fast Formula (Child) that can be called from another Fast Formula (Parent). In other words, we will explain the Parent-Child concept of a Fast Formula.

Why do we need to call another Fast Formula?

The Fast Formula documentation reads the functionality of the Formula calling Formula as

“Instead of writing large formulas, common calculations can be put into their own smaller formulas. These calculation formulas can then be called from other formulas that need the calculation.”

Yes, it can be used for separating the common routine in different Fast Formulas but this is more of a convenience than necessity. I have different reasons on why we need this feature more than just separating the common routines. Below are two examples.

- If an organization has employees from different legislation, we need to have different Fast Formulas to support data from different Legislation.
- If we need to get data from different applications, we need to work with different DBI and Context. Since our Formula Type does not support the entire Context, we need to use different Formula Types.

1) Legislative Data Group (LDG): There are many DBIs that can be used only in a Legislative Data Group specific formula. For example, in Payroll Element Entry values DBI, I believe that the entire Payroll related DBIs can be used only in LDG specific Fast Formula (needs to be verified though).

In case your organization has employees from more than one LDG (assume US, UK) and you want to use the element entry values in your Fast Formula (assume Salary Element Entry value), you cannot achieve your requirement in one Fast Formula. If you create a Fast Formula in US LDG, then this Fast Formula will error for UK employees.

In this case, you can follow these steps:

- You can create a Fast Formula (Parent) that is non LDG specific. This can be used by both UK and US employees.
- Create a Fast Formula (Child) in US LDG. Let us assume this child Fast Formula is named as TILAKSHMI_FF_CHILD_US. We will use the Salary Element Entry from US LDG.
- Create a Fast Formula in UK LDG with the same name, TILAKSHMI_FF_CHILD_US. We use the Salary Element Entry for UK in this Fast Formula. If you remember, since we provided a LDG, the uniqueness is validated within the LDG while saving the Fast Formula. Please check the [second section](#) where we explained the uniqueness of Fast Formula name.
- Now, you can call the child Fast Formula, TILAKSHMI_FF_CHILD_US from your parent Fast Formula either by “**CALL_FORMULA()**” or “**EXECUTE()**”.
- You will attach the Parent Formula in your UI or process.
- In case you do not have same name for Child Fast Formula in UK and US LDG, you need to use IF condition in parent Fast Formula to call the right Fast Formula for the LDG.

For the above scenario, when the parent Fast Formula is executed:

- It will call the child Fast Formula as per the employee’s LDG.
 - If the employee belongs to US, then the parent will call the child, TILAKSHMI_FF_CHILD_US that is created in US LDG.
 - If the employee belongs to UK, then the parent will call the TILAKSHMI_FF_CHILD_US that is created in UK LDG.

Create a Child Formula in US LDG.

Create a Child Formula of same type in UK LDG using the same **Formula Name**.

Formula Text:

/*

Author : Tilak

Type : Compensation Default and Override

NAME : TILAKSHMI_FF_CHILD_US

Remarks: This is a Child formula in US LDG

*/

/*

DEFAULT FOR MONTHLY_SALARYBASIS_ELEMENT_AMOUNT_ENTRY_VALUE IS -1

l_hr_assign_id = get_context(HR_ASSIGNMENT_ID, -1)

l_eff_date = get_context(DATE_EARNED, '1900/01/01 00:00:00' (date))

l_pay_assign_id = get_context(PAYROLL_ASSIGNMENT_ID, -1)

L_DATA = ESS_LOG_WRITE('BEGIN TILAKSHMI_FF_CHILD_US')

L_DATA = ESS_LOG_WRITE('ASG ID :' + TO_CHAR(*l_hr_assign_id*))

L_DATA = ESS_LOG_WRITE('PAY ASG ID :' + TO_CHAR(*l_pay_assign_id*))

L_DATA = ESS_LOG_WRITE('Date :' + TO_CHAR(*l_eff_date*, 'YYYY/MM/DD'))

L_VALUE = MONTHLY_SALARYBASIS_ELEMENT_AMOUNT_ENTRY_VALUE

L_DATA = ESS_LOG_WRITE('EXIT TILAKSHMI_FF_CHILD_US ' + TO_CHAR(*L_VALUE*))

RETURN *L_VALUE*

Explanation of Fast Formula:

- This is a simple Fast Formula which just returns a value from a DBI, *MONTHLY_SALARYBASIS_ELEMENT_AMOUNT_ENTRY_VALUE*.

Parent Formula is created without LDG.

Formula Text:

/*

Author : Tilak

Type : Compensation Default and Override

NAME : TILAKSHMI_FF_PARENT_US

Remarks: This is a Parent formula without any LDG

**/*

```
l_hr_assign_id = get_context(HR_ASSIGNMENT_ID, -1)
l_eff_date = get_context(DATE_EARNED, '1900/01/01 00:00:00' (date))
l_pay_assign_id = get_context(PAYROLL_ASSIGNMENT_ID, -1)

L_DATA = ESS_LOG_WRITE( 'BEGIN TILAKSHMI_FF_PARENT_US' )
L_DATA = ESS_LOG_WRITE( 'ASG ID :' + TO_CHAR(l_hr_assign_id))
L_DATA = ESS_LOG_WRITE( 'PAY ASG ID :' + TO_CHAR(l_pay_assign_id))
L_DATA = ESS_LOG_WRITE( 'Date :' + TO_CHAR(l_eff_date, 'YYYY/MM/DD') )

L_VALUE = 0
/* check if the child formula is available*/
IF (IS_EXECUTABLE('TILAKSHMI_FF_CHILD_US')) THEN
(
  /* call the child formula */
  EXECUTE('TILAKSHMI_FF_CHILD_US')
  L_VALUE = GET_OUTPUT('L_VALUE', 0)
)

L_DATA = ESS_LOG_WRITE( 'EXIT TILAKSHMI_FF_PARENT_US ' + TO_CHAR(L_VALUE) )
RETURN L_VALUE
```

Explanation of Fast Formula

This is again a simple Fast Formula. There are three points you need to understand here.

1. In one of our [previous sessions](#), we explained that the Formula engine only passes the Contexts that are used in the Formula. Therefore, it is the developer's responsibility to make sure the value for Context that is used in the Child formula is also passed to the Parent Formula. For this, you need to get all the Contexts in Parent formula though it is not used in the Parent Fast Formula. You can use GET_CONTEXT, NEED_CONTEXT etc to make sure that the value of the context are passed to the Parent Formula.

```
l_hr_assign_id = get_context(HR_ASSIGNMENT_ID, -1)
l_eff_date = get_context(DATE_EARNED, '1900/01/01 00:00:00' (date))
l_pay_assign_id = get_context(PAYROLL_ASSIGNMENT_ID, -1)
```

2. Before we call the child formula we make sure it is available.

```
IF (IS_EXECUTABLE('TILAKSHMI_FF_CHILD_US')) THEN  
(
```

3. Call the child formula and get the output from the Child Formula.

```
/* call the child formula */  
EXECUTE('TILAKSHMI_FF_CHILD_US')  
L_VALUE = GET_OUTPUT('L_VALUE', 0)
```

2) Context: As we discussed in our previous sessions, different Formula Types support different context sets. Therefore, different Formula Types support different sets of DBI. If you need to understand the relation between Formula Types, Context and DBIs, please read our [first session](#).

Since some DBIs can be used only in some Formula Types, we need to use different Formula Types to achieve our requirements. But our UI Field supports only one Formula Type, therefore, you need to use the child Fast Formula of different Formula Type to get the information from the DBI.

Let us go through an example to understand this part.

- For example, Compensation Default and Override (for CWB) needs to find the DBA Certification of a person to determine the Bonus.
- Let us assume the bonus is calculated as: 20% of Salary + \$1000, if the DBA Certification is obtained in the plan period.
- The DBA Certification information is available in the Talent Management. And we will use the DBI; 'HRT_PERSON_GENERIC_CONTENT_TYPE_CONTENT_ITEM_NAME' to find the certification.
- For using the DBI, 'HRT_PERSON_GENERIC_CONTENT_TYPE_CONTENT_ITEM_NAME', we need to use the context, 'TALENT_PROFILE_CONTENT_TYPE' and set the context value as 'CERTIFICATION'.
- This Context is supported only by Talent Formula Types. For example, "Generic Formula type for Profile Content" supports the Context, 'TALENT_PROFILE_CONTENT_TYPE'.

To achieve the above requirement:

- We will create a Parent Formula of Formula Type, "Compensation Default and Override"

- The Parent Formula will call a Child Formula Type, “Generic Formula type for Profile Content” after setting the context, 'TALENT_PROFILE_CONTENT_TYPE'.

Create a Child Formula in Talent Type and use the Talent DBI.

Formula Text:

```

/*
TYPE      : Generic Formula type for Profile Content
Name      : TILAKSHMI_TALENT_DBI_CERT
Author    : Tilak
*/

/*DEFAULT_DATA_VALUE FOR HRT_PERSON_GENERIC_CONTENT_TYPE_CONTEXT_NAME IS 'XXX'*/
DEFAULT_DATA_VALUE FOR HRT_PERSON_GENERIC_CONTENT_TYPE_CONTENT_ITEM_NAME IS 'XXX'
DEFAULT_DATA_VALUE FOR HRT_PERSON_GENERIC_CONTENT_TYPE_DATE_FROM IS '1900/01/01 00:00:00' (date)

INPUTS ARE IV_PLAN_START_DATE (text), IV_PLAN_END_DATE (text)
/*Default for Input values */

DEFAULT FOR IV_PLAN_START_DATE  IS '2001/01/01'
DEFAULT FOR IV_PLAN_END_DATE    IS '2001/01/01'

L_RETURN_VALUE = 'N'
L_DATA = ESS_LOG_WRITE( 'BEGIN TILAKSHMI_TALENT_DBI_CERT' )

l_person_id = get_context(PERSON_ID,-1)
l_eff_date = get_context(EFFECTIVE_DATE,'1900/01/01 00:00:00' (date))
l_type     = get_context(TALENT_PROFILE_CONTENT_TYPE , 'NONE')

```

```

L_DATA = ESS_LOG_WRITE( 'Person ID Context ' || TO_CHAR(l_person_id) )
L_DATA = ESS_LOG_WRITE( 'EFFECTIVE_DATE Context ' || TO_CHAR(l_eff_date) )
L_DATA = ESS_LOG_WRITE( 'TALENT_PROFILE_CONTENT_TYPE Context ' || l_type )
L_DATA = ESS_LOG_WRITE( 'CMP_IV_PLAN_START_DATE ' || IV_PLAN_START_DATE )
L_DATA = ESS_LOG_WRITE( 'CMP_IV_PLAN_END_DATE ' || IV_PLAN_END_DATE )

/* convert plan dates into date value */
l_start_date = TO_DATE(IV_PLAN_START_DATE, 'YYYY/MM/DD')
l_end_date = TO_DATE(IV_PLAN_END_DATE, 'YYYY/MM/DD')

L_DATA = ESS_LOG_WRITE( 'Start of the Loop in DBI ' )
l_index = HRT_PERSON_GENERIC_CONTENT_TYPE_CONTENT_ITEM_NAME.FIRST(-1)
l_counter = 1
/* loop the all the item value */
WHILE HRT_PERSON_GENERIC_CONTENT_TYPE_CONTENT_ITEM_NAME.EXISTS(l_index)
LOOP(
    L_DATA = ESS_LOG_WRITE( 'IN Index ' || TO_CHAR(l_index) )
    L_VALUE= HRT_PERSON_GENERIC_CONTENT_TYPE_CONTENT_ITEM_NAME[l_index]
    L_DATA = ESS_LOG_WRITE( 'ITEM VALUE ' || L_VALUE )
    /* Check the item value us DBA Certification */
    IF (L_VALUE = 'ORACLE DBA CERTIFIED') THEN (
        L_DATA = ESS_LOG_WRITE( 'DBA ' )
        l_DATE = HRT_PERSON_GENERIC_CONTENT_TYPE_DATE_FROM[l_index]
        /* if the certification received within the plan period */
        if (l_DATE >= l_start_date AND l_DATE <= l_end_date) THEN (
            L_DATA = ESS_LOG_WRITE( 'DBS IN CURRENT PLAN YEAR ' )
            L_RETURN_VALUE = 'Y'
            EXIT
        )
    )
    l_index = HRT_PERSON_GENERIC_CONTENT_TYPE_CONTENT_ITEM_NAME.NEXT(l_index,-1)
)

L_DATA = ESS_LOG_WRITE( 'End of the Loop' )

L_DATA = ESS_LOG_WRITE( 'END TILAKSHMI_TALENT_DBI_CERT: ' || L_RETURN_VALUE )
RETURN L_RETURN_VALUE

```

Explanation of Fast Formula:

This Child Formula uses array DBI. Since the objective of this session is not to go over Arrays, we will cover this part in the next session.

```
INPUTS ARE IV_PLAN_START_DATE (text) , IV_PLAN_END_DATE (text)
/*Default for Input values */
```

```
DEFAULT FOR IV_PLAN_START_DATE IS '2001/01/01'
DEFAULT FOR IV_PLAN_END_DATE IS '2001/01/01'
```

In the Business logic, the Fast Formula loops the through the DBI values and validates the value of the DBI. If the value is 'DBA Certification', and if the 'DBA Certification' has been received within the plan period, then we return 'Y'.

```
/* loop the all the item values */
WHILE HRT_PERSON_GENERIC_CONTENT_TYPE_CONTENT_ITEM_NAME.EXISTS(l_index)
LOOP(
  L_DATA = ESS_LOG_WRITE( 'IN Index ' || TO_CHAR(l_index) )
  L_VALUE= HRT_PERSON_GENERIC_CONTENT_TYPE_CONTENT_ITEM_NAME[l_index]
  L_DATA = ESS_LOG_WRITE( 'ITEM VALUE ' || L_VALUE )
  /* Check the item value us DBA Certification */
  IF (L_VALUE = 'ORACLE DBA CERTIFIED') THEN (
    L_DATA = ESS_LOG_WRITE( 'DBA ' )
    I_DATE = HRT_PERSON_GENERIC_CONTENT_TYPE_DATE_FROM[l_index]
    /* if the certification received within the plan period */
    if (I_DATE >= l_start_date AND I_DATE <= l_end_date) THEN (
      L_DATA = ESS_LOG_WRITE( 'DBS IN CURRENT PLAN YEAR ' )
      L_RETURN_VALUE = 'Y'
      EXIT
    )
  )
)
I_index = HRT_PERSON_GENERIC_CONTENT_TYPE_CONTENT_ITEM_NAME.NEXT(l_index,-1)
```

)

Create Parent Formula in Compensation Type to use it in Compensation.

Formula Text:

```
/*
TYPE      : Compensation Default and Override
Name      : TILAKSHMI_CWB_PARENT_DBA_CERT
Author    : Tilak

*/

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS 0

INPUTS ARE CMP_IV_PLAN_START_DATE (text) ,CMP_IV_PLAN_END_DATE (text)
DEFAULT FOR CMP_IV_PLAN_START_DATE  IS '2001/01/01'
DEFAULT FOR CMP_IV_PLAN_END_DATE    IS '4012/01/01'

L_DATA = ESS_LOG_WRITE( 'BEGIN TILAKSHMI_CWB_PARENT_DBA_CERT' )

l_person_id = get_context(PERSON_ID,-1)
l_eff_date = get_context(EFFECTIVE_DATE,'1900/01/01 00:00:00' (date))
L_RETURN_VALUE = 'N'
l_end_date = TO_DATE(CMP_IV_PLAN_END_DATE, 'YYYY/MM/DD')

L_DATA = ESS_LOG_WRITE( 'Person ID Context : ' || TO_CHAR(l_person_id) )
L_DATA = ESS_LOG_WRITE( 'EFFECTIVE_DATE context : ' || TO_CHAR(l_eff_date) )
L_DATA = ESS_LOG_WRITE( 'START DATE : ' || CMP_IV_PLAN_START_DATE )
L_DATA = ESS_LOG_WRITE( 'END DATE : ' || CMP_IV_PLAN_END_DATE )

L_VALUE = -1
/* When the salary is defined
*/
IF CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED THEN
```

```

(
  L_VALUE = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
  L_VALUE = (L_VALUE/100)*20
  L_DATA = ESS_LOG_WRITE( '20 % of Sal ' + TO_CHAR(L_VALUE) )

  IF (IS_EXECUTABLE('TILAKSHMI_TALENT_DBI_CERT')) THEN
  (
    SET_INPUT('TALENT_PROFILE_CONTENT_TYPE','CERTIFICATION')
    SET_INPUT('EFFECTIVE_DATE',l_end_date)
    SET_INPUT('IV_PLAN_START_DATE', CMP_IV_PLAN_START_DATE )
    SET_INPUT('IV_PLAN_END_DATE',CMP_IV_PLAN_END_DATE )
    EXECUTE('TILAKSHMI_TALENT_DBI_CERT')
    L_CERT_VALUE = GET_OUTPUT('L_RETURN_VALUE','N')
    L_DATA = ESS_LOG_WRITE( ' RETURN VALUE OF CERT ' || L_CERT_VALUE )
    /* add 1000 if cert is Y
    */
    IF (L_CERT_VALUE = 'Y') THEN
    (
      L_VALUE = L_VALUE + 1000
      L_DATA = ESS_LOG_WRITE( ' Bonus after certification : ' || TO_CHAR(L_VALUE))

    )
  )
)

L_DATA = ESS_LOG_WRITE( 'END TEST_CMP_CPR_ENROLLED_FF : ' || TO_CHAR(L_VALUE))
RETURN L_VALUE

```

Explanation of Fast Formula

Define the DBI and Input values that are used in parent and child Fast Formula.

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS 0

INPUTS ARE CMP_IV_PLAN_START_DATE (text) , CMP_IV_PLAN_END_DATE (text)

DEFAULT FOR CMP_IV_PLAN_START_DATE IS '2001/01/01'

DEFAULT FOR CMP_IV_PLAN_END_DATE IS '4012/01/01'

```
L_DATA = ESS_LOG_WRITE( 'BEGIN TILAKSHMI_CWB_PARENT_DBA_CERT' )
```

```
l_person_id = get_context(PERSON_ID,-1)  
l_eff_date = get_context(EFFECTIVE_DATE,'1900/01/01 00:00:00' (date))  
L_RETURN_VALUE = 'N'  
l_end_date = TO_DATE(CMP_IV_PLAN_END_DATE, 'YYYY/MM/DD')
```

As per the business logic, validate the salary. If the salary is not defined, do not proceed. If the salary is defined, calculate 20% for salary for Bonus.

```
L_VALUE = -1  
/* When the salary is defined  
*/  
IF CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED THEN  
(  
  L_VALUE = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT  
  L_VALUE = (L_VALUE/100)*20  
  L_DATA = ESS_LOG_WRITE( '20 % of Sal ' + TO_CHAR(L_VALUE) )
```

Check if the Child Fast Formula is available. If so, set the required context and input values.

```
IF (IS_EXECUTABLE('TILAKSHMI_TALENT_DBI_CERT')) THEN  
(  
  SET_INPUT('TALENT_PROFILE_CONTENT_TYPE','CERTIFICATION')  
  SET_INPUT('EFFECTIVE_DATE',l_end_date)  
  SET_INPUT('IV_PLAN_START_DATE', CMP_IV_PLAN_START_DATE )  
  SET_INPUT('IV_PLAN_END_DATE',CMP_IV_PLAN_END_DATE )
```

Just for this example, we use different input values. You do not need to do so in a real Fast Formula.

```
SET_INPUT('IV_PLAN_START_DATE', CMP_IV_PLAN_START_DATE )  
SET_INPUT('IV_PLAN_END_DATE',CMP_IV_PLAN_END_DATE )
```

If you notice, the Context is also set like the input values.


```
SET_INPUT('EFFECTIVE_DATE',l_end_date)
```

Execute the Child Fast Formula and get the return value. You need to use the return variables from Child Fast Formula ('L_RETURN_VALUE') to get the value. The second parameter ('N') is the default value. If the child formula returns 'Y' then we add \$1000 to the Bonus.

```
SET_INPUT('IV_PLAN_END_DATE',CMP_IV_PLAN_END_DATE )
EXECUTE('TILAKSHMI_TALENT_DBI_CERT')
L_CERT_VALUE = GET_OUTPUT('L_RETURN_VALUE','N')
L_DATA = ESS_LOG_WRITE( ' RETURN VALUE OF CERT ' || L_CERT_VALUE )
/*
    add 1000 if cert is Y
*/
IF (L_CERT_VALUE = 'Y') THEN
(
    L_VALUE = L_VALUE + 1000
    L_DATA = ESS_LOG_WRITE( ' Bonus after certification : ' || TO_CHAR(L_VALUE))
)
```

Please look at our [previous section](#) to see how the Fast Formula is added in the UI. In the same manner, please add the parent Fast Formula to the UI.

YouTube Tutorial 14

```
/******
Name: TCS_CHILD_FF
Type: Compensation Person Selection
Simple Child Formula
*****/
l_temp = ESS_LOG_WRITE('Entering TCS_CHILD_FF' )
VALUE = 'Y'
l_person = get_context(PERSON_ID, -1)
l_asg = get_context(HR_ASSIGNMENT_ID, -1)
l_temp = ESS_LOG_WRITE('Person ' + to_char(l_person) )
l_temp = ESS_LOG_WRITE('Asg ' + to_char(l_asg) )
if l_person = -1 OR l_asg = -1 THEN
```

```

(
  VALUE = 'Y'
)
l_temp = ESS_LOG_WRITE('Leaving TCS_CHILD_FF ' + value )
Return VALUE
/*****
Name: TCS_PARENT_FF
Type: Compensation Person Selection
Simple Parent Formula that calls Child Formula
*****/

l_temp = ESS_LOG_WRITE('Entering TCS_PARENT_FF' )
RETVALUE = 'N'
person = get_context(PERSON_ID, -1)
asg = get_context(HR_ASSIGNMENT_ID, -1)
l_temp = ESS_LOG_WRITE('Person ID ' + TO_CHAR(person) )
IF IS_EXECUTABLE('TCS_CHILD_FF') THEN
(
  set_input('MYTEST' , 'XXXXXX')
  set_input('PERSON_ID' 20000 )

  l_temp = ESS_LOG_WRITE('Entering TCS_PARENT_FF' )
  EXECUTE('TCS_CHILD_FF')
  l_var = GET_OUTPUT('VALUE', 'x')
  l_temp = ESS_LOG_WRITE('Child Return ' + l_var)
  if l_var = 'Y' then
  (
    RETVALUE = 'Y'
  )
)
)

```

YouTube Tutorial 16

```

/*****
Name: TCS_CHILD_FF
Type: Age Calculation
Simple Child Formula
*****/

```

```

DEFAULT FOR BEN_PGM_NAME is ' '
l_temp = ESS_LOG_WRITE('Entering TCS_CHILD_FF' )
VALUE = 'N'
ldate = get_Context(EFFECTIVE_DATE, '1900/01/01' (date) )
lbg = get_context(BUSINESS_GROUP_ID, -1)
lpgm = get_context(PGM_ID , -1)
lpgmname = BEN_PGM_NAME
l_temp = ESS_LOG_WRITE('Date ' + to_char(ldate) )
l_temp = ESS_LOG_WRITE('Bg ' + to_char(lbg) )
l_temp = ESS_LOG_WRITE('pgm ' + to_char(lpgm) )
l_temp = ESS_LOG_WRITE('pgm NAME ' + lpgmname)
l_temp = ESS_LOG_WRITE('Entering TCS_CHILD_FF ' + VALUE)
Return VALUE

```

/*****

Name: TCS_PARENT_FF
Type: Compensation Person Selection
Simple Parent Formula that calls Child Formula

*****/

```

l_temp = ESS_LOG_WRITE('Entering TCS_PARENT_FF' )
RETVALUE = 'N'
person = get_context(PERSON_ID, -1)
asg = get_context(HR_ASSIGNMENT_ID, -1)
l_temp = ESS_LOG_WRITE('Person ID ' + TO_CHAR(person) )
IF IS_EXECUTABLE('TCS_CHILD_FF') THEN
(
  l_var = 'N'
  CALL_FORMULA('TCS_CHILD_FF' ,
    100010025071996 > 'BUSINESS_GROUP_ID' ,
    100100081493359 > 'PGM_ID' ,
    l_var < 'VALUE' default 'X'
  )
  l_temp = ESS_LOG_WRITE('Child Return ' + l_var)
  if l_var = 'Y' then
  (
    RETVALUE = 'Y'
  )
)
)
l_temp = ESS_LOG_WRITE('Leaving TCS_PARENT_FF ' + RETVALUE )
RETURN RETVALUE

```

Fast Formulas and Arrays

Fast Formulas and Arrays

In this session, we will cover the usages of Arrays in Fast Formula.

Fast Formula uses Arrays since its inception. Even in **EBS** days, the return values are passed as name and value pair of Arrays.

In Fusion, the Fast Formula started using Arrays extensively. The knowledge of the Array in Fast Formula is so important that the developer would need to understand Arrays to write a meaningful Fast Formula. For example, Arrays can be used in

1. Input Values
2. DBI
3. Variables
4. Return values.

In this session, we will explain the above usages in detail with an example. Before we explain the usages, we will explain about the different types of Arrays.

In Fusion Fast Formula, Arrays are defined with two attributes:

1. **Type of Value** - indicates the value type (Number, Date or Char) stored in the Array.
2. **Type of Index** - indicates how the Array is indexed.

The following options are available for defining an Array in Fusion Fast Formula.

Array Type	Meaning
DATE_NUMBER	Date value and number index Array
DATE_TEXT	Date value and text index Array
NUMBER_NUMBER	Number value and number index Array

NUMBER_TEXT	Number value and text index Array
TEXT_NUMBER	Text value and number index Array
TEXT_TEXT	Text value and text Index Array

Usage of Arrays:

Input Values: There are many products in Fusion that pass the Input Values as Array. For example, Total Compensation Statement (TCS) process the data at a person level. Therefore, when the Fast Formula is executed for a person, it sets the primary Assignment Id as Context, HR_ASSIGNMENT_ID. In case if the person has more than one assignment, it passes all the assignments as Array to the Fast Formula so that the formula can use the Assignments to process the data.

The syntax for declaring the Array Input Value is:

```
INPUTS ARE CMP_IVR_ASSIGNMENT_ID (NUMBER_NUMBER)
```

You can also default the Array with an empty Array as follows.

```
DEFAULT for CMP_IVR_PAY_ASSIGNMENT_ID is EMPTY_NUMBER_NUMBER
```

To find the Array input values, you need to check the documentation of each of the products.

DBI: The Array DBI is also called range DBI. Every product in HCM has seeded these Array DBIs. It looks like the Array DBIs are more popular than the conventional ones in Fusion.

The Array DBIs are declared and defaulted as follows

```
DEFAULT_DATA_VALUE FOR PER_HIST_ASG_EFFECTIVE_START_DATE IS '1900/01/01 00:00:00' (date)
```

If you use the wrong syntax, you will get a compilation error.

If the Array DBI is not handled in a proper manner, it may lead to a performance issue. Since the Array DBI holds a range of data, you will need to implement the proper validation to get the desired results.

Variables: The variables are like any other Array, the only difference is how you declare them. You can declare the variable Array as follows.

DEFAULT FOR COMPENSATION_DATES is EMPTY_TEXT_NUMBER

Return values: Fast Formula engine allows returning Array variables provided the Formula Type supports it. The Product that executes the Formula determines whether the Array variables that are returned from Fast Formula are supported. As far as I know, in Compensation product, TCS supports Arrays for their Formula Type, **“Total Compensation Item”** in their latest release. (please check the documentation for the availability).

Fast Formula has provided the following functions to handle the Arrays.

Function	Syntax	Meaning
COUNT	<Array> .COUNT	Return the count of an Array elements Example: l_count = l_array.count
DELETE	<Array>. DELETE(<index value>)	Delete a element from the Array Example: l_array.delete(2) or l_array.delete('TWO') depending on the index type of the Array.
DELETE	<Array>. DELETE()	Delete all the elements. Example: l_array.delete()
EXISTS	<Array>. EXISTS (<index value>)	If a value exists at an index of an Array. It returns a Boolean. Example: if l_array.exists(1) then
FIRST	<Array>.FIRST(<default value>)	Returns first index of an Array. If there are no elements, then return the default value. Example: l_index = l_array.first(-1)

LAST	< Array>.LAST (<default value>)	Returns last index of an Array. If there is no elements then returns the default value. Example: I_index = I_array.last(-1)
NEXT	< Array>.NEXT (<index>, <default value>)	Returns the next index of the index parameter. If there is no next element, returns default value. Example: : I_index =I_array.next(I_index, -1)
PRIOR	< Array>.PRIOR (<index>, <default value>)	Returns the previous index of the index parameter. If there is no previous element, returns default value. Example: I_index =I_array.prior(I_index, -1)
	Array[<index>]	To get/set a value from/to an Array element. Example: L_number = I_array[100] I_array[100] = L_number

Sample **Fast** Formula to handle **Arrays**:

/*

Author : Tilak-Lakshmi

Name : TCS_ITEM_ARRAY_TEST

Type : Total Compensation Item

Purpose : Example of using Array DBI, input values and return variables. This formula returns changed salary of every assignment.

*/

/*Defaulting DBI */

DEFAULT_DATA_VALUE FOR CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT IS 0

/* Defaulting Input Values and variables*/

DEFAULT for CMP_IVR_ASSIGNMENT_ID is EMPTY_NUMBER_NUMBER

DEFAULT FOR COMPENSATION_DATES is EMPTY_TEXT_NUMBER

DEFAULT FOR VALUES is EMPTY_TEXT_NUMBER

DEFAULT FOR ASSIGNMENTS is EMPTY_TEXT_NUMBER

/* Declaring Input Values */

INPUTS ARE CMP_IV_PERSON_ID , CMP_IV_PERIOD_START_DATE(DATE), CMP_IV_PERIOD_END_DATE
(DATE),

CMP_IVR_ASSIGNMENT_ID(NUMBER_NUMBER) , CMP_IVR_TERM_ID(NUMBER_NUMBER),

CMP_IVR_PAY_ASSIGNMENT_ID (NUMBER_NUMBER), CMP_IVR_PAY_TERM_ID(NUMBER_NUMBER),

CMP_IVR_PAY_RELATION_ID(NUMBER_NUMBER)

I_data= ESS_LOG_WRITE('ENTERING TCS_ITEM_ARRAY_TEST ')

/* Get the first index of assignment */

index = CMP_IVR_ASSIGNMENT_ID.FIRST(-1)

I_data= ESS_LOG_WRITE(' first index ' + TO_CHAR(index))

I_counter = 1

/* Loop through the assignments */

WHILE (CMP_IVR_ASSIGNMENT_ID.EXISTS(index))

LOOP (

I_data= ESS_LOG_WRITE(' asg ' + TO_CHAR(CMP_IVR_ASSIGNMENT_ID[index]))

/*Set the context to assignment id */


```

CHANGE_CONTEXTS(HR_ASSIGNMENT_ID= CMP_IVR_ASSIGNMENT_ID[index] )

(

  l_asg = CMP_IVR_ASSIGNMENT_ID[index]

  l_data= ESS_LOG_WRITE( ' ASG CONTEXT IS SET ' + TO_CHAR(l_asg))

  /* get salary index and loop through the Array */

  l_sal_indx = CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT.FIRST(-1)

  WHILE (CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT.EXISTS(l_sal_indx ) )

  LOOP (

    if (CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT[l_sal_indx] <> 0 ) THEN

    (

      COMPENSATION_DATES[l_counter]= TO_CHAR(CMP_IV_PERIOD_START_DATE,'YYYY/MM/DD')

      ASSIGNMENTS[l_counter] = TO_CHAR(l_asg)

      VALUES[l_counter] = TO_CHAR( CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT[l_sal_indx])

      l_counter = l_counter + 1

    )

    l_sal_indx = CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT.NEXT(l_sal_indx , -1)

  )

)

index = CMP_IVR_ASSIGNMENT_ID.NEXT(index ,-1)

)

l_data= ESS_LOG_WRITE( ' LEAVING TCS_ITEM_ARRAY_TEST ' )

RETURN COMPENSATION_DATES, VALUES, ASSIGNMENTS

```

Explanation of the Formula:

When declaring an Array DBI, please note the syntax. It is a little different than the normal DBI declaration.

DEFAULT_DATA_VALUE FOR CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT IS 0

We use the same syntax to default the Input Values and for declaring/defaulting variables.

DEFAULT FOR CMP_IVR_ASSIGNMENT_ID is EMPTY_NUMBER_NUMBER

DEFAULT FOR COMPENSATION_DATES is EMPTY_TEXT_NUMBER

DEFAULT FOR VALUES is EMPTY_TEXT_NUMBER

DEFAULT FOR ASSIGNMENTS is EMPTY_TEXT_NUMBER

The Input Values are declared as below.

INPUTS ARE CMP_IV_PERSON_ID, CMP_IV_PERIOD_START_DATE(DATE),

CMP_IV_PERIOD_END_DATE (DATE),

CMP_IVR_ASSIGNMENT_ID(NUMBER_NUMBER) , CMP_IVR_TERM_ID(NUMBER_NUMBER),

CMP_IVR_PAY_ASSIGNMENT_ID (NUMBER_NUMBER), CMP_IVR_PAY_TERM_ID(NUMBER_NUMBER),

CMP_IVR_PAY_RELATION_ID(NUMBER_NUMBER)

To loop through the Array, whether it is DBI or Input Values; we need to identify the index type. In our example, all the Arrays are indexed by number. We can identify the index type from the document or the DBI list from the Fast Formula UI.

Once we know the index type, we need to identify the first index. Many developers use 1 as starting index but I prefer to get the first index from the Array itself. In the .FIRST method, we use -1 as default value. In case there is no element in the Array, the method will return the default value, -1.

```
index = CMP_IVR_ASSIGNMENT_ID.FIRST(-1)
```

```
I_data= ESS_LOG_WRITE( ' first index ' + TO_CHAR(index ) )
```

```
l_counter = 1
```

In the loop, we iterate as long as an element exists. We use the .EXISTS method to find the existence of the element for an index.

```
/* Loop through the assignments */  
  
WHILE (CMP_IVR_ASSIGNMENT_ID.EXISTS(index))  
  
LOOP (
```

We get the assignment id and set the Context, HR_ASSIGNMENT_ID with the assignment id, so that the DBI for the Assignment Id can be used.

```
/*Set the Context to the Assignment Id */  
  
CHANGE_CONTEXTS(HR_ASSIGNMENT_ID= CMP_IVR_ASSIGNMENT_ID[index] )  
  
(  
  
    l_asg = CMP_IVR_ASSIGNMENT_ID[index]  
  
    l_data= ESS_LOG_WRITE( ' ASG CONTEXT IS SET ' + TO_CHAR(l_asg) )
```

In the following code, we get the salary changes for the assignment and **then** assign the value in the Array variables.

```
l_sal_indx = CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT.FIRST(-1)  
  
WHILE (CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT.EXISTS(l_sal_indx ) )  
  
LOOP (
```

```
    if ( CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT[l_sal_indx] <> 0 ) THEN
```

```
    (
```

```
        COMPENSATION_DATES[l_counter]=  
TO_CHAR(CMP_IV_PERIOD_START_DATE,'YYYY/MM/DD')
```

```
        ASSIGNMENTS[l_counter] = TO_CHAR(l_asg)
```

```

VALUES[l_counter] = TO_CHAR(
CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT[l_sal_idx] )

l_counter = l_counter + 1

)

```

Get the next index for the DBI and Input values to move to the next element.

```

l_sal_idx = CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT.NEXT(l_sal_idx, -1)

)

index = CMP_IVR_ASSIGNMENT_ID.NEXT(index, -1)

)

l_data= ESS_LOG_WRITE( ' LEAVING TCS_ITEM_ARRAY_TEST ' )

RETURN COMPENSATION_DATES , VALUES,ASSIGNMENTS

```

Hope this section has helped you to get an understanding of the usages of Arrays in Fast Formula. Please let us know if you have any questions.

YouTube Tutorial on Array DBI

```

/*****

```

Name: TCS_PERSON_SELECTION

type: Compensation Person Selection

Date: today

Dev: Tilak

Requirement: Select the Person Whose total salary of all the Job is > 50K as of effective date

```

*****/

```

```

/* Declare DBI */

```

```

DEFAULT_DATA_VALUE FOR PER_HIST_ASG_EFFECTIVE_START_DATE is '1900/01/01' (date)

```

DEFAULT_DATA_VALUE FOR PER_HIST_ASG_EFFECTIVE_END_DATE is '1900/01/01' (date)

DEFAULT_DATA_VALUE FOR PER_HIST_ASG_ASSIGNMENT_ID is -1

DEFAULT_DATA_VALUE FOR PER_HIST_ASG_STATUS_TYPE is ''

DEFAULT_DATA_VALUE FOR PER_HIST_ASG_EFFECTIVE_LATEST_CHANGE is ''

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT is 0

l_temp = ESS_LOG_WRITE('Entering TCS_PERSON_SELECTION ')

RETAVL = 'Y'

SALARY = 0

l_person_id = get_context(PERSON_ID, -1)

l_asg_id = get_context(HR_ASSIGNMENT_ID, -1)

l_date = get_context(EFFECTIVE_DATE, '1900/01/01' (date))

l_temp = ESS_LOG_WRITE('person id ' + TO_CHAR(*l_person_id*))

l_temp = ESS_LOG_WRITE('Asg id ' + TO_CHAR(*l_asg_id*))

l_temp = ESS_LOG_WRITE('Eff Date ' + TO_CHAR(*l_date*))

/* Loop through the person assignment */

count = PER_HIST_ASG_ASSIGNMENT_ID.count

l_temp = ESS_LOG_WRITE('Count of asg ' + TO_CHAR(*count*))

index = PER_HIST_ASG_ASSIGNMENT_ID.FIRST(-1)

While (PER_HIST_ASG_ASSIGNMENT_ID.EXISTS(*index*))

Loop (

l_asg = PER_HIST_ASG_ASSIGNMENT_ID[*index*]

l_s_date = PER_HIST_ASG_EFFECTIVE_START_DATE[*index*]

l_e_date = PER_HIST_ASG_EFFECTIVE_END_DATE[*index*]

l_type = PER_HIST_ASG_STATUS_TYPE[*index*]

l_change = PER_HIST_ASG_EFFECTIVE_LATEST_CHANGE[*index*]

l_temp = ESS_LOG_WRITE('l_asg ' + TO_CHAR(*l_asg*))

l_temp = ESS_LOG_WRITE('l_s_date ' + TO_CHAR(*l_s_date*))

l_temp = ESS_LOG_WRITE('l_e_date ' + TO_CHAR(*l_e_date*))

l_temp = ESS_LOG_WRITE('l_type ' + *l_type*)

l_temp = ESS_LOG_WRITE('l_change ' + *l_change*)

/* Validation */

```

IF l_date >= l_s_date AND l_date <= l_e_date THEN

(
  IF l_type = 'ACTIVE' and l_change = 'Y' then
  (
    l_temp = ESS_LOG_WRITE('setting context for ' + TO_CHAR(l_asg) )
    CHANGE_CONTEXTS(HR_ASSIGNMENT_ID = l_asg)(
      SALARY = SALARY + CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
      l_temp = ESS_LOG_WRITE('SALARY ' + TO_CHAR(SALARY) )
    )
  )
)

index = PER_HIST_ASG_ASSIGNMENT_ID.NEXT(index, -1)
)
IF SALARY > 5000 then
(
  RETVAL = 'Y'
  l_temp = ESS_LOG_WRITE('FINAL SALARY ' + TO_CHAR(SALARY) )
)
l_temp = ESS_LOG_WRITE('Returning TCS_PERSON_SELECTION ' || RETAVL)
Return RETAVL

```

YouTube Tutorial on Array Variables and Input Values

/*****

FF : TCS_ARRAY_ITEM_FF

Type : Total Compensation Item

Developer: Tilak

Purpose: Find the salary of each person for every assignment and returns with asg id and start date

*****/

/* Declare DBI */

```

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT is 0

DEFAULT FOR CMP_ASSIGNMENT_SALARY_DATE_FROM is '1900/01/01' (date)

/* Default IV */

DEFAULT for CMP_IVR_ASSIGNMENT_ID is EMPTY_NUMBER_NUMBER

/* Default Variables */

DEFAULT FOR COMPENSATION_DATES is EMPTY_TEXT_NUMBER

DEFAULT FOR VALUES is EMPTY_TEXT_NUMBER

DEFAULT FOR ASSIGNMENTS is EMPTY_TEXT_NUMBER

/* Declare IV */

INPUTS ARE  CMP_IV_PERIOD_START_DATE (DATE),

            CMP_IV_PERIOD_END_DATE (DATE),

            CMP_IVR_ASSIGNMENT_ID (NUMBER_NUMBER) ,

            CMP_IVR_TERM_ID (NUMBER_NUMBER),

            CMP_IVR_PAY_ASSIGNMENT_ID (NUMBER_NUMBER),

            CMP_IVR_PAY_TERM_ID (NUMBER_NUMBER),

            CMP_IVR_PAY_RELATION_ID (NUMBER_NUMBER)

/* Main */

l_temp = ESS_LOG_WRITE( 'ENTERING TCS_ARRAY_ITEM_FF ' )

l_count = CMP_IVR_ASSIGNMENT_ID.count

l_temp = ESS_LOG_WRITE( 'count asg ' + TO_char(l_count) )

counter = 1

index = CMP_IVR_ASSIGNMENT_ID.FIRST(-1)

WHILE (CMP_IVR_ASSIGNMENT_ID.EXISTS(index) )

LOOP

(

    l_asg = CMP_IVR_ASSIGNMENT_ID[index]

    l_temp = ESS_LOG_WRITE( 'asg ' + TO_char(l_asg) )

    CHANGE_CONTEXTS(HR_ASSIGNMENT_ID = l_asg)

    (

        salary = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT

```

```

fromDate = CMP_ASSIGNMENT_SALARY_DATE_FROM

COMPENSATION_DATES[counter] = to_char(fromDate, 'YYYY/MM/DD')

VALUES[counter] = to_char(salary)

ASSIGNMENTS[counter] = to_char(l_asg)

counter = counter + 1

l_temp = ESS_LOG_WRITE( 'salary ' + TO_char(salary) )

l_temp = ESS_LOG_WRITE( 'fromDate ' + TO_char(fromDate) )

l_temp = ESS_LOG_WRITE( 'l_asg ' + TO_char(l_asg) )

)

index = CMP_IVR_ASSIGNMENT_ID.NEXT(index, -1)

)

l_temp = ESS_LOG_WRITE( 'Leaving TCS_ARRAY_ITEM_FF ' )

RETURN COMPENSATION_DATES,VALUES,ASSIGNMENTS

-----*****-----

```

Fast Formula - Functions

Fast Formula and Functions

Fast Formula allows executing a PL/SQL Function through Fast Formula function definitions.

Fast Formula function is an interface between Fast Formula engine and the PL/SQL functions. Each product that uses Fast Formula has seeded various Functions that can be used in the Fast Formula. Fast Formula Functions is one of the most used features in EBS, especially in Oracle Benefit and Oracle Compensation. In EBS, the developer can write any function in PL/SQL and define that as Fast Formula Function and this function can be executed in the application via the Fast Formula. EBS has even provided an UI to define the Fast Formula Functions.

But in Fusion, unless it is an onsite implementation, the user does not have access to the database.

Therefore, it is not possible for the user to create PL/SQL Function.

Since there is no option to define the Fast Formula Function, we are not going into the details of defining the Fast Formula Function. If your implementation is onsite, you can follow the EBS documents. Only difference from the EBS days is that the Fast Formula is executed in a different schema. So, you need to create synonyms and provide proper grants to execute the functions.

In this section, we will explain the different components of the Fast Formula Functions and how it is used in Fast Formula from a user point of view.

Components of Fast Formula:

- 1) PL/SQL Function
- 2) Return Type
- 3) Context
- 4) Parameters

PL/SQL Function: Fast Formula does not support any PL/SQL function that uses an array or any other collection as a parameter or return value. The name of the PL/SQL Function does not have to necessarily match the Fast Formula Function names. Function overloads are allowed.

Return Type: Return Type should be one of the simple data types – Text, Number or Date. The return type must match with what is returned by the PL/SQL Function.

Context: This is a hidden part of the Fast Formula Function. Some parameter values in the PL/SQL Function are passed by the Fast Formula engine from the Context values. This way the user does not have to worry about these parameters. For example, to get the currency conversion rate, we use the Function GET_RATE and we pass parameters: **From Currency, To Currency, Conversion Rate Type** and **Default value**. We do not pass the date when the currency conversion is extracted and it is passed from the EFFECTIVE_DATE Context. You might have noticed that we never pass the person id or the assignment id or the effective date to the Fast Formula Functions because these values are passed from the Context behind the scenes. If the Context is not available in your Fast Formula Type then you will not be able to use the Function in your Fast Formula.

Parameter: Parameter needs to be one of the basic types (Text, Date, or Number). It can be optional or required and it can be IN or OUT or Both. The Fast Formula developer needs to pass the values to these parameters. It is always passed in sequential order. I have never tried the Named Parameter. When I try this option, I will update the section. Usually the optional parameters are defined at the end and therefore, you do not need to worry about the Named Parameter. Sometimes they define the same function with different parameters for overloading. For example, TO_CHAR is an overloaded function with different parameter type. This is for date and number.

Example:

```
L_DATA = ESS_LOG_WRITE( 'BEGIN MY_FIRST_CWB_DEFAULT_FF' )
```

Ess_Log_write is a Fast Formula Function with 1 parameter and returns a Text Value.

```
L_date = ADD_MONTHS(L_PL_START_DATE, 1)
```

This is another Fast Formula Function with 2 parameters, 1 with Date, another with Number and returns a Date value.

We have tried to list all the available Fusion Functions below. You can always approach the Oracle team for further explanation. Some functions may be obsolete or not available in your release. Please check with the documentation or with the Oracle support.

Using Global Variables in Fast Formula -WSA

Fast Formula – WSA

In this session, we will explain how to talk to another Fast Formula that is executed in the same session. In many processes, we define more than one Fast Formula and sometimes these Fast Formulas execute the same routine again and again. This can be avoided if we could talk to each other or if we can pass a value between Fast Formulas.

We will explain this concept with a hypothetical use case.

Assume Total Compensation Statement needs to calculate the Annual FTE (Full Time Equivalent) Salary and Bonus for every person. The Bonus is 20% of the Annual FTE Salary.

In the above example, the Annual FTE needs to be calculated in the Salary Fast Formula and the Bonus Fast Formula. Instead of doing the calculation in every Fast Formula, we want to calculate the Annual FTE salary in the Fast Formula that is executed first and pass the value to the Fast Formula that is executed later.

As per the setup, we assume, the Annual FTE Salary Fast Formula is executed first and the Bonus Fast Formula is executed later. To find out which process is executed first, you can run the process for a person and the Fast Formula ESS Logs can help you.

How we achieved this requirement:

1. Steps for FTE Salary Fast Formula:
2. We get the Annual salary from the Annual Salary DBI.
3. Get the FTE value from the FTE DBI.
4. If the DBI is not null or 1, then divide the Annual salary with FTE

5. If the Salary is defined for Hourly, then multiply the salary with FTE.
6. Set the value in Working Storage Area using the function WSA_SET(<item>, <value>).

Steps for Bonus Fast Formula:

1. Check the salary WSA Item exists by using WSA_EXISTS(<item>).
2. If the WSA exists, get the value from WSA by using WSA_GET(<item>, <default-value>).
3. Otherwise, calculate the value from the DBI.
4. If the WSA exists, delete the WSA by using WSA_DELETE(<item>) so that we make sure the WSA value is not used for the next person.

What is WSA:

As per the Fast Formula documentation,

“The working storage area is a mechanism for storing global values across formulas. The values are accessed by name. The names are case-independent”.

There are four working storage area methods:

WSA_EXISTS

WSA_DELETE

WSA_SET

WSA_GET

FTE Salary Fast Formula:

/*

Author : Ti(lak)shmi

Type : Total Compensation Item

NAME : TCS_FTE_SAL_TEST_FF

Remarks: This formula is for demonstrating the communication between Fast Formulas.

This formula calculates the Full time FTE salary and assigns it to a WSA Variable.

*/

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS -1

DEFAULT FOR PER_ASG_FTE_VALUE IS 1

INPUTS ARE CMP_IV_PERIOD_START_DATE (DATE) ,CMP_IV_PERIOD_END_DATE (DATE)

```
L_DATA = ESS_LOG_WRITE( 'BEGIN CS_FTE_SAL_TEST_FF' )
COMPENSATION_DATES = TO_CHAR(CMP_IV_PERIOD_END_DATE, 'YYYY/MM/DD')
VALUES = 0
IF (CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED ) THEN
(

VALUES = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
IF (PER_ASG_FTE_VALUE WAS NOT DEFAULTED) THEN
(
VALUES = VALUES * PER_ASG_FTE_VALUE
)

L_DATA = ESS_LOG_WRITE( 'Salary Amount is ' + TO_CHAR(VALUES) )
WSA_SET('TCS_WSA_TEST_FTE_SALARY', VALUES)

) ELSE (
L_DATA = ESS_LOG_WRITE( 'Salary Amount is 0 to test WSA ' )
WSA_SET('TCS_WSA_TEST_FTE_SALARY', VALUES)
)

L_DATA = ESS_LOG_WRITE( 'END CS_FTE_SAL_TEST_FF' )
RETURN COMPENSATION_DATES ,VALUE
```

Bonus Fast Formula:

/*

Author : Ti(lak)shmi

Type : Total Compensation Item

NAME : TCS_BONUS_TEST_FF

Remarks: This formula is for demonstrating the communication between Fast Formulas.

This formula gets the salary value that is calculated at TCS_FTE_SAL_TEST_FF by using WSA.

*/

```

INPUTS ARE CMP_IV_PERIOD_START_DATE (DATE) ,CMP_IV_PERIOD_END_DATE (DATE)
L_DATA = ESS_LOG_WRITE( 'BEGIN TCS_BONUS_TEST_FF' )
COMPENSATION_DATES = CMP_IV_PERIOD_END_DATE
VALUES = 0
IF WSA_EXISTS('TCS_WSA_TEST_FTE_SALARY') THEN
(
L_DATA = ESS_LOG_WRITE( 'WSA FOUND' )
VALUES = WSA_GET('TCS_WSA_TEST_FTE_SALARY', 0 )
L_DATA = ESS_LOG_WRITE( 'WSA VALUE ' + TO_CHAR(VALUES))
IF (VALUES != 0 ) THEN
(
VALUES = VALUES * .20
L_DATA = ESS_LOG_WRITE( 'Bonus Amount ' + TO_CHAR(VALUES))
)

WSA_DELETE('TCS_WSA_TEST_FTE_SALARY')
)
L_DATA = ESS_LOG_WRITE( 'END TCS_BONUS_TEST_FF' )
RETURN COMPENSATION_DATES ,VALUE

```

TCS Setups:

Item:

Category Setup

Statement Setup

Sample Log for the Formula Execution

```
BEGIN CS_FTE_SAL_TEST_FF
Salary Amount is 54963
END CS_FTE_SAL_TEST_FF
BEGIN TCS_BONUS_TEST_FF
WSA FOUND
WSA VALUE 54963
Bonus Amount 10992.6
END TCS_BONUS_TEST_FF
```

As per the log, Salary Fast Formula calculated the Salary and assigned the Salary value to a WSA Variable and the Bonus Fast Formula was able to get the value from the WSA Variable.

Hope this has helped you with an understanding of WSA and how communicating between Fast Formulas is possible. As always, if you have any questions or suggestions, please feel free to comment.

YouTube Tutorial No 17:

```
/*
Author : Tilak
Type : Compensation Person Selection
NAME : CMP_PERSON_SELECTION
Remarks: Eligible if the person annual salary is < 200000
and set the salary to WSA with person id
*/

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS -1
DEFAULT FOR PER_ASG_FTE_VALUE IS 1
INPUTS ARE CMP_IV_PERIOD_START_DATE (DATE) ,CMP_IV_PERIOD_END_DATE (DATE)
L_DATA = ESS_LOG_WRITE( 'Entering CMP_PERSON_SELECTION' )
VALUES = 0
retValue = 'Y'
personid = get_context(PERSON_ID, -1)
L_DATA = ESS_LOG_WRITE( 'person id ' + to_char( personid ) )
IF (CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED ) THEN
(
VALUES = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
IF (PER_ASG_FTE_VALUE WAS NOT DEFAULTED) THEN
(
```

```

VALUES = VALUES * PER_ASG_FTE_VALUE
)
)
L_DATA = ESS_LOG_WRITE( 'Salary Amount is ' + TO_CHAR(VALUEs) )
WSA_SET('TCS_WSA_TEST_FTE_SALARY', VALUEs)
WSA_SET('TCS_WSA_TEST_PERSON', personid)
if VALUEs > 200000 OR VALUEs <= 0 then (
    retValue = 'N'
)
L_DATA = ESS_LOG_WRITE( 'Leaving CMP_PERSON_SELECTION ' + retValue )
RETURN retValue
/*
Author : Tilak
Type  : Participation and Rate Eligibility
NAME  : CMP_ELIGIBLE
Remarks: Eligible if the person annual salary is > 100000
get the salary from WSA and delete the WSA
*/
L_DATA = ESS_LOG_WRITE( 'Entering CMP_ELIGIBLE ' )
VALUEs = 0
ELIGIBLE = 'Y'
personid = get_context(PERSON_ID, -1)
L_DATA = ESS_LOG_WRITE( 'person ' + TO_CHAR(personid) )
if WSA_EXISTS('TCS_WSA_TEST_FTE_SALARY') then (
    L_DATA = ESS_LOG_WRITE( ' WSA Exists' )
    lperson = WSA_GET('TCS_WSA_TEST_PERSON', -1)
    L_DATA = ESS_LOG_WRITE( 'WSA person ' + TO_CHAR(lperson) )
    if lperson = personid then (
        VALUEs = WSA_GET('TCS_WSA_TEST_FTE_SALARY', -1)
        L_DATA = ESS_LOG_WRITE( 'WSA salary ' + TO_CHAR(VALUEs) )
        if VALUEs > 100000 then (
            ELIGIBLE = 'N'
        )
    )
    WSA_DELETE('TCS_WSA_TEST_FTE_SALARY')
    WSA_DELETE('TCS_WSA_TEST_PERSON')
)
)
ELSE (
    L_DATA = ESS_LOG_WRITE( 'WSA NOT EXIST CMP_ELIGIBLE ' )
    ELIGIBLE = 'N'
)
L_DATA = ESS_LOG_WRITE( 'Leaving CMP_ELIGIBLE ' + ELIGIBLE )

```

Fast Formula – Absence

Fast Formula – Absence

In our previous sections, we went over the basics of Fast Formula and how Compensation uses the Formula in their products.

In this session, we will cover the Fast Formula usages in Absences.

Since we covered the Types, Input Values, Context and how to write a Fast Formula in detail in our previous sections, we will not get into that again in these sections.

If you have any questions or doubts, please post us a message.

Absence Formula Types:

Formula Type Name	Description	Usage
Global Absence Accrual	This formula type can be used to define the bands that identify how much paid time eligible workers accrue over the course of an accrual period. The criteria may be years of service, grades, hours worked etc. However, use this formula only if using the accrual matrix option and its corresponding formulas does not meet the requirement.	Manage Absence Plans -> In the Plan Type field, select Accrual -> Accruals tab Accrual Attributes section, select Formula
Global Absence Accrual Event	The Global Absence Accrual Event fast formula can be used to capture information about events that occur during a calendar year which would cause a change in the accrual band that the worker belongs to. This formula can capture such dates and return to the accrual matrix formula which would automatically fetch the respective band values as of each of the dates fed into the accrual matrix formula	You cannot attach this formula to the plan definition at any point. When you create the formula, the formula name needs to be the same as the absence plan name. This automatically links the formula to the plan.
Global Absence Accrual Matrix	The Global Absence Accrual Matrix fast formula can be used in conjunction with the accrual matrix to implement requirements such as band change pro-ration, FTE pro-ration etc.	Manage Absence Plans -> Plan Type field, select Accrual -> Accrual Matrix section -> the defined formula in the Accrual Formula column
Global Absence Band Entitlement	The Global Absence Band Entitlement fast formula can be used to define the bands of entitlement duration and percentage of payment that is applicable	Manage Absence Plans -> Plan Type field, select Qualification -> Qualification Band Matrix table -> Qualification Details -> Entitlement Formula

	against a qualification plan entitlement.	
Global Absence Carryover	The Global Absence Carryover fast formula can be used in cases where a single carryover rule does not apply to the entire population that belongs to the accrual plan.	Manage Absence Plans -> Plan Type field, select Accrual -> in Accrual Tab, Plan Limits -> Carryover Rule
Global Absence Carryover Proration	The Global Absence Carryover Proration fast formula can be used in cases where a pro-ration factor (or a multiplication factor) needs to be applied onto the maximum carryover limit.	Manage Absence Plans -> Plan Type field, select Accrual -> in Accrual Tab, Plan Limits -> Carryover Proration Formula
Global Absence Ceiling	The Global Absence Ceiling fast formula can be used in cases where a single ceiling rule does not apply to the entire population that belongs to the accrual plan.	Manage Absence Plans -> Plan Type field, select Accrual -> in Accrual Tab, Plan Limits -> Ceiling Formula
Global Absence Ceiling Proration	The Global Absence Ceiling Proration fast formula can be used in cases where a pro-ration factor (or a multiplication factor) needs to be applied onto the maximum ceiling limit.	Manage Absence Plans -> Plan Type field, select Accrual -> in Accrual Tab, Plan Limits -> Ceiling Proration Formula
Global Absence Entry Validation	The Global Absence Entry Validation fast formula can be used in cases where a custom data validation or business rule validation needs to be performed when an absence entry is being made	Manage Absence type -> Absence Type page Type Attributes tab, General Attributes section -> the Validation Formula
Global Absence Partial Period Accrual Rate	The Global Absence Partial Period Accrual Rate fast formula is where any logic required for pro-ration of accrual balance during enrollment year and un-enrollment year needs to be entered.	Manage Absence Plans -> Plan Type field, select Accrual -> in Accrual Tab, Plan Limits -> Partial Accrual Period Formula
Global Absence Plan Duration	The Global Absence Plan Duration fast formula can be used to over-ride the default duration calculation logic for daily accrual duration entries against accrual plans. For example, if the accrual deduction to be considered for an absence entry in an accrual plan in an organization depends on the location of the worker, then this formula can be leveraged to specify this dynamic calculation logic. This formula will be invoked once for each day of absence.	Manage Absence Plans -> Plan Type field, select Accrual -> in Accrual Tab, Plan Limits -> Conversion Formula
Global Absence Plan Enrollment End	The Global Absence Plan Enrollment End fast formula can be used to over-ride the default enrollment end date rule for the absence plan when workers are being terminated from the organization or when the Update Accrual Plan Enrollments batch job is being run.	Manage Absence Plans -> Plan Type field, select Accrual -> in Accrual Tab, Plan Limits -> End Date Formula

Global Absence Plan Enrollment Start	The Global Absence Plan Enrollment Start fast formula can be used to over-ride the default enrollment start date rule for the absence plan when workers are being hired into the organization or when the Update Accrual Plan Enrollments batch job is being run.	Manage Absence Plans -> Plan Type field, select Accrual -> in Accrual Tab, Plan Limits -> Start Date Formula
Global Absence Plan Entitlement	The Global Absence Plan Entitlement fast formula can be used to define the entire entitlement structure for a qualification plan for cases where matrix architecture does not fit the bill.	Manage Absence Plans -> Plan page Entitlements tab , Entitlement Attributes section -> Entitlement Formula
Global Absence Plan Period Anniversary Event Date	The Global Absence Plan Period Anniversary Event Date fast formula can be used to specify the date to be used as the calendar leave year start date for accrual plans whose term is of type Anniversary year.	Manage Absence Plans -> Plan Type field, select Accrual -> in Accrual Tab, Plan Limits -> Anniversary Event Formula
Global Absence Plan Roll Backward End	The Global Absence Plan Roll Backward End fast formula is to be used to determine the start date of a plan term that uses the Roll Backward term rule.	Manage Absence Plans -> Plan Type field, Qualification -> Plan page Plan Attributes tab, Plan Term section -> Start Formula
Global Absence Plan Use Rate	The Global Absence Plan Use Rate fast formula can be used to dynamically specify the rate definition associated with the qualification plan depending on custom conditions.	Manage Absence Plans -> Plan Type field, Qualification -> Plan page Entries and Balances tab -> Formula
Global Absence Proration	The Global Absence Proration fast formula can be used to apply a pro-rata factor (or a multiplication factor), onto the final accrual calculated and returned by the accrual calculation rules in an accrual based absence plan.	Manage Absence Plans -> Plan Type field, select Accrual ->Plan page Accruals tab, Accrual Attributes section -> Accrual Proration Formula
Global Absence Type Duration	The Global Absence Type Duration fast formula can be used to over-ride the default calculation logic that the application uses to determine the absence entry duration and accrual plan usage against absence entries.	Manage Absence Type -> Type Attributes tab, General Attributes section -> Conversion Formula
Global Absence Vesting Period	The Global Absence Vesting Period fast formula can be used to enforce a custom vesting period (a period during which the Worker is enrolled into the plan and accruing balance but cannot use them) logic while defining an absence plan.	Manage Absence Plans -> Plan Type field, select Accrual ->Plan page Accruals tab, Accrual Attributes section -> Vesting Period Formula

Absence Formula Type's Contexts:

Formula Type Name	Context Name
Global Absence Accrual	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID

	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Accrual Event	
	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE

Global Absence Accrual Matrix	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Band Entitlement	ABSENCE_CATEGORY_ID
	ABSENCE_ENTRY_ID
	ABSENCE_MATERNITY_ID
	ABSENCE_REASON_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID

	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Carryover	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Carryover Proration	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID

	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Ceiling	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Ceiling Proration	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID

	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Entitlement End Date	ABSENCE_CATEGORY_ID
	ABSENCE_ENTRY_ID
	ABSENCE_MATERNITY_ID
	ABSENCE_REASON_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Entry Validation	ABSENCE_ENTRY_ID
	ABSENCE_REASON_ID
	ABSENCE_TYPE_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID

	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Linkage Exclusion	ABSENCE_ENTRY_ID
	ABSENCE_REASON_ID
	ABSENCE_TYPE_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Partial Period Accrual Rate	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE

	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Plan Duration	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Plan Enrollment End	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID

	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Plan Enrollment Start	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Plan Entitlement	ABSENCE_CATEGORY_ID

	ABSENCE_ENTRY_ID
	ABSENCE_MATERNITY_ID
	ABSENCE_REASON_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Plan Period Anniversary Event Date	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID

	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Plan Roll Backward End	ABSENCE_CATEGORY_ID
	ABSENCE_ENTRY_ID
	ABSENCE_MATERNITY_ID
	ABSENCE_REASON_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Plan Roll Forward Start	ABSENCE_CATEGORY_ID
	ABSENCE_ENTRY_ID
	ABSENCE_MATERNITY_ID
	ABSENCE_REASON_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID

	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Plan Use Rate	ABSENCE_CATEGORY_ID
	ABSENCE_ENTRY_ID
	ABSENCE_MATERNITY_ID
	ABSENCE_REASON_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Proration	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID
	DATE_EARNED

	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Type Duration	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE
Global Absence Vesting Period	ABSENCE_ENTRY_ID
	ABSENCE_TYPE_ID
	ACCRUAL_PLAN_ID

	DATE_EARNED
	EFFECTIVE_DATE
	END_DATE
	ENTERPRISE_ID
	HR_ASSIGNMENT_ID
	HR_RELATIONSHIP_ID
	HR_TERM_ID
	JOB_ID
	LEGAL_EMPLOYER_ID
	LEGISLATIVE_DATA_GROUP_ID
	ORGANIZATION_ID
	PAYROLL_ASSIGNMENT_ID
	PAYROLL_ID
	PAYROLL_RELATIONSHIP_ID
	PAYROLL_TERM_ID
	PERSON_ID
	START_DATE

Absence Formula Type Input values:

Formula Type Name	Input variables	Type	Description
Global Absence Accrual	IV_ACCRUALPERIODSTARTDATE	Date	Start date of accrual period
	IV_ACCRUALPERIODENDDATE	Date	End date of accrual period
	IV_CALENDARSTARTDATE	Date	Start date of accrual calendar year
	IV_CALEDARENDDATE	Date	End date of accrual calendar year
	IV_PLANENROLLMENTSTARTDATE	Date	Start date of enrollment
	IV_PLANENROLLMENTENDDATE	Date	End date of enrollment
Global Absence Accrual Event	IV_CARRYOVER	Date	Carryover limit received from accrual matrix
	IV_CEILING	Date	Ceiling limit received from the accrual matrix
	IV_ACCRUALPERIODSTARTDATE	Date	Start date of accrual period
	IV_ACCRUALPERIODENDDATE	Date	End date of accrual period
	IV_CALENDARSTARTDATE	Date	Start date of accrual calendar year

	IV_CALEDARENDDATE	Date	End date of accrual calendar year
	IV_PLANENROLLMENTSTARTDATE	Date	Start date of enrollment
	IV_PLANENROLLMENTENDDATE	Date	End date of enrollment
	IV_BAND_CHG_DT1 .. 5	Date	If the accrual matrix band definition consists of grades or derived factors length of service or age, then this input value fetches the date on which worker has had a change in any of these factors within the period.
	IV_BAND_CHG_BEFVAL1 .. 5	Number	Accrual value as per the matrix before IV_BAND_CHG_DT1
	IV_BAND_CHG_AFTVAL1 .. 5	Number	Accrual value as per the matrix after IV_BAND_CHG_DT1
Global Absence Accrual Matrix	IV_ACCRUAL	Number	Accrual value received from the accrual matrix
	IV_CARRYOVER	Number	Carryover limit received from accrual matrix
	IV_CEILING	Number	Ceiling limit received from the accrual matrix
	IV_ACCRUALPERIODSTARTDATE	Date	Start date of accrual period
	IV_ACCRUALPERIODENDDATE	Date	End date of accrual period
	IV_CALENDARSTARTDATE	Date	Start date of accrual calendar year
	IV_CALEDARENDDATE	Date	End date of accrual calendar year
	IV_PLANENROLLMENTSTARTDATE	Date	Start date of enrollment
	IV_PLANENROLLMENTENDDATE	Date	End date of enrollment

	IV_BAND_CHG_DT1 .. 5	Date	If the accrual matrix band definition consists of grades or derived factors length of service or age, then this input value fetches the date on which worker has had a change in any of these factors within the period.
	IV_BAND_CHG_BEFVAL1 .. 5	Number	Accrual value as per the matrix before IV_BAND_CHG_DT1
	IV_BAND_CHG_AFTVAL1 .. 5	Number	Accrual value as per the matrix after IV_BAND_CHG_DT1
	IV_EVENT_DATES	Date_Number	Array of dates returned by the Accrual Event formula.
	IV_ACCRUAL_VALUES	Number_Number	Array of accrual values fetched from the accrual matrix as of the dates contained in the accrual event dates array.
Global Absence Band Entitlement	IV_START_DATE	Date	Start date of absence entry
	IV_END_DATE	Date	End date of absence entry
	IV_TOTALDURATION	Number	Duration of absence entry
	IV_START_DURATION	Number	Start date duration of the absence entry. Applicable when absence entry is being recorded against an Elapsed based work schedule
	IV_END_DURATION	Number	End date duration of the absence entry. Applicable when absence entry is being recorded against an Elapsed

			based work schedule
	IV_START_TIME	Text	Start time on the start date of the absence entry. Applicable when absence entry is being recorded against a Time based work schedule. For elapsed work schedules it will be 00:00.
	IV_END_TIME	Text	End time on the end date for the absence being entered. Applicable when absence entry is being recorded against a Time based work schedule. For elapsed work schedules it will be 23:59.
	IV_ACTUALCHILDBIRTHDATE	Date	Actual date of childbirth entered for the absence. Applicable for Childbirth/placement absence pattern
	IV_ACTUALSTARTDATE	Date	Actual start date of absence. Applicable for Childbirth/placement absence pattern
	IV_ACTUALENDDATE	Date	Actual end date of absence. Applicable for Childbirth/placement absence pattern
	IV_EXPECTEDCHILDBIRTHDATE	Date	Expected date of childbirth entered for the absence. Applicable for Childbirth/placement absence pattern
	IV_PLANNEDSTARTDATE	Date	Expected start date of absence. Applicable for Childbirth/placement absence pattern

	IV_PLANNEDENDDATE	Date	Expected end date of absence. Applicable for Childbirth/placement absence pattern
	IV_ABSENCE_REASON	Text	Absence reason in the language of the current session
Global Absence Carryover	IV_ACCRUAL	Number	Accrual value received from the accrual matrix
	IV_CARRYOVER	Number	Carryover limit received from accrual matrix
	IV_CEILING	Number	Ceiling limit received from the accrual matrix
	IV_ACCRUALPERIODSTARTDATE	Date	Start date of accrual period
	IV_ACCRUALPERIODENDDATE	Date	End date of accrual period
	IV_CALENDARSTARTDATE	Date	Start date of accrual calendar year
	IV_CALEDARENDDATE	Date	End date of accrual calendar year
	IV_PLANENROLLMENTSTARTDATE	Date	Start date of enrollment
	IV_PLANENROLLMENTENDDATE	Date	End date of enrollment
Global Absence Carryover Proration	IV_ACCRUAL	Number	Accrual value received from the accrual matrix
	IV_CARRYOVER	Number	Carryover limit received from accrual matrix
	IV_CEILING	Number	Ceiling limit received from the accrual matrix
	IV_ACCRUALPERIODSTARTDATE	Date	Start date of accrual period
	IV_ACCRUALPERIODENDDATE	Date	End date of accrual period
	IV_CALENDARSTARTDATE	Date	Start date of accrual calendar year
	IV_CALEDARENDDATE	Date	End date of accrual calendar year
	IV_PLANENROLLMENTSTARTDATE	Date	Start date of enrollment

	IV_PLANENROLLMENTENDDATE	Date	End date of enrollment
Global Absence Ceiling	IV_ACCRUAL	Number	Accrual value received from the accrual matrix
	IV_CARRYOVER	Number	Carryover limit received from accrual matrix
	IV_CEILING	Number	Ceiling limit received from the accrual matrix
	IV_ACCRUALPERIODSTARTDATE	Date	Start date of accrual period
	IV_ACCRUALPERIODENDDATE	Date	End date of accrual period
	IV_CALENDARSTARTDATE	Date	Start date of accrual calendar year
	IV_CALEDARENDDATE	Date	End date of accrual calendar year
	IV_PLANENROLLMENTSTARTDATE	Date	Start date of enrollment
	IV_PLANENROLLMENTENDDATE	Date	End date of enrollment
Global Absence Ceiling Proration	IV_ACCRUAL	Number	Accrual value received from the accrual matrix
	IV_CARRYOVER	Number	Carryover limit received from accrual matrix
	IV_CEILING	Number	Ceiling limit received from the accrual matrix
	IV_ACCRUALPERIODSTARTDATE	Date	Start date of accrual period
	IV_ACCRUALPERIODENDDATE	Date	End date of accrual period
	IV_CALENDARSTARTDATE	Date	Start date of accrual calendar year
	IV_CALEDARENDDATE	Date	End date of accrual calendar year
	IV_PLANENROLLMENTSTARTDATE	Date	Start date of enrollment
	IV_PLANENROLLMENTENDDATE	Date	End date of enrollment
Global Absence Entitlement End Date			
Global Absence Entry Validation	IV_START_DATE	Date	Start date of absence entry
	IV_END_DATE	Date	End date of absence entry

	IV_TOTALDURATION	Number	Duration of absence entry
	IV_START_DURATION	Number	Start date duration of the absence entry. Applicable when absence entry is being recorded against an Elapsed based work schedule
	IV_END_DURATION	Number	End date duration of the absence entry. Applicable when absence entry is being recorded against an Elapsed based work schedule
	IV_START_TIME	Text	Start time on the start date of the absence entry. Applicable when absence entry is being recorded against a Time based work schedule. For elapsed work schedules it will be 00:00.
	IV_END_TIME	Text	End time on the end date for the absence being entered. Applicable when absence entry is being recorded against a Time based work schedule. For elapsed work schedules it will be 23:59.
	IV_ACTUALCHILDBIRTHDATE	Date	Actual date of childbirth entered for the absence. Applicable for Childbirth/placement absence pattern
	IV_ACTUALSTARTDATE	Date	Actual start date of absence. Applicable for Childbirth/placement absence pattern

	IV_ACTUALENDDATE	Date	Actual end date of absence. Applicable for Childbirth/placement absence pattern
	IV_EXPECTEDCHILDBIRTHDATE	Date	Expected date of childbirth entered for the absence. Applicable for Childbirth/placement absence pattern
	IV_PLANNEDSTARTDATE	Date	Expected start date of absence. Applicable for Childbirth/placement absence pattern
	IV_PLANNEDENDDATE	Date	Expected end date of absence. Applicable for Childbirth/placement absence pattern
	IV_ABSENCE_REASON	Text	Absence reason in the language of the current session
Global Absence Linkage Exclusion			
Global Absence Partial Period Accrual Rate	IV_ACCRUAL	Number	Accrual value received from the accrual matrix
	IV_CARRYOVER	Number	Carryover limit received from accrual matrix
	IV_CEILING	Number	Ceiling limit received from the accrual matrix
	IV_ACCRUALPERIODSTARTDATE	Date	Start date of accrual period
	IV_ACCRUALPERIODENDDATE	Date	End date of accrual period
	IV_CALENDARSTARTDATE	Date	Start date of accrual calendar year
	IV_CALEDARENDDATE	Date	End date of accrual calendar year
	IV_PLANENROLLMENTSTARTDATE	Date	Start date of enrollment
	IV_PLANENROLLMENTENDDATE	Date	End date of enrollment

	IV_BAND_CHG_DT1 .. 5	Date	If the accrual matrix band definition consists of grades or derived factors length of service or age, then this input value fetches the date on which worker has had a change in any of these factors within the period.
	IV_BAND_CHG_BEFVAL1 .. 5	Number	Accrual value as per the matrix before IV_BAND_CHG_DT1
	IV_BAND_CHG_AFTVAL1 .. 5	Number	Accrual value as per the matrix after IV_BAND_CHG_DT1
Global Absence Plan Duration	IV_START_DATE	Date	This is the start date of the absence being entered
	IV_END_DATE	Date	This is the end date of the absence being entered
	IV_START_DURATION	Number	This is the start date duration for the absence being entered. This is applicable when absence entry is being recorded against an Elapsed based work schedule
	IV_END_DURATION	Number	This is the end date duration for the absence being entered. This is applicable when absence entry is being recorded against an Elapsed based work schedule

	IV_START_TIME	Text	This is the start time on the start date for the absence being entered. This is applicable when absence entry is being recorded against a Time based work schedule. For elapsed work schedules it will be 00:00.
	IV_END_TIME	Text	This is the end time on the end date for the absence being entered. This is applicable when absence entry is being recorded against a Time based work schedule. For elapsed work schedules it will be 23:59.
	IV_UOM	Text	Unit of measure of the current absence entry
Global Absence Plan Enrollment End			
Global Absence Plan Enrollment Start			
Global Absence Plan Entitlement			
Global Absence Plan Period Anniversary Event Date			
Global Absence Plan Roll Backward End	IV_START_DATE	Date	Start date of absence entry
	IV_END_DATE	Date	End date of absence entry
	IV_TOTALDURATION	Number	Duration of absence entry
	IV_START_DURATION	Number	Start date duration of the absence entry. Applicable when absence entry is being recorded against an Elapsed based work schedule

	IV_END_DURATION	Number	End date duration of the absence entry. Applicable when absence entry is being recorded against an Elapsed based work schedule
	IV_START_TIME	Text	Start time on the start date of the absence entry. Applicable when absence entry is being recorded against a Time based work schedule. For elapsed work schedules it will be 00:00.
	IV_END_TIME	Text	End time on the end date for the absence being entered. Applicable when absence entry is being recorded against a Time based work schedule. For elapsed work schedules it will be 23:59.
	IV_ACTUALCHILDBIRTHDATE	Date	Actual date of childbirth entered for the absence. Applicable for Childbirth/placement absence pattern
	IV_ACTUALSTARTDATE	Date	Actual start date of absence. Applicable for Childbirth/placement absence pattern
	IV_ACTUALENDDATE	Date	Actual end date of absence. Applicable for Childbirth/placement absence pattern
	IV_EXPECTEDCHILDBIRTHDATE	Date	Expected date of childbirth entered for the absence. Applicable for Childbirth/placement absence pattern

	IV_PLANNEDSTARTDATE	Date	Expected start date of absence. Applicable for Childbirth/placement absence pattern
	IV_PLANNEDENDDATE	Date	Expected end date of absence. Applicable for Childbirth/placement absence pattern
	IV_ABSENCE_REASON	Text	Absence reason in the language of the current session
Global Absence Plan Roll Forward Start			
Global Absence Plan Use Rate			
Global Absence Proration	IV_ACCRUAL	Number	Accrual value received from the accrual matrix
	IV_CARRYOVER	Number	Carryover limit received from accrual matrix
	IV_CEILING	Number	Ceiling limit received from the accrual matrix
	IV_ACCRUALPERIODSTARTDATE	Date	Start date of accrual period
	IV_ACCRUALPERIODENDDATE	Date	End date of accrual period
	IV_CALEDARSTARTDATE	Date	Start date of accrual calendar year
	IV_CALEDARENDDATE	Date	End date of accrual calendar year
	IV_PLANENROLLMENTSTARTDATE	Date	Start date of enrollment
	IV_PLANENROLLMENTENDDATE	Date	End date of enrollment
Global Absence Type Duration	IV_START_DATE	Date	Start date of absence entry
	IV_END_DATE	Date	End date of absence entry
	IV_START_DURATION	Number	Start date duration of the absence entry. Applicable when absence entry is being recorded

			against an Elapsed based work schedule
	IV_END_DURATION	Number	End date duration of the absence entry. Applicable when absence entry is being recorded against an Elapsed based work schedule
	IV_START_TIME	Text	Start time on the start date of the absence entry. Applicable when absence entry is being recorded against a Time based work schedule. For elapsed work schedules it will be 00:00.
	IV_END_TIME	Text	End time on the end date for the absence being entered. Applicable when absence entry is being recorded against a Time based work schedule. For elapsed work schedules it will be 23:59.
	IV_UOM	Text	Unit of Measure
Global Absence Vesting Period	IV_ACCRUAL	Number	Accrual value received from the accrual matrix
	IV_CARRYOVER	Number	Carryover limit received from accrual matrix
	IV_CEILING	Number	Ceiling limit received from the accrual matrix
	IV_ACCRUALPERIODSTARTDATE	Date	Start date of accrual period
	IV_ACCRUALPERIODENDDATE	Date	End date of accrual period
	IV_CALENDARSTARTDATE	Date	Start date of accrual calendar year

	IV_CALEDARENDDATE	Date	End date of accrual calendar year
	IV_PLANENROLLMENTSTARTDATE	Date	Start date of enrollment
	IV_PLANENROLLMENTENDDATE	Date	End date of enrollment

Absence Formula Type's Return values and variables:

Formula Type Name	Return Variable Name	Return Type	Description
Global Absence Accrual	accrual	Number	Amount of accrual that the worker accrues in the specific period.
	ceiling	Number	Maximum time that a worker can accrue
	carryover	Number	Maximum unused time that a worker can transfer to the next accrual term
	prorationFactor	Number	Multiplying factor that the accrual is multiplied.
	vestingUnits	Number	Period for which time-off is accrued, but cannot be used by worker.
	vestingUOM	Text	Unit of measure for the vesting period duration. UoM can be Calendar Days ('C'), Weeks ('W'), Months ('M') or Years ('Y')
	carryOverProration	Number	Multiplication factor for carryover
	ceilingProration	Number	Multiplication factor for ceiling
	adjustmentvalues	Number_Number	Array of balance adjustments that when returned by the formula is applied against the accrual plan enrollment.
	adjustmentdates	Date_Number	Array of dates associated with the array of balance adjustment values to specify the dates on which each of the balance adjustments need to be applied
	adjustmenttypes	Text_Number	Array of adjustment reasons used for the balance adjustments

	absvalues	Number_Number	Array of absence accrual usages that can be applied against an accrual plan enrollment. Useful for cases where the absence entry itself is not captured in Global Absence Management, but rather via other modules (such as payroll element entries) or even third party systems and the absence deduction against the accrual plan needs to happen in Fusion. When using this, it should be noted that absence entries should not be made for the same dates.
	absdates	Date_Number	Array of dates associated with the absence accrual usages on which the deduction is applied.
Global Absence Accrual Event	IV_EVENT_DATES	Date_Number	Array of dates that feeds into the accrual matrix which can be used for band change pro-ration. Corresponding accrual values derived from the matrix also feeds into the matrix formula
Global Absence Accrual Matrix	accrual	Number	Amount of accrual that the worker accrues in the specific period.
	ceiling	Number	Maximum time that a worker can accrue
	carryover	Number	Maximum unused time that a worker can transfer to the next accrual term
	adjustmentvalues	Number_Number	Array of balance adjustments that when returned by the formula is applied against the accrual plan enrollment.

	adjustmentdates	Date_Number	Array of dates associated with the array of balance adjustment values to specify the dates on which each of the balance adjustments need to be applied
	adjustmenttypes	Text_Number	Array of adjustment reasons used for the balance adjustments
	absvalues	Number_Number	Array of absence accrual usages that can be applied against an accrual plan enrollment. Useful for cases where the absence entry itself is not captured in Global Absence Management, but rather via other modules (such as payroll element entries) or even third party systems and the absence deduction against the accrual plan needs to happen in Fusion. When using this, it should be noted that absence entries should not be made for the same dates.
	absdates	Date_Number	Array of dates associated with the absence accrual usages on which the deduction is applied.
Global Absence Band Entitlement	BANDPAYFACTOR	Number	Pay factor for entitlement band value
	BANDENTITLEMENT	Number	Entitlement duration that worker is eligible for
Global Absence Carryover	carryover	Number	Maximum unused time that a worker can transfer to the next accrual term
Global Absence Carryover Proration	prorationFactor	Number	Multiplication factor for carryover

Global Absence Ceiling	ceiling	Number	Maximum time that a worker can accrue
Global Absence Ceiling Proration	prorationFactor	Number	Multiplication factor for ceiling
Global Absence Entry Validation	VALID	Text	Determines validity of absence entry. 'N' throws an error message and stops the transaction from being submitted
	ERROR_MESSAGE	Text	Custom error message for invalid absence entry. Error message can either be written into the formula or defined via Manage Messages and invoked here.
Global Absence Partial Period Accrual Rate	accrual	Number	Partial period pro-rated accrual value returned for the particular partial repeating period. Value is returned gets added as an accrual line. For processing termination of enrollment for front loaded plans, the variable should return a negative value, to deduct pro-rated amount from the existing annual accrual value.
Global Absence Plan Duration	DURATION	Number	Duration of plan entries
Global Absence Plan Enrollment End	enrollmentEndDate	Date	End date of enrollment
Global Absence Plan Enrollment Start	enrollmentStartDate	Date	Start date of enrollment
Global Absence Plan Entitlement	BAND1PAYFACTOR	Number	Pay factor for the entitlement BAND1ENTITLEMENT

	BAND1ENTITLEMEN T	Number	Entitlement duration that the worker is eligible for
	BAND2PAYFACTOR	Number	Pay factor for the entitlement BAND2ENTITLEME NT
	BAND2ENTITLEMEN T	Number	Entitlement duration that the worker is eligible for
	BAND3PAYFACTOR	Number	Pay factor for the entitlement BAND3ENTITLEME NT
	BAND3ENTITLEMEN T	Number	Entitlement duration that the worker is eligible for
	BAND4PAYFACTOR	Number	Pay factor for the entitlement BAND4ENTITLEME NT
	BAND4ENTITLEMEN T	Number	Entitlement duration that the worker is eligible for
	BAND5PAYFACTOR	Number	Pay factor for the entitlement BAND5ENTITLEME NT
	BAND5ENTITLEMEN T	Number	Entitlement duration that the worker is eligible for
Global Absence Plan Period Anniversary Event Date	anniversaryDate	Date	Date used to determine the leave year start date for anniversary based accrual plan term
Global Absence Plan Roll Backward End	REFERENCEDATE	Date	Start date for the Rolling Backward plan term.
Global Absence Plan Use Rate	RATECODE	Text	Rate code for rate associated with qualification plan
Global Absence Proration	prorationFactor	Number	Multiplication factor used to pro-rate final accrual amount
Global Absence Type Duration	DURATION	Number	Duration of absence entry

Global Absence Vesting Period	vestingUOM	Text	Unit of measure for the vesting period duration. UoM can be Calendar Days ('C'), Weeks ('W'), Months ('M') or Years ('Y')
	vestingUnits	Number	Period for which time-off is accrued, but cannot be used by worker.

The above information is collected as of the latest release from Absence, this may not be true to your release, please contact Absence Support /Development or us for any questions or doubts.

Fast Formula – Without DBI

Fast Formula & Value Set

In EBS days, Fast Formula allows us to write SQL through their [Formula Functions](#). This feature allows accessing any table and columns through the Fast Formula. In Fusion, [Fast Formula functions](#) are limited to customers who have onsite implementation.

In Fusion, the only way to access the data is using DBI or Input Values. In a real life scenario, many times, it is hard to find the DBI we want. Sometimes the DBIs are not available, sometimes the Context are not available, even sometime, the DBI are available in an array format that may lead to performance issues.

For a long time, we were looking for a way where we can access the data without using DBI. Finally we found the Holy Grail.

Holy Grail of Fast Formula:

We know that we cannot access the database and therefore writing our own SQL or a Function is out of the question. So, we are looking for an option where SQL can be defined through a setup and that can be executed from Formula Function.

After some search , we found out exactly what we were looking for.

Value Set: Value Set is a setup where customer can define a table, id and value columns with **Where**, **Condition** and **Order By** clause. This setup allows us to define a simple **Select** statement from a table with condition.

This option, Value Set can be navigated as follows.

Select **Setup and Maintenance**

From the **Setup and Maintenance** Overview, search for **Manage Payroll Value Sets** as shown in the following screen.

From the search Result, click 'Go to Task' to navigate to the page where **Value Set** can be created as follows.

In the above example, the Value Set is defined as

select the **lookup** code and meaning from HR_LOOKUP for lookup type 'CMP_TCS_COLS_BNFTS'

The Value Set needs to have at a minimum, Table, ID and Value columns for using Fast Formula Function (ID is not a required column in the Value Set UI).

Formula Function: [GET VALUE SET](#):

This Formula Function, **Get_Value_Set** is delivered by Oracle Payroll. This function is defined with 4 parameters and returns a character value.

The parameters are:

P_VALUESET: This is a required parameter and is used for passing the Value Set name.

1.

P_BINDS: this is an optional parameter. If you have defined your Value Set with bind variable in **where clause**, you can pass a value to the bind variable with this parameter. The Formula Function uses a predefined format for passing the value.

2.

Predefined Format : The first character defines the delimiter and the second character defines the assignment character. From third character onwards, you can pass the bind variable name and value to the bind variable. We will go through the format in detail in our examples.

P_FILTER: This is an optional parameter. With this parameter, we can pass additional filter condition. The Filter value is validated against the **Value column Name** of the Value Set. The function uses **like** condition to validate the

value.

3.

P_ENABLE_CACHE: I don't think this parameter is used anywhere in Fast Formula, so we can ignore this parameter.

4.

Now, we will go through a few examples to understand how we can use this Function and Value Set in our Fast Formulas. For our example, we use the same set of data so that we can understand the different options with ease.

Data: We use the following lookup data for the type, CMP_TCS_COLS_BNFTS for our example.

Example 1

Value Set: The first Value Set is a very simple value set.

The Value Set needs to have at a minimum - **Table, ID and Value** columns. The Formula Function, Get_Value_Set returns **ID column value to the Fast Formula**. The following Value Set is defined for returning **Lookup Code** for the lookup type, CMP_TCS_COLS_BNFTS.

Fast Formula: The following Formula is defined for TCS item. The Formula calls the function, Get_Value_Set with our Value Set. And we log the return value in the ESS log.

The Fast Formula attached to the TCS Item as follows.

We are not going into the details of the TCS setup here. Assume the item is added to a Category and the Category is added to a Statement and the Statement is processed in ESS.

The following is the result of the ESS process and the result of the Get_Value_Set. Since there is no “**order by**” clause defined in the Value Set, we expect any one value from the Lookup type.

The result shows the first value of the column ‘Code’, **DESC** from our select statement.

Example 2

We changed the first example to bind the lookup type instead of hard coding to ‘CMP_TCS_COLS_BNFTS’.

Value Set: Bind variable are defined with the following format

:{PARAMETER.<BIND_NAME>}

Fast formula: As we explained earlier, the bind variable is passed to the function in a specific format.

In our example, the bind value is passed as '|=TYPE_CODE='CMP_TCS_COLS_BNFTS' '

The first character, | defines the delimiter.

1.

The second character, = defines the assignment character.

2.

TYPE_CODE is the bind variable name from the Set Value and = is the assignment character.

3.

Finally we pass the value of the bind variables, 'CMP_TCS_COLS_BNFTS'.

4.

In theory, you can use any delimiter and assignment character but with our experience, we would recommend using the same delimiter (|) and assignment character (=).

We expect the same result as in the previous example.

Example 3

In this example, we try to bind more variables to explain how more than one value is passed.

Value Set: With two bind variables, one is **equi** joint and other one is a **like** condition. Please note that the **like** condition is already defined within quotes.

Fast Formula: The value of both bindings is passed in a different way. The first binding is not defined with quotes and so we need to pass the value with a quote. The second binding is defined within quotes and so the value is passed without any quote.

As per our value set, we are expecting one of the contribution values, EEC_CNTRB or ERC_CNTRB.

Example 4 : In this example, we use the Value Set from Example 3. We pass the Value to the third variable, P_FILTER.

This parameter value is validated against the **Value Column Name** of the Value Set in 'like' condition.

In our example, 'Meaning' of the lookup is the value column Name. We pass the value 'Company' to filter the row.

As per our Value Set and bind values, we expect company contribution Lookup Code, ERC_CNTRB.

Hope the above examples helped you understand how a Formula and Value Set can be used to execute any SQL in Fast Formula.

YouTube Video 19 Formula:

/******

Name: CMP_PERSON_SELECTION

Type: Compensation Person Selection

Programmer: Tilak

purpose: Do not process the people who where already process by

checking the period processed flag and row in person statement

*****/

INPUTS ARE CMP_IV_STMT_ID, CMP_IV_PERIOD_ID

Item = ess_log_write('Entering CMP_PERSON_SELECTION')

I_value = 'Y'

stmtprocessed = GET_VALUE_SET('TCS_STMT_PROCESSED')

Item = ess_log_write('TCS_STMT_PROCESSED ' + stmtprocessed)

IF stmtprocessed = 'Y' then (

I_value = 'N'

)

Item = ess_log_write('Leaving CMP_PERSON_SELECTION ' + I_value)

RETURN I_value

YouTube Video 20 Formula:

/*****

Name: CMP_PERSON_SELECTION

Type: Compensation Person Selection

Programmer: Tilak

purpose: Do not process the people who where already process by

checking the period processed flag and row in person statement

/*****

Name: CMP_PERSON_SELECTION

Type: Compensation Person Selection

Programmer: Tilak

purpose: Do not process the people who where already process by

checking the period processed flag and row in person statement

*****/

INPUTS ARE CMP_IV_STMT_ID, CMP_IV_PERIOD_ID

```
ltemp = ess_log_write(' Entering CMP_PERSON_SELECTION')
```

```
l_value = 'Y'
```

```
/* Calling valueSet */
```

```
/*
```

```
stmtprocessed = GET_VALUE_SET('TCS_PERSON_SELECTION')
```

```
*/
```

```
stmtperd = TO_CHAR(CMP_IV_PERIOD_ID)
```

```
ltemp = ess_log_write(' CMP_IV_STMT_PERD_ID: ' + stmtperd )
```

```
param = '|=STMTPERDID=||' || stmtperd
```

```
ltemp = ess_log_write(' param ' + param)
```

```
stmtprocessed = GET_VALUE_SET('TCS_STMT_PROCESSED', param)
```

```
ltemp = ess_log_write(' TCS_PERSON_SELECTION: ' + stmtprocessed)
```

```
/* Validating result of value set */
```

```
If stmtprocessed = 'Y' Then (
```

```
l_value = 'N'
```

)

Item = ess_log_write(' Leaving CMP_PERSON_SELECTION: ' + I_value)

RETURN I_value

Youtube Video 21 Formula

/*****

Name: CMP_PERSON_SELECTION

Type: Compensation Person Selection

Programmer: Tilak

purpose: Do not process the people who where already process by

checking the period processed flag and row in person statement

*****/

INPUTS ARE CMP_IV_STMT_ID, CMP_IV_PERIOD_ID

Item = ess_log_write('Entering CMP_PERSON_SELECTION')

I_value = 'Y'

```
personid = get_context(PERSON_ID, -1)
```

```
perid = TO_CHAR(CMP_IV_PERIOD_ID)
```

```
Item = ess_log_write('stmt id ' + to_char(CMP_IV_STMT_ID) )
```

```
Item = ess_log_write('CMP_IV_PERIOD_ID ' + perid)
```

```
Item = ess_log_write('person id ' + to_char(personid) )
```

```
param = '|=STMTPERDID=||'perid
```

```
Item = ess_log_write('param ' + param)
```

```
/* calling value set */
```

```
stmtprocessed = GET_VALUE_SET('TCS_PERSON_SELECTION',param )
```

```
Item = ess_log_write('TCS_PERSON_SELECTION ' + stmtprocessed)
```

```
IF stmtprocessed = 'Y' then (
```

```
param = '|=STMTPERDID=||'perid ||' |STMTID=|| TO_CHAR(CMP_IV_STMT_ID)||' |PERSON_ID=|| to_char(personid)
```

```
Item = ess_log_write('param ' + param)
```

```
perperdid = GET_VALUE_SET('TCS_STMT_PROCESSED',param )
```

```
Item = ess_log_write('perperdid ' + perperdid )
```

```
if perperdid != ' ' then (  
  
    l_value = 'N'  
  
)  
  
)  
  
Item = ess_log_write('Leaving CMP_PERSON_SELECTION ' + l_value )  
  
RETURN l_value
```

Fast Formula – Benefits

Fast Formula – Benefits, Part 1

In this session, we have covered the Fast Formula usages in Benefits.

In HCM, Benefits is one of the products that extensively make use of Fast Formula Functionality. I understand that Benefit has created the most number of Formula types.

This session only covers the Types, context and input values of Benefit Formulas. We will try to cover the Benefit Formula Functions in the future.

I have collected the information from different sources. Therefore, the information provided in this blog may not be up to date or the functionality may not be available in your release.

Please confirm with your support/development for the accuracy of this information. If there is any discrepancy, please let us know so that we can correct the blog

Benefit Formula Types:

Formula Type Name	Description	Usage	Return
Action Type Due Date	Determines the date on which the Action Type must be completed (e.g. 90 days from the Life Event Creation Date).	Enrollment requirements	Date
Age Calculation	Used to determine an Age value to be used for Eligibility, Coverage or Benefit, Premium, and Rate calculations	Derived factors	Amount
Age Determination Date	Determines the date from which age will be calculated (e.g., the first day of the next plan year).	Derived factors	Date
Automatic Enrollment Method	Determines the conditions under which a person should be automatically enrolled in a compensation object.		A/E/N
Beneficiary Certification Required	Determines the conditions under which a person must provide certification for his or her designated beneficiaries.	Enrollment requirements	Y/N
Benefits Extract Custom Data Rule			
Break In Service	Determines whether a break in service has occurred and if this break should not be ignored.		Y/N
Break In Service Value	Break In Service Value		Amount

Certification Allow Reimbursement			
Change Dependent Coverage	Used to determine if a dependent can be 'Added Only', 'Removed Only', 'Added and Removed', or 'Neither'.	Dependent requirements	Y/N
Collapse Configuration			
Compensation Calculation	Used to determine a Total Compensation amount to be used for Eligibility, Coverage or Benefit, Premium and Rate calculations	Derived factors	Amount
Compensation Determination Date	Determines the high date from which hours compensation will be calculated (e.g., the first day of the next plan year).	Derived factors	Date
Compensation Object Selection	Determines which compensation objects are to be included for processing in a concurrent manager process.		Y/N
Contribution Non-discriminatory	Compares individual employee pretax contribution amounts to total pretax contributions for highly and non-highly compensated persons.		Y/N
Coverage Amount Calculation	Calculates a coverage or benefit amount for a person.	Coverage	Amount
Coverage Amount Limit		Coverage	
Coverage Lower Limit		Coverage	
Coverage Non-discriminatory	Compares the total number of persons eligible to participate in a plan minus the persons who are not eligible due to legislated allowable factors to the number actually participating.	Coverage	Y/N
Coverage Sequence Limit		Coverage	
Coverage Upper Limit		Coverage	

Deduction Schedule	Determines the deduction schedule to be used for this person.		One of the BEN_DE D_S CHED values: 1N3WKN MO, 1PRDIM O, 1SYPNY R, 1WKNBI WK, 1WKNM O, 2N4WKN MO, 2PRDIM O, 2SYPNY R, 2WKNBI WK, 2WKNM O, 3WKNM O, 4WKNM O, APR, AUG, DEC, EPP, FEB, JAN, JUL, JUN, MAR, MAY, NOV, OCT, SEP,
Default Enrollment	Determines whether this option in plan should be assigned to a person as part of the default enrollment process.	Enrollment requirements	

Default Action	to Assign Pending	Determines the applicable default option or benefit to assign to the person when his or her choice has been suspended.	Enrollment requirements	One of the BEN_DF LT_T O_ASN_PND G_CTFN values : CASDFN DF, CASDFN MN, CASDFN NL, CASDFN NT, CASMN NDF, CASMN NMN, CASMN NNL, CASMN NNT, CASNLN DF, CASNLN MN, CASNLN NL, CASNLN NT, CASNTN DF, CASNTN MN, CASNTN NL, CASNTN NT, CASSMN DF, CASSMN MN, CASSMN NL, CASSMN NT, CDND, CDNL, CDNM, CDNN, CLND, CLNL, CLNM, CLNN, CMND, CMNL, CMNM, CMNN, CNND, CNNL, CNNM, CNNN, CSEDFN DF, CSEDFN MN, CSEDFN
-------------------	-------------------	--	-------------------------	---

			NL, CSEDFN NT, CSEMNN DF, CSEMNN MN, CSEMNN NL, CSEMNN NT, CSENLN DF, CSENLN MN, CSENLN NL, CSENLN NT, CSENTN DF, CSENTN MN, CSENTN NL, CSENTN NT, CSESMN DF, CSESMN MN, CSESMN NL, CSESMN NT, CSND, CSNL, CSNM, CSNN, CSODFN DF, CSODFN MN, CSODFN NL, CSODFN NT, CSOMN NDF, CSOMN NMN, CSOMN NNL, CSOMN NNT, CSONLN DF, CSONLN MN, CSONLN NL, CSONLN NT, CSONTN DF, CSONTN MN, CSONTN NL,
--	--	--	---

			CSONTN NT, CSOSMN DF, CSOSMN MN, CSOSMN NL, CSOSMN NT, RL,
Dependent Certification Required	Determines the conditions under which a person must provide certification for his or her designated dependents.	Dependent requirements	Y/N
Dependent Coverage End Date	Determines when coverage for a dependent should end	Dependent requirements	Date
Dependent Coverage Start Date	Determines when coverage for a dependent should start.	Dependent requirements	Date
Dependent Eligibility	Determines whether a person is eligible to be covered by a participant in a compensation object	Eligibility	Y/N
Election Value Range Rule			
Element Type and Input Value Determination			
Eligibility Access Calculation Rule	Returns a person's value for a user-defined criteria so it can be evaluated in the eligibility determination process.	Eligibility	Criteria (and subcriteri a) values
Enrollment Certification Required	Determines the conditions under which a person must provide certification in order to enroll or elect a particular plan or option in plan, or benefit.	Enrollment requirements	Y/N
Enrollment Coverage Start Date	Determines when enrollment coverage for a participant should start.	Enrollment requirements	Date
Enrollment End	Determines when enrollment coverage for a participant should end.	Enrollment requirements	Date

Enrollment Opportunity	Determines whether the compensation object should be an electable choice for the person..	Enrollment requirements	A/E/N
Enrollment Period End Date	Determines the date on which the enrollment period Ends.	Enrollment requirements	Date
Enrollment Period Start Date	Determines the date on which the enrollment period starts.	Enrollment requirements	Date
Evaluate Life Event	Determines if this life event is valid, or if the life event information needs to be changed, e.g. status, voided date, unprocessed date.		Date
Excess Credits Treatment		Life events	
Extra Input		Rates	
Family Member Determination		Eligibility	
Handling of Non-winning events			
Hours Worked Calculation	Used to determine an Hours Worked amount to be used for Eligibility, Coverage or Benefit, Premium,and Rate calculations	Derived factors	Amount
Hours Worked Determination Date	Determines the date from which hours worked will be calculated (e.g., the first day of the next plan year).	Derived factors	Date
Inspection Required	Determines whether inspection of the communication is required. If so, the "Inspection Flag" is set to 'yes' for this person's communication.		Y/N
Length of Service Calculation	Used to determine a Service value to be used for Eligibility, Coverage or Benefit, Premium, and Rate calculations	Derived factors	Amount
Length of Service Date to Use	Determines the low date from which LOS will be calculated (e.g., original hire date or rehire date).	Derived factors	Date

Length of Service Determination Date	Determines the high date for which LOS will be calculated (e.g., the first day of the following month).	Derived factors	
Life Event Occurred Date		Life events	Date
Life Event Reason Timeliness	Determines whether a life event has been reported in a timely manner.	Life events	Y/N
Mandatory Determination	Determines whether this option in plan should be assigned to a person (and not be optional) as part of the enrollment process.		Y/N
Matching Amount Calculation	Calculates the matching amount.		Amount
Maximum Coverage Amount Calculation	Calculates a maximum coverage or benefit amount for a person.	Coverage	Amount
Maximum Period of Enrollment	Period of Enrollment Rule determines whether a person has been enrolled for the maximum length of time allowed for a plan or option in a plan.	Enrollment requirements	Y/N
Maximum Period of Enrollment Determination Date	Determines the low date to be used when determining whether the person has been enrolled in a plan or option in plan for the maximum period of time.	Enrollment requirements	Date
Maximum Period of Enrollment Value and UOM	Determines the maximum enrollment period and unit of measure for a compensation object.	Enrollment requirements	Amount and UOM
Maximum Waiting Period Date to Use		Enrollment requirements	
Maximum Waiting Period to Participate			
Minimum Coverage Amount Calculation	Calculates a minimum coverage or benefit amount for a person.	Coverage	Amount

Partial Month Effective Date Determination	Determines which value to use: Date Earned, Pay Period End Date, or Payment Date. This tells the proration process which date to use when determining how many pay periods remain, and when the element entry should start.		One of the Values of : BEN_PR TL_ MO_EFF _DT _DET: DTERND , DTPD, PPED, RL,
Partial Month Proration Method	Determines which value to use: Date Earned, Pay Period End Date, or Payment Date. This tells the proration process which date to use when determining how many pay periods remain, and when the element entry should start.		One of the Values of : BEN_PR TL_ MO_DET _MTHD: ALL, NONE, PRTVAL, RL, WASHR ULE,
Partial Month Proration Rule	Calculates a value for a partial month enrollment.		Percent
Partial Year Coverage Restriction	Determines the maximum coverage amount for partial years of coverage. Allows determination of values other than delivered values of: 1) Standard maximum as defined in the plan. 2) Prorate maximum based on the number of months remaining in the plan year.		Amount
Participation Eligibility End Date	Determines when eligibility for a person should End.	Eligibility	Date
Participation Eligibility Start Date	Determines when eligibility for a person should start.	Eligibility	Date
Participation and Rate Eligibility	Determines whether a person is eligible for the associated compensation object. (This is the rule used in the eligibility profile rule entities.)		Y/N in variable ELIGIBLE
Payment Must Be Received	Determines whether a payment is missing or late.		Y/N

Payment Schedule	Determines the payment schedule to be used for this person.		One of the BEN_PY MT_SCHED Values: 1N3WKN MO, 1PRDIM O, 1QTRNY R, 1SYRPN YR, 1WKINB IWK, 1WKINM O, 2N4WKN MO, 2PRDIM O, 2QYRNY R, 2SYRPN YR, 2WKNBI WK, 2WKNM O, 3QTRNY R, 3WKNM O, 4QTRNY R, 4WKNM O, APR, AUG, DEC, EPP, FEB, JAN, JUL, JUN, MAR, MAY, NOV, OCT, SEP,
Period to Date Amount Calculation	Determines the maximum period to date amount a person may have for a particular activity rate.		Amount
Person Change Causes Life Event	Determines whether this life event is valid for a person based on the data that changed.	Life events	Y/N
Person Selection	Determines which people are to be included for processing in a concurrent manager process.	Batch parameter	Y/N

Postelection Edit	Determines whether this option in plan should be assigned to a person (and not be optional) as part of the enrollment process.	Enrollment requirements	Y/N and Text
Preferential Rate Calculation	For a given criteria rate definition, if an employee is found eligible for multiple rates by the Rate by Criteria API, the Preferential Rate Calculation formula can resolve which of these rates/ combination of rates must be paid to the employee.		MIN, MID, MAX, DFLT
Premium Lower Limit	Calculates a lower limit of premium amount for a person.		Amount
Premium Upper Limit	Calculates a higher limit of premium amount for a person.		Amount
Prorate Annual Election Value	Determines how a minimum or maximum annual election value is prorated.		Amount
Range of Scheduled Hours		Eligibility	
Rate End Date	Determines when a rate for an enrollment result for a participant should End	Rates	Date
Rate Lower Limit	Calculates an amount used to determine the lower limit value to which an activity rate or variable rate value is compared.	Rates	Amount
Rate Periodization		Rates	
Rate Start Date	Determines when a rate for an enrollment result for a participant should start.	Rates	Date
Rate Upper Limit	Calculates an amount used to determine the upper limit value to which an activity rate or variable rate value is compared.	Rates	Amount
Rate Value Calculation	Calculates a rate amount for a person. May be used to calculate a base or variable rate	Rates	Amount
Related Person Change Causes Life Event	Determines whether this life event is valid for a person based on the data that changed.	Life events	Y/N

Required Period of Enrollment	Determines the earliest de-enrollment date for a person's electable choice for a compensation object.	Enrollment requirements	Date
Rounding	Rounds a number to the specified place or decimal.		Amount
To Be Sent Date	Determines the date on which the communication should be sent to the person.		Date
Unsuspend Date to Use			
Variable Rate Add On Calculation	Calculates a new value when a variable rate result is used.		Amount
Vested Value			
Waiting Period Value and UOM	Determines the waiting period and unit of measure to be applied to a person.	Enrollment requirements	Amount and UOM
Waive Certification Required	Determines the conditions under which a person must provide certification when he or she waives participation.	Enrollment requirements	Y/N

Fast Formula – Benefits, part 2

Benefit Formula Types Contexts:

Formula Type Base Name	Contxt Base Name	Context Name
Action Type Due Date	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned

	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person

	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Age Calculation	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer

	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Age Determination Date	BENEFIT_RELATION_ID	Benefit Relation

	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization

	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Automatic Enrollment Method	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ID	Eligible Person
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship

	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type

Beneficiary Certification Required	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term

	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Benefits Extract Custom Data Rule	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From

	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type

Break In Service	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person

Break In Service Value	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person

Certification Allow Reimbursement	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person

Change Dependent Coverage	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term

	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Collapse Configuration	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
Compensation Calculation	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned

	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship

	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Compensation Determination Date	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From

	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ACTION_ID	Payroll Action
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type

Compensation Object Selection	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	EFFECTIVE_DATE	EFFECTIVE_DATE
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Contribution Non-discriminatory	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term

	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Coverage Amount Calculation	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice

	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person

	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type

-----*****-----

Fast Formula – Benefits, part 3

Benefit Formula Types Contexts:

Formula Type Name	Context Base Name	Context Name
Coverage Amount Limit	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment

	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program

	PL_ID	Plan
	PL_TYP_ID	Plan Type
Coverage Lower Limit	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer

	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Coverage Non-discriminatory	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE

	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Coverage Sequence Limit	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE

	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person

	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Coverage Upper Limit	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From

	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Deduction Schedule	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID

	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Default Enrollment	BENEFIT_RELATION_ID	Benefit Relation

	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ID	Eligible Person
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization

	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Default to Assign Pending Action	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job

	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Dependent Certification Required	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned

	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person

	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Dependent Coverage End Date	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer

	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Dependent Coverage Start Date	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID

	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll

	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Dependent Eligibility	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job

	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type

Election Value Range Rule	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Element Type and Input Value Determination	ACTY_BASE_RT_ID	Activity Base Rate

	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship

	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type

Fast Formula – Benefits, part 4

Benefit Formula Types Contexts

Formula Type Name	Context Base Name	Context Base Name
Eligibility Access Calculation Rule	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE

	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan

	PL_TYP_ID	Plan Type
Enrollment Certification Required	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	ENRT_CTFN_TYP_CD	Certification type
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option

	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Enrollment Coverage Start Date	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship

	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program

	PL_ID	Plan
	PL_TYP_ID	Plan Type
Enrollment End	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason

	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Enrollment Opportunity	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment

	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program

	PL_ID	Plan
	PL_TYP_ID	Plan Type
Enrollment Period End Date	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group

	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Enrollment Period Start Date	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned

	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship

	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Evaluate Life Event	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From

	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Excess Credits Treatment	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment

	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGAL_ENTITY_ID	Benefit Relation
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event
	PGM_ID	Program

	PL_ID	Plan
	PL_TYP_ID	Plan Type
Extra Input	ACTY_BASE_RT_ID	Activity Base Rate
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option

	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Family Member Determination	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship

	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan

	PL_TYP_ID	Plan Type
Handling of Non-winning events	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LER_ID	Life Event Reason
Hours Worked Calculation	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship

	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan

	PL_TYP_ID	Plan Type
Hours Worked Determination Date	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ACTION_ID	Payroll Action
	PAYROLL_ID	Payroll
	PERSON_ID	Person

	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Inspection Required	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option

	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Length of Service Calculation	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship

	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program

	PL_ID	Plan
	PL_TYP_ID	Plan Type
Length of Service Date to Use	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group

	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Length of Service Determination Date	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned

	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship

	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Life Event Occurred Date	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LER_ID	Life Event Reason
Life Event Reason Timeliness	BENEFIT_RELATION_ID	Benefit Relation

	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship

	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Mandatory Determination	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization

	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Matching Amount Calculation	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job

	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Maximum Coverage Amount Calculation	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship

	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan

	PL_TYP_ID	Plan Type
Maximum Period of Enrollment	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option

	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Maximum Period of Enrollment Determination Date	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term

	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type

Maximum Period of Enrollment Value and UOM	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person

Maximum Waiting Period Date to Use	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization

	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Maximum Waiting Period to Participate	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job

	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Minimum Coverage Amount Calculation	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship

	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan

	PL_TYP_ID	Plan Type
--	-----------	-----------

-----*****-----

Fast Formula – Benefits, part 5

Benefit Formula Types Contexts

Formula Type Name	Context Base Name	Context Name
Partial Month Effective Date Determination	ACTY_BASE_RT_ID	Activity Base Rate
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job

	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Partial Month Proration Method	ACTY_BASE_RT_ID	Activity Base Rate
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID

	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term

	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Partial Month Proration Rule	ACTY_BASE_RT_ID	Activity Base Rate
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer

	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Partial Year Coverage Restriction	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE

	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Participation Eligibility End Date	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned

	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship

	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Participation Eligibility Start Date	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From

	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type

Participation and Rate Eligibility	ACTY_BASE_RT_ID	Activity Base Rate
	ADDRESS_ID	Address
	ADDRESS_TYPE	Address Type
	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	DEDUCTION_CARD_ID	Deduction Card
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer

	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
	TAX_UNIT_ID	Tax Unit

Payment Must Be Received	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	PERSON_ID	Person
Payment Schedule	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option

	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Period to Date Amount Calculation	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term

	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Person Change Causes Life Event	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship

	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Person Selection	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment

	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program

	PL_ID	Plan
	PL_TYP_ID	Plan Type
Postelection Edit	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To

	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event
	PL_ID	Plan
Preferential Rate Calculation	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned

	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Prorate Annual Election Value	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation

	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term

	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Range of Scheduled Hours	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group

	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Rate End Date	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned

	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll

	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Rate Lower Limit	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment

	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event

	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Rate Periodization	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job

	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event
	PGM_ID	Program
	PL_ID	Plan

	PL_TYP_ID	Plan Type
Rate Start Date	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer

	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Rate Upper Limit	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation

	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option

	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Rate Value Calculation	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation
	BNFTS_BAL_ID	Benefit Balance
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned

	EFFECTIVE_DATE	EFFECTIVE_DATE
	ELIG_PER_ELCTBL_CHC_ID	Electable Choice
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_ID	Payroll

	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Related Person Change Causes Life Event	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term

	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Required Period of Enrollment	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term

	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Rounding	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID

	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person

Fast Formula – Benefits, part 6

Benefit Formula Types Contexts

To Be Sent Date	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization

	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Unsuspend Date to Use	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term

	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Variable Rate Add On Calculation	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE

	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event
	PL_ID	Plan

Fast Formula – Benefits, part 6

Benefit Formula Types Contexts

Formula Type Name	Context Base Name	Context Name
To Be Sent Date	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option

	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Unsuspend Date to Use	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship

	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
Variable Rate Add On Calculation	ACTY_BASE_RT_ID	Activity Base Rate
	BENEFIT_RELATION_ID	Benefit Relation
	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned

	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PER_IN_LER_ID	Person Life Event

	PL_ID	Plan
Vested Value	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term

	PERSON_ID	Person
Waiting Period Value and UOM	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term
	JOB_ID	Job
	LC_DATE_FROM	Life Event Collapse - Date From
	LC_DATE_TO	Life Event Collapse - Date To
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option

	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type
Waive Certification Required	BUSINESS_GROUP_ID	BUSINESS_GROUP_ID
	DATE_EARNED	Date Earned
	EFFECTIVE_DATE	EFFECTIVE_DATE
	HR_ASSIGNMENT_ID	HR Assignment
	HR_RELATIONSHIP_ID	HR Relationship
	HR_TERM_ID	HR Term

	JOB_ID	Job
	LEGAL_EMPLOYER_ID	Legal Employer
	LEGISLATIVE_DATA_GROUP_ID	Legislative Data Group
	LER_ID	Life Event Reason
	OPT_ID	Option
	ORGANIZATION_ID	Organization
	PAYROLL_ASSIGNMENT_ID	Payroll Assignment
	PAYROLL_RELATIONSHIP_ID	Payroll Relationship
	PAYROLL_TERM_ID	Payroll Term
	PERSON_ID	Person
	PGM_ID	Program
	PL_ID	Plan
	PL_TYP_ID	Plan Type

Fast Formula – Benefits, part 7

Benefit Formula Input Values:

Formula Type Base Name	Input Values
Coverage Amount Limit	BEN_IV_RT_STRT_DT
	BEN_IV_RT_STRT_DT
Coverage Lower Limit	BEN_IV_RT_STRT_DT
	BEN_IV_RT_STRT_DT
Default to Assign Pending Action	BEN_PEN_IV_PRTT_ENRT_RSLT_ID
	BEN_PER_IV_PERSON_ID
	BEN_PIL_IV_PER_IN_LER_ID
Dependent Coverage End Date	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID
	BEN_IV_EFFECTIVE_DATE
Dependent Coverage Start Date	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID
	BEN_IV_EFFECTIVE_DATE
Dependent Eligibility	BEN_ELP_I_ELIGY_PRFL_ID
	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID
Eligibility Access Calculation Rule	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
Enrollment Coverage Start Date	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID
	BEN_IV_EFFECTIVE_DATE
Enrollment End	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID
	BEN_IV_EFFECTIVE_DATE
Enrollment Period End Date	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID

	BEN_IV_EFFECTIVE_DATE
Enrollment Period Start Date	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID
	BEN_IV_EFFECTIVE_DATE
Family Member Determination	PERSON_ID
	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
Maximum Coverage Amount Calculation	BEN_IV_PERSON_ID
Maximum Period of Enrollment Value and UOM	BEN_IV_PERSON_ID
Participation Eligibility End Date	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID
Participation Eligibility Start Date	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID
Participation and Rate Eligibility	BEN_ELP_I_ELIGY_PRFL_ID
	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID
Person Change Causes Life Event (Different set of inputvalues passed for different table/columns)	BEN_SAL_IN_ASSIGNMENT_ID
	BEN_SAL_IO_ASSIGNMENT_ID
	BEN_SAL_IN_SALARY_AMOUNT
	BEN_SAL_IO_SALARY_AMOUNT
	BEN_SAL_IN_SALARY_BASIS_ID
	BEN_SAL_IO_SALARY_BASIS_ID
	BEN_SAL_IN_NEXT_PERF_REVIEW_DATE
	BEN_SAL_IO_NEXT_PERF_REVIEW_DATE
	BEN_SAL_IN_NEXT_SAL_REVIEW_DATE
	BEN_SAL_IO_NEXT_SAL_REVIEW_DATE
	BEN_SAL_IN_SALARY_REASON_CODE
	BEN_SAL_IO_SALARY_REASON_CODE
	BEN_SAL_IN_DATE_FROM
	BEN_SAL_IO_DATE_FROM
	BEN_SAL_IN_DATE_TO
	BEN_SAL_IO_DATE_TO
	BEN_SAL_IN_ELEMENT_ENTRY_ID
	BEN_SAL_IO_ELEMENT_ENTRY_ID
	BEN_SAL_IN_FORCED_RANKING
	BEN_SAL_IO_FORCED_RANKING

	BEN_SAL_IN_PERFORMANCE_RATING
	BEN_SAL_IN_PERFORMANCE_REVIEW_ID
	BEN_SAL_IN_REVIEW_DATE
	BEN_SAL_IO_REVIEW_DATE
	BEN_SAL_IN_SALARY_BASIS_NAME
	BEN_SAL_IO_SALARY_BASIS_NAME
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_PTU_IN_PERSON_TYPE_USAGE_ID
	BEN_PTU_IO_PERSON_TYPE_USAGE_ID
	BEN_PTU_IN_EFFECTIVE_START_DATE
	BEN_PTU_IO_EFFECTIVE_START_DATE
	BEN_PTU_IN_EFFECTIVE_END_DATE
	BEN_PTU_IO_EFFECTIVE_END_DATE
	BEN_PTU_IN_PERSON_ID
	BEN_PTU_IO_PERSON_ID
	BEN_IV_LER_ID
	BEN_PTU_IN_PERSON_TYPE_ID
	BEN_PTU_IO_PERSON_TYPE_ID
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_PSS_IN_PERSON_ID
	BEN_PSS_IO_PERSON_ID
	BEN_PSS_IN_BUSINESS_GROUP_ID
	BEN_PSS_IO_BUSINESS_GROUP_ID
	BEN_PSS_IN_EXPIRATION_DATE
	BEN_PSS_IO_EXPIRATION_DATE
	BEN_PSS_IN_ISSUE_DATE
	BEN_PSS_IO_ISSUE_DATE
	BEN_PSS_IN_ISSUING_AUTHORITY
	BEN_PSS_IO_ISSUING_AUTHORITY
	BEN_PSS_IN_ISSUING_COUNTRY
	BEN_PSS_IO_ISSUING_COUNTRY
	BEN_PSS_IN_ISSUING_LOCATION
	BEN_PSS_IO_ISSUING_LOCATION

	BEN_PSS_IN_LEGISLATION_CODE
	BEN_PSS_IO_LEGISLATION_CODE
	BEN_PSS_IN_PASSPORT_ID
	BEN_PSS_IN_PASSPORT_NUMBER
	BEN_PSS_IO_PASSPORT_NUMBER
	BEN_PSS_IN_PASSPORT_TYPE
	BEN_PSS_IO_PASSPORT_TYPE
	BEN_PSS_IN_PERSON_ID
	BEN_PSS_IO_PERSON_ID
	BEN_PSS_IN_PROFESSION
	BEN_PSS_IO_PROFESSION
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_PRV_IN_ACTY_TYP_CD
	BEN_PRV_IO_ACTY_TYP_CD
	BEN_PRV_IN_ANN_RT_VAL
	BEN_PRV_IO_ANN_RT_VAL
	BEN_PRV_IN_CMCD_RT_VAL
	BEN_PRV_IO_CMCD_RT_VAL
	BEN_PRV_IN_ELCTNS_MADE_DT
	BEN_PRV_IO_ELCTNS_MADE_DT
	BEN_PRV_IN_PRTT_RT_VAL_STAT_CD
	BEN_PRV_IO_PRTT_RT_VAL_STAT_CD
	BEN_PRV_IN_RT_END_DT
	BEN_PRV_IO_RT_END_DT
	BEN_PRV_IN_RT_OVRIDN_FLAG
	BEN_PRV_IO_RT_OVRIDN_FLAG
	BEN_PRV_IN_RT_OVRIDN_THRU_DT
	BEN_PRV_IO_RT_OVRIDN_THRU_DT
	BEN_PRV_IN_RT_STRT_DT
	BEN_PRV_IO_RT_STRT_DT
	BEN_PRV_IN_RT_VAL
	BEN_PRV_IO_RT_VAL
	BEN_PRV_IO_TX_TYP_CD
	BEN_PRV_IN_TX_TYP_CD
	NEW_VAL
	OLD_VAL
	PK_ID

	COLUMN
	BEN_PPS_IN_DATE_START
	BEN_PPS_IO_DATE_START
	BEN_PPS_IN_ACCEPTED_TERMINATION_DATE
	BEN_PPS_IO_ACCEPTED_TERMINATION_DATE
	BEN_PPS_IN_ACTION_OCCURRENCE_ID
	BEN_PPS_IO_ACTION_OCCURRENCE_ID
	BEN_PPS_IN_ACTUAL_TERMINATION_DATE
	BEN_PPS_IO_ACTUAL_TERMINATION_DATE
	BEN_PPS_IN_ADJUSTED_SVC_DATE
	BEN_PPS_IO_ADJUSTED_SVC_DATE
	BEN_PPS_IN_DATE_START
	BEN_PPS_IO_DATE_START
	BEN_PPS_IN_LAST_WORKING_DATE
	BEN_PPS_IO_LAST_WORKING_DATE
	BEN_PPS_IN_LEGAL_ENTITY_ID
	BEN_PPS_IO_LEGAL_ENTITY_ID
	BEN_PPS_IN_LEGISLATION_CODE
	BEN_PPS_IO_LEGISLATION_CODE
	BEN_PPS_IN_NOTIFIED_TERMINATION_DATE
	BEN_PPS_IO_NOTIFIED_TERMINATION_DATE
	BEN_PPS_IN_ON_MILITARY_SERVICE
	BEN_PPS_IO_ON_MILITARY_SERVICE
	BEN_PPS_IN_ORIGINAL_DATE_OF_HIRE
	BEN_PPS_IO_ORIGINAL_DATE_OF_HIRE
	BEN_PPS_IN_PRIMARY_FLAG
	BEN_PPS_IO_PRIMARY_FLAG
	BEN_PPS_IN_PROJECTED_TERMINATION_DATE
	BEN_PPS_IO_PROJECTED_TERMINATION_DATE
	BEN_PPS_IN_WORKER_NUMBER
	BEN_PPS_IO_WORKER_NUMBER
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_PPL_IN_PERSON_ID
	BEN_PPL_IO_PERSON_ID
	BEN_PPL_IN_BUSINESS_GROUP_ID
	BEN_PPL_IO_BUSINESS_GROUP_ID

	BEN_PPL_IN_EFFECTIVE_END_DATE
	BEN_PPL_IO_EFFECTIVE_END_DATE
	BEN_PPL_IN_EFFECTIVE_START_DATE
	BEN_PPL_IO_EFFECTIVE_START_DATE
	BEN_PPL_IN_HIGHEST_EDUCATION_LEVEL
	BEN_PPL_IO_HIGHEST_EDUCATION_LEVEL
	BEN_PPL_IN_LEGISLATION_CODE
	BEN_PPL_IO_LEGISLATION_CODE
	BEN_PPL_IN_MARITAL_STATUS
	BEN_PPL_IO_MARITAL_STATUS
	BEN_PPL_IN_MARITAL_STATUS_DATE
	BEN_PPL_IO_MARITAL_STATUS_DATE
	BEN_PPL_IN_PERSON_ID
	BEN_PPL_IO_PERSON_ID
	BEN_PPL_IN_PERSON_LEGISLATIVE_ID
	BEN_PPL_IO_PERSON_LEGISLATIVE_ID
	BEN_PPL_IN_SEX
	BEN_PPL_IO_SEX
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_PPF_IN_ATTRIBUTE1 .. 6
	BEN_PPF_IO_ATTRIBUTE1 .. 6
	BEN_PPF_IN_ATTRIBUTE_CATEGORY
	BEN_PPF_IO_ATTRIBUTE_CATEGORY
	BEN_PPF_IN_MAILING_ADDRESS_ID
	BEN_PPF_IO_MAILING_ADDRESS_ID
	BEN_PPF_IN_PERSON_NUMBER
	BEN_PPF_IO_PERSON_NUMBER
	BEN_PPF_IN_PRIMARY_EMAIL_ID
	BEN_PPF_IO_PRIMARY_EMAIL_ID
	BEN_PPF_IN_PRIMARY_NID_ID
	BEN_PPF_IO_PRIMARY_NID_ID
	BEN_PPF_IN_APPLICANT_NUMBER
	BEN_PPF_IO_APPLICANT_NUMBER
	BEN_PPF_IN_WAIVE_DATA_PROTECT
	BEN_PPF_IO_WAIVE_DATA_PROTECT
	BEN_PPF_IN_START_DATE
	BEN_PPF_IO_START_DATE

	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_PPE_IN_PERSON_ID
	BEN_PPE_IO_PERSON_ID
	BEN_PPE_IN_BLOOD_TYPE
	BEN_PPE_IO_BLOOD_TYPE
	BEN_PPE_IN_BUSINESS_GROUP_ID
	BEN_PPE_IO_BUSINESS_GROUP_ID
	BEN_PPE_IN_CORRESPONDENCE_LANGUAGE
	BEN_PPE_IO_CORRESPONDENCE_LANGUAGE
	BEN_PPE_IN_COUNTRY_OF_BIRTH
	BEN_PPE_IO_COUNTRY_OF_BIRTH
	BEN_PPE_IN_DATE_OF_BIRTH
	BEN_PPE_IO_DATE_OF_BIRTH
	BEN_PPE_IN_DATE_OF_DEATH
	BEN_PPE_IO_DATE_OF_DEATH
	BEN_PPE_IN_PARTY_ID
	BEN_PPE_IO_PARTY_ID
	BEN_PPE_IN_PERSON_ID
	BEN_PPE_IO_PERSON_ID
	BEN_PPE_IN_REGION_OF_BIRTH
	BEN_PPE_IO_REGION_OF_BIRTH
	BEN_PPE_IN_START_DATE
	BEN_PPE_IO_START_DATE
	BEN_PPE_IN_TOWN_OF_BIRTH
	BEN_PPE_IO_TOWN_OF_BIRTH
	BEN_PPE_IN_USER_GUID
	BEN_PPE_IO_USER_GUID
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_NID_IN_NATIONAL_IDENTIFIER_ID
	BEN_NID_IO_NATIONAL_IDENTIFIER_ID
	BEN_NID_IN_NATIONAL_IDENTIFIER_TYPE
	BEN_NID_IO_NATIONAL_IDENTIFIER_TYPE
	BEN_NID_IN_BUSINESS_GROUP_ID

	BEN_NID_IO_BUSINESS_GROUP_ID
	BEN_NID_IN_CREATED_BY
	BEN_NID_IO_CREATED_BY
	BEN_NID_IN_ISSUE_DATE
	BEN_NID_IO_ISSUE_DATE
	BEN_NID_IN_EXPIRATION_DATE
	BEN_NID_IO_EXPIRATION_DATE
	BEN_NID_IN_LAST_UPDATED_BY
	BEN_NID_IO_LAST_UPDATED_BY
	BEN_NID_IN_LAST_UPDATE_LOGIN
	BEN_NID_IO_LAST_UPDATE_LOGIN
	BEN_NID_IN_NATIONAL_IDENTIFIER_NUMBER
	BEN_NID_IO_NATIONAL_IDENTIFIER_NUMBER
	BEN_NID_IN_LEGISLATION_CODE
	BEN_NID_IO_LEGISLATION_CODE
	BEN_NID_IN_PLACE_OF_ISSUE
	BEN_NID_IO_PLACE_OF_ISSUE
	BEN_NID_IN_PERSON_ID
	BEN_NID_IO_PERSON_ID
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_PEI_IN_PERSON_ID
	BEN_PEI_IO_PERSON_ID
	BEN_PEI_IN_EFFECTIVE_END_DATE
	BEN_PEI_IO_EFFECTIVE_END_DATE
	BEN_PEI_IN_EFFECTIVE_START_DATE
	BEN_PEI_IO_EFFECTIVE_START_DATE
	BEN_PEI_IN_PERSON_EXTRA_INFO_ID
	BEN_PEI_IO_PERSON_EXTRA_INFO_ID
	BEN_PEI_IN_PERSON_ID
	BEN_PEI_IO_PERSON_ID
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_PEE_IN_PERSON_ID

	BEN_PEE_IO_PERSON_ID
	BEN_PEE_IN_EFFECTIVE_END_DATE
	BEN_PEE_IO_EFFECTIVE_END_DATE
	BEN_PEE_IN_EFFECTIVE_START_DATE
	BEN_PEE_IO_EFFECTIVE_START_DATE
	BEN_PEE_IN_ELEMENT_ENTRY_ID
	BEN_PEE_IO_ELEMENT_ENTRY_ID
	BEN_PEE_IN_ELEMENT_ENTRY_VALUE_ID
	BEN_PEE_IO_ELEMENT_ENTRY_VALUE_ID
	BEN_PEE_IN_ENTRY_USAGE_ID
	BEN_PEE_IO_ENTRY_USAGE_ID
	BEN_PEE_IN_INPUT_VALUE_ID
	BEN_PEE_IO_INPUT_VALUE_ID
	BEN_PEE_IN_SCREEN_ENTRY_VALUE
	BEN_PEE_IO_SCREEN_ENTRY_VALUE
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_PDP_IN_PERSON_ID
	BEN_PDP_IO_PERSON_ID
	BEN_PDP_IN_CVG_PNDG_FLAG
	BEN_PDP_IO_CVG_PNDG_FLAG
	BEN_PDP_IN_CVG_STRT_DT
	BEN_PDP_IO_CVG_STRT_DT
	BEN_PDP_IN_CVG_THRU_DT
	BEN_PDP_IO_CVG_THRU_DT
	BEN_PDP_IN_DPNT_PERSON_ID
	BEN_PDP_IO_DPNT_PERSON_ID
	BEN_PDP_IN_ELIG_PER_ELCTBL_CHC_ID
	BEN_PDP_IO_ELIG_PER_ELCTBL_CHC_ID
	BEN_PDP_IN_ENDED_CVG_THRU_DT
	BEN_PDP_IO_ENDED_CVG_THRU_DT
	BEN_PDP_IN_ENDED_PER_IN_LER_ID
	BEN_PDP_IO_ENDED_PER_IN_LER_ID
	BEN_PDP_IN_OVRDN_FLAG
	BEN_PDP_IO_OVRDN_FLAG
	BEN_PDP_IN_OVRDN_THRU_DT
	BEN_PDP_IO_OVRDN_THRU_DT
	BEN_PDP_IN_PRTT_ENRT_RSLT_ID

	BEN_PDP_IO_PRTT_ENRT_RSLT_ID
	BEN_PDP_IN_PRVS_CVG_STRT_DT
	BEN_PDP_IO_PRVS_CVG_STRT_DT
	BEN_PDP_IN_PRVS_CVG_THRU_DT
	BEN_PDP_IO_PRVS_CVG_THRU_DT
	BEN_PDP_IN_RLNSHP_CD
	BEN_PDP_IO_RLNSHP_CD
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_PBB_IN_PERSON_ID
	BEN_PBB_IO_PERSON_ID
	BEN_PBB_IN_EFFECTIVE_START_DATE
	BEN_PBB_IO_EFFECTIVE_START_DATE
	BEN_PBB_IN_EFFECTIVE_END_DATE
	BEN_PBB_IO_EFFECTIVE_END_DATE
	BEN_PBB_IN_VAL
	BEN_PBB_IO_VAL
	BEN_PBB_IN_BNFTS_BAL_ID
	BEN_PBB_IO_BNFTS_BAL_ID
	BEN_IV_LER_ID
	BEN_PBB_IN_UOM
	BEN_PBB_IO_UOM
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_ADD_IN_ADDRESS_ID
	BEN_ADD_IO_ADDRESS_ID
	BEN_ADD_IN_ADDRESS_TYPE
	BEN_ADD_IO_ADDRESS_TYPE
	BEN_ADD_IN_BUSINESS_GROUP_ID
	BEN_ADD_IO_BUSINESS_GROUP_ID
	BEN_ADD_IN_CREATED_BY
	BEN_ADD_IO_CREATED_BY
	BEN_ADD_IN_EFFECTIVE_END_DATE
	BEN_ADD_IO_EFFECTIVE_END_DATE
	BEN_ADD_IN_EFFECTIVE_START_DATE

	BEN_ADD_IO_EFFECTIVE_START_DATE
	BEN_ADD_IN_LAST_UPDATED_BY
	BEN_ADD_IO_LAST_UPDATED_BY
	BEN_ADD_IN_LAST_UPDATE_LOGIN
	BEN_ADD_IO_LAST_UPDATE_LOGIN
	BEN_ADD_IN_OBJECT_VERSION_NUMBER
	BEN_ADD_IO_OBJECT_VERSION_NUMBER
	BEN_ADD_IN_PERSON_ADDR_USAGE_ID
	BEN_ADD_IO_PERSON_ADDR_USAGE_ID
	BEN_ADD_IN_PERSON_ID
	BEN_ADD_IO_PERSON_ID
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_LEH_IN_PERSON_ID
	BEN_LEH_IO_PERSON_ID
	BEN_LEH_IN_EFFECTIVE_START_DATE
	BEN_LEH_IO_EFFECTIVE_START_DATE
	BEN_LEH_IN_EFFECTIVE_END_DATE
	BEN_LEH_IO_EFFECTIVE_END_DATE
	BEN_LEH_IN_LE_HABITS_COV_ID
	BEN_LEH_IO_LE_HABITS_COV_ID
	BEN_LEH_IN_LEGISLATIVE_DATA_GROUP_ID
	BEN_LEH_IO_LEGISLATIVE_DATA_GROUP_ID
	BEN_IV_LER_ID
	BEN_LEH_IN_DISABILITY_STATUS
	BEN_LEH_IO_DISABILITY_STATUS
	BEN_LEH_IN_DPDNT_ADOPTION_DATE
	BEN_LEH_IO_DPDNT_ADOPTION_DATE
	BEN_LEH_IN_DPDNT_VLNTRY_SVCE_FLAG
	BEN_LEH_IO_DPDNT_VLNTRY_SVCE_FLAG
	BEN_LEH_IN_ON_MILITARY_SERVICE
	BEN_LEH_IO_ON_MILITARY_SERVICE
	BEN_LEH_IN_RECEIPT_OF_DEATH_CERT_DATE
	BEN_LEH_IO_RECEIPT_OF_DEATH_CERT_DATE
	BEN_LEH_IN_REGISTERED_DISABLED_FLAG
	BEN_LEH_IO_REGISTERED_DISABLED_FLAG
	BEN_LEH_IN_STUDENT_STATUS
	BEN_LEH_IO_STUDENT_STATUS

	BEN_LEH_IN_TOBACCO_TYPE_USAGE
	BEN_LEH_IO_TOBACCO_TYPE_USAGE
	BEN_IV_CONTACT_TYPE
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_EGD_IN_INELG_RSN_CD
	BEN_EGD_IO_INELG_RSN_CD
	BEN_EGD_IN_DPNT_INELIG_FLAG
	BEN_EGD_IO_DPNT_INELIG_FLAG
	BEN_EGD_IN_ELIG_STRT_DT
	BEN_EGD_IO_ELIG_STRT_DT
	BEN_EGD_IN_ELIG_THRU_DT
	BEN_EGD_IO_ELIG_THRU_DT
	BEN_EGD_IN_OVRDN_FLAG
	BEN_EGD_IO_OVRDN_FLAG
	BEN_EGD_IN_OVRDN_THRU_DT
	BEN_EGD_IO_OVRDN_THRU_DT
	BEN_EGD_IN_CREATE_DT
	BEN_EGD_IO_CREATE_DT
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_DIS_IN_PERSON_ID
	BEN_DIS_IO_PERSON_ID
	BEN_DIS_IN_ASSESSMENT_DUE_DATE
	BEN_DIS_IO_ASSESSMENT_DUE_DATE
	BEN_DIS_IN_CATEGORY
	BEN_DIS_IO_CATEGORY
	BEN_DIS_IN_DEGREE
	BEN_DIS_IO_DEGREE
	BEN_DIS_IN_DESCRIPTION
	BEN_DIS_IO_DESCRIPTION
	BEN_DIS_IN_EFFECTIVE_END_DATE
	BEN_DIS_IO_EFFECTIVE_END_DATE
	BEN_DIS_IN_EFFECTIVE_START_DATE

	BEN_DIS_IO_EFFECTIVE_START_DATE
	BEN_DIS_IN_INCIDENT_ID
	BEN_DIS_IO_INCIDENT_ID
	BEN_DIS_IN_LEGISLATION_CODE
	BEN_DIS_IO_LEGISLATION_CODE
	BEN_DIS_IN_ORGANIZATION_ID
	BEN_DIS_IO_ORGANIZATION_ID
	BEN_DIS_IN_QUOTA_FTE
	BEN_DIS_IO_QUOTA_FTE
	BEN_DIS_IN_REGISTRATION_DATE
	BEN_DIS_IO_REGISTRATION_DATE
	BEN_DIS_IN_REGISTRATION_EXP_DATE
	BEN_DIS_IO_REGISTRATION_EXP_DATE
	BEN_DIS_IN_REGISTRATION_ID
	BEN_DIS_IO_REGISTRATION_ID
	BEN_DIS_IN_STATUS
	BEN_DIS_IO_STATUS
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_CRD_IN_CRT_ORDR_PL_DPNT_ID
	BEN_CRD_IO_CRT_ORDR_PL_DPNT_ID
	BEN_CRD_IN_BUSINESS_GROUP_ID
	BEN_CRD_IO_BUSINESS_GROUP_ID
	BEN_CRD_IN_CONTACT_PERSON_ID
	BEN_CRD_IO_CONTACT_PERSON_ID
	BEN_CRD_IN_CRT_ORDR_ID
	BEN_CRD_IO_CRT_ORDR_ID
	BEN_CRD_IN_PL_TYP_ID
	BEN_CRD_IO_PL_TYP_ID
	BEN_CRD_IN_PL_ID
	BEN_CRD_IO_PL_ID
	BEN_CRD_IN_RLSHP_TYP_CD
	BEN_CRD_IO_RLSHP_TYP_CD
	BEN_CRD_IN_ORDR_END_DATE
	BEN_CRD_IO_ORDR_END_DATE
	BEN_CRD_IN_ORDR_START_DATE
	BEN_CRD_IO_ORDR_START_DATE

	BEN_CRD_IN_PRVNT_DEENRT_FLAG
	BEN_CRD_IO_PRVNT_DEENRT_FLAG
	BEN_CRD_IN_CRD_ATTRIBUTE_CATEGORY
	BEN_CRD_IO_CRD_ATTRIBUTE_CATEGORY
	BEN_CRD_IN_CRD_ATTRIBUTE1 ..30
	BEN_CRD_IO_CRD_ATTRIBUTE1 ..30
	BEN_CRD_IN_CRD_ATTRIBUTE_NUMBER1 .. 20
	BEN_CRD_IO_CRD_ATTRIBUTE_NUMBER1 .. 20
	BEN_CRD_IN_CRD_ATTRIBUTE_DATE1 .. 15
	BEN_CRD_IO_CRD_ATTRIBUTE_DATE1 .. 15
	BEN_CRD_IN_LAST_UPDATE_DATE
	BEN_CRD_IO_LAST_UPDATE_DATE
	BEN_CRD_IN_LAST_UPDATED_BY
	BEN_CRD_IO_LAST_UPDATED_BY
	BEN_CRD_IN_LAST_UPDATE_LOGIN
	BEN_CRD_IO_LAST_UPDATE_LOGIN
	BEN_CRD_IN_CREATED_BY
	BEN_CRD_IO_CREATED_BY
	BEN_CRD_IN_CREATION_DATE
	BEN_CRD_IO_CREATION_DATE
	BEN_CRD_IN_OBJECT_VERSION_NUMBER
	BEN_CRD_IO_OBJECT_VERSION_NUMBER
	BEN_CRD_IN_CONFIG_ID_1 .. 10
	BEN_CRD_IO_CONFIG_ID_1 .. 10
	BEN_CRD_IN_CONFIG_NUM_1 .. 10
	BEN_CRD_IO_CONFIG_NUM_1 .. 10
	BEN_CRD_IN_CONFIG_CHAR_1 .. 10
	BEN_CRD_IO_CONFIG_CHAR_1 .. 10
	BEN_CRD_IN_CONFIG_DATE_1 .. 10
	BEN_CRD_IO_CONFIG_DATE_1 .. 10
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_CRT_IN_CRT_ORDR_ID
	BEN_CRT_IO_CRT_ORDR_ID
	BEN_CRT_IN_EFFECTIVE_START_DATE
	BEN_CRT_IO_EFFECTIVE_START_DATE
	BEN_CRT_IN_EFFECTIVE_END_DATE

	BEN_CRT_IO_EFFECTIVE_END_DATE
	BEN_CRT_IN_CRT_ORDR_TYP_CD
	BEN_CRT_IO_CRT_ORDR_TYP_CD
	BEN_CRT_IN_CRT_ORDR_STAT_CD
	BEN_CRT_IO_CRT_ORDR_STAT_CD
	BEN_CRT_IN_BUSINESS_GROUP_ID
	BEN_CRT_IO_BUSINESS_GROUP_ID
	BEN_CRT_IN_CONTACT_PERSON_ID
	BEN_CRT_IO_CONTACT_PERSON_ID
	BEN_CRT_IN_CRT_ORDR_ID
	BEN_CRT_IO_CRT_ORDR_ID
	BEN_CRT_IN_PL_TYP_ID
	BEN_CRT_IO_PL_TYP_ID
	BEN_CRT_IN_PL_ID
	BEN_CRT_IO_PL_ID
	BEN_CRT_IN_RLSHP_TYP_CD
	BEN_CRT_IO_RLSHP_TYP_CD
	BEN_CRT_IN_ORDR_END_DATE
	BEN_CRT_IO_ORDR_END_DATE
	BEN_CRT_IN_ORDR_START_DATE
	BEN_CRT_IO_ORDR_START_DATE
	BEN_CRT_IN_PRVNT_DEENRT_FLAG
	BEN_CRT_IO_PRVNT_DEENRT_FLAG
	BEN_CRT_IN_CRD_ATTRIBUTE_CATEGORY
	BEN_CRT_IO_CRD_ATTRIBUTE_CATEGORY
	BEN_CRT_IN_CRD_ATTRIBUTE1 ..30
	BEN_CRT_IO_CRD_ATTRIBUTE1 ..30
	BEN_CRT_IN_CRD_ATTRIBUTE_NUMBER1 .. 20
	BEN_CRT_IO_CRD_ATTRIBUTE_NUMBER1 .. 20
	BEN_CRT_IN_CRD_ATTRIBUTE_DATE1 .. 15
	BEN_CRT_IO_CRD_ATTRIBUTE_DATE1 .. 15
	BEN_CRT_IN_LAST_UPDATE_DATE
	BEN_CRT_IO_LAST_UPDATE_DATE
	BEN_CRT_IN_LAST_UPDATED_BY
	BEN_CRT_IO_LAST_UPDATED_BY
	BEN_CRT_IN_LAST_UPDATE_LOGIN
	BEN_CRT_IO_LAST_UPDATE_LOGIN
	BEN_CRT_IN_CREATED_BY
	BEN_CRT_IO_CREATED_BY
	BEN_CRT_IN_CREATION_DATE
	BEN_CRT_IO_CREATION_DATE

	BEN_CRT_IN_OBJECT_VERSION_NUMBER
	BEN_CRT_IO_OBJECT_VERSION_NUMBER
	BEN_CRT_IN_CONFIG_ID_1 .. 10
	BEN_CRT_IO_CONFIG_ID_1 .. 10
	BEN_CRT_IN_CONFIG_NUM_1 .. 10
	BEN_CRT_IO_CONFIG_NUM_1 .. 10
	BEN_CRT_IN_CONFIG_CHAR_1 .. 10
	BEN_CRT_IO_CONFIG_CHAR_1 .. 10
	BEN_CRT_IN_CONFIG_DATE_1 .. 10
	BEN_CRT_IO_CONFIG_DATE_1 .. 10
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_CRT_IN_PERSON_ID
	BEN_CRT_IO_PERSON_ID
	BEN_CRT_IN_APLS_PERD_ENDG_DT
	BEN_CRT_IN_APLS_PERD_STRTG_DT
	BEN_CRT_IN_CRT_ORDR_ID
	BEN_CRT_IN_CRT_ORDR_TYP_CD
	BEN_CRT_IN_DETD_QLFD_ORDR_DT
	BEN_CRT_IN_ISSUE_DT
	BEN_CRT_IN_PL_ID
	BEN_CRT_IN_PL_TYP_ID
	BEN_CRT_IN_QDRO_AMT
	BEN_CRT_IN_QDRO_DSTR_MTHD_CD
	BEN_CRT_IN_QDRO_NUM_PYMT_VAL
	BEN_CRT_IN_QDRO_PCT
	BEN_CRT_IN_RCVD_DT
	BEN_CRT_IN_UOM
	BEN_CRT_IO_APLS_PERD_ENDG_DT
	BEN_CRT_IO_APLS_PERD_STRTG_DT
	BEN_CRT_IO_CRT_ORDR_ID
	BEN_CRT_IO_CRT_ORDR_TYP_CD
	BEN_CRT_IO_DETD_QLFD_ORDR_DT
	BEN_CRT_IO_ISSUE_DT
	BEN_CRT_IO_PL_ID
	BEN_CRT_IO_PL_TYP_ID
	BEN_CRT_IO_QDRO_AMT

	BEN_CRT_IO_QDRO_DSTR_MTHD_CD
	BEN_CRT_IO_QDRO_NUM_PYMT_VAL
	BEN_CRT_IO_QDRO_PCT
	BEN_CRT_IO_RCVD_DT
	BEN_CRT_IO_UOM
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_CON_IN_BENEFICIARY_FLAG
	BEN_CON_IN_CONTACT_PERSON_ID
	BEN_CON_IN_CONTACT_TYPE
	BEN_CON_IN_DEPENDENT_FLAG
	BEN_CON_IN_EFFECTIVE_END_DATE
	BEN_CON_IN_EFFECTIVE_START_DATE
	BEN_CON_IN_EMERGENCY_CONTACT_FLAG
	BEN_CON_IN_END_LIFE_REASON_ID
	BEN_CON_IN_PERSONAL_FLAG
	BEN_CON_IN_PERSON_ID
	BEN_CON_IN_PRIMARY_CONTACT_FLAG
	BEN_CON_IN_RLTD_PER_RSDS_W_DSGNTR_FLAG
	BEN_CON_IN_START_LIFE_REASON_ID
	BEN_CON_IN_THIRD_PARTY_PAY_FLAG
	BEN_CON_IO_BENEFICIARY_FLAG
	BEN_CON_IO_CONTACT_PERSON_ID
	BEN_CON_IO_CONTACT_TYPE
	BEN_CON_IO_DEPENDENT_FLAG
	BEN_CON_IO_EFFECTIVE_END_DATE
	BEN_CON_IO_EFFECTIVE_START_DATE
	BEN_CON_IO_EMERGENCY_CONTACT_FLAG
	BEN_CON_IO_END_LIFE_REASON_ID
	BEN_CON_IO_PERSONAL_FLAG
	BEN_CON_IO_PERSON_ID
	BEN_CON_IO_PRIMARY_CONTACT_FLAG
	BEN_CON_IO_RLTD_PER_RSDS_W_DSGNTR_FLAG
	BEN_CON_IO_START_LIFE_REASON_ID
	BEN_CON_IO_THIRD_PARTY_PAY_FLAG

	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_BRE_IN_PERSON_ID
	BEN_BRE_IN_EFFECTIVE_START_DATE
	BEN_BRE_IN_EFFECTIVE_END_DATE
	BEN_BRE_IN_BENEFIT_RELATION_ID
	BEN_BRE_IN_LEGAL_ENTITY_ID
	BEN_IV_LER_ID
	BEN_BRE_IN_UPDATED_SOURCE
	BEN_BRE_IN_ORIGINAL_SOURCE
	BEN_BRE_IN_PRIMARY_REL
	BEN_BRE_IN_REL_PRMRY_ASG_ID
	BEN_BRE_IN_STATUS
	BEN_BRE_IO_PERSON_ID
	BEN_BRE_IO_EFFECTIVE_START_DATE
	BEN_BRE_IO_EFFECTIVE_END_DATE
	BEN_BRE_IO_BENEFIT_RELATION_ID
	BEN_BRE_IO_LEGAL_ENTITY_ID
	BEN_BRE_IO_UPDATED_SOURCE
	BEN_BRE_IO_ORIGINAL_SOURCE
	BEN_BRE_IO_PRIMARY_REL
	BEN_BRE_IO_REL_PRMRY_ASG_ID
	BEN_BRE_IO_STATUS
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_BBG_IN_PERSON_ID
	BEN_BBG_IN_EFFECTIVE_START_DATE
	BEN_BBG_IN_EFFECTIVE_END_DATE
	BEN_BBG_IN_BENEFIT_GROUP_ID
	BEN_BBG_IN_LEGISLATIVE_DATA_GROUP_ID
	BEN_IV_LER_ID
	BEN_BBG_IO_PERSON_ID
	BEN_BBG_IO_EFFECTIVE_START_DATE
	BEN_BBG_IO_EFFECTIVE_END_DATE
	BEN_BBG_IO_BENEFIT_GROUP_ID

	BEN_BBG_IO_LEGISLATIVE_DATA_GROUP_ID
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_AWM_IN_PERSON_ID
	BEN_AWM_IN_ACTION_OCCURRENCE_ID
	BEN_AWM_IN_ADDS_TO_BUDGET
	BEN_AWM_IN_ASSIGNMENT_ID
	BEN_AWM_IN_ASSIGN_WORK_MEASURE_ID
	BEN_AWM_IN_BUSINESS_GROUP_ID
	BEN_AWM_IN_EFFECTIVE_END_DATE
	BEN_AWM_IN_EFFECTIVE_START_DATE
	BEN_AWM_IN_UNIT
	BEN_AWM_IN_VALUE
	BEN_AWM_IO_PERSON_ID
	BEN_AWM_IO_ACTION_OCCURRENCE_ID
	BEN_AWM_IO_ADDS_TO_BUDGET
	BEN_AWM_IO_ASSIGNMENT_ID
	BEN_AWM_IO_ASSIGN_WORK_MEASURE_ID
	BEN_AWM_IO_BUSINESS_GROUP_ID
	BEN_AWM_IO_EFFECTIVE_END_DATE
	BEN_AWM_IO_EFFECTIVE_START_DATE
	BEN_AWM_IO_UNIT
	BEN_AWM_IO_VALUE
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_ASG_IN_ACTION_CODE
	BEN_ASG_IN_ASSIGNMENT_NAME
	BEN_ASG_IN_ASSIGNMENT_STATUS_TYPE
	BEN_ASG_IN_BUSINESS_UNIT_ID
	BEN_ASG_IN_DUTIES_TYPE
	BEN_ASG_IN_EMPLOYEE_CATEGORY
	BEN_ASG_IN_LABOUR_UNION_MEMBER_FLAG
	BEN_ASG_IN_PERSON_TYPE_ID
	BEN_ASG_IN_USER_PERSON_TYPE
	BEN_ASG_IN_POSITION_ID

	BEN_ASG_IN_PROBATION_PERIOD
	BEN_ASG_IN_PROBATION_UNIT
	BEN_ASG_IN_WORK_AT_HOME
	BEN_ASG_IN_EFFECTIVE_START_DATE
	BEN_ASG_IN_EFFECTIVE_END_DATE
	BEN_ASG_IN_ORGANIZATION_ID
	BEN_ASG_IN_GRADE_ID
	BEN_ASG_IN_JOB_ID
	BEN_ASG_IN_WORK_TERMS_ASSIGNMENT_ID
	BEN_ASG_IN_LOCATION_ID
	BEN_ASG_IN_ASSIGNMENT_STATUS_TYPE_ID
	BEN_ASG_IN_ASSIGNMENT_TYPE
	BEN_ASG_IN_PRIMARY_FLAG
	BEN_ASG_IN_EMPLOYMENT_CATEGORY
	BEN_ASG_IN_BARGAINING_UNIT_CODE
	BEN_ASG_IN_HOURLY_SALARIED_CODE
	BEN_ASG_IN_NORMAL_HOURS
	BEN_IV_LER_ID
	BEN_ASG_IO_ACTION_CODE
	BEN_ASG_IO_ASSIGNMENT_NAME
	BEN_ASG_IO_ASSIGNMENT_STATUS_TYPE
	BEN_ASG_IO_BUSINESS_UNIT_ID
	BEN_ASG_IO_DUTIES_TYPE
	BEN_ASG_IO_EMPLOYEE_CATEGORY
	BEN_ASG_IO LABOUR UNION_MEMBER_FLAG
	BEN_ASG_IO_PERSON_TYPE_ID
	BEN_ASG_IO_USER_PERSON_TYPE
	BEN_ASG_IO_POSITION_ID
	BEN_ASG_IO_PROBATION_PERIOD
	BEN_ASG_IO_PROBATION_UNIT
	BEN_ASG_IO_WORK_AT_HOME
	BEN_ASG_IO_EFFECTIVE_START_DATE
	BEN_ASG_IO_EFFECTIVE_END_DATE
	BEN_ASG_IO_ORGANIZATION_ID
	BEN_ASG_IO_GRADE_ID
	BEN_ASG_IO_JOB_ID
	BEN_ASG_IO_WORK_TERMS_ASSIGNMENT_ID
	BEN_ASG_IO_LOCATION_ID
	BEN_ASG_IO_ASSIGNMENT_STATUS_TYPE_ID
	BEN_ASG_IO_ASSIGNMENT_TYPE
	BEN_ASG_IO_PRIMARY_FLAG

	BEN_ASG_IO_EMPLOYMENT_CATEGORY
	BEN_ASG_IO_BARGAINING_UNIT_CODE
	BEN_ASG_IO_HOURLY_SALARIED_CODE
	BEN_ASG_IO_NORMAL_HOURS
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_AEI_IN_PERSON_ID
	BEN_AEI_IN_ACTION_OCCURRENCE_ID
	BEN_AEI_IN_ASSIGNMENT_EXTRA_INFO_ID
	BEN_AEI_IN_ASSIGNMENT_ID
	BEN_AEI_IN_EFFECTIVE_END_DATE
	BEN_AEI_IN_EFFECTIVE_LATEST_CHANGE
	BEN_AEI_IN_EFFECTIVE_SEQUENCE
	BEN_AEI_IN_EFFECTIVE_START_DATE
	BEN_AEI_IN_LEGISLATION_CODE
	BEN_AEI_IO_PERSON_ID
	BEN_AEI_IO_ACTION_OCCURRENCE_ID
	BEN_AEI_IO_ASSIGNMENT_EXTRA_INFO_ID
	BEN_AEI_IO_ASSIGNMENT_ID
	BEN_AEI_IO_EFFECTIVE_END_DATE
	BEN_AEI_IO_EFFECTIVE_LATEST_CHANGE
	BEN_AEI_IO_EFFECTIVE_SEQUENCE
	BEN_AEI_IO_EFFECTIVE_START_DATE
	BEN_AEI_IO_LEGISLATION_CODE
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_ADD_IN_ADDRESS_ID
	BEN_ADD_IN_ADDRESS_TYPE
	BEN_ADD_IN_COUNTRY
	BEN_ADD_IN_COUNTRY
	BEN_ADD_IN_POSTAL_CODE
	BEN_ADD_IN_REGION_1 .. 3
	BEN_ADD_IN_TOWN_OR_CITY

	BEN_ADD_IN_BUSINESS_GROUP_ID
	BEN_ADD_IN_EFFECTIVE_END_DATE
	BEN_ADD_IN_EFFECTIVE_START_DATE
	BEN_ADD_IO_ADDRESS_ID
	BEN_ADD_IO_ADDRESS_TYPE
	BEN_ADD_IO_COUNTRY
	BEN_ADD_IO_COUNTRY
	BEN_ADD_IO_POSTAL_CODE
	BEN_ADD_IO_REGION_1..3
	BEN_ADD_IO_TOWN_OR_CITY
	BEN_ADD_IO_BUSINESS_GROUP_ID
	BEN_ADD_IO_EFFECTIVE_END_DATE
	BEN_ADD_IO_EFFECTIVE_START_DATE
	NEW_VAL
	OLD_VAL
	PK_ID
	COLUMN
	BEN_ABA_IN_PER_ABSENCE_ENTRY_ID
	BEN_ABA_IO_PER_ABSENCE_ENTRY_ID
	BEN_ABA_IO_PERIOD_OF_SERVICE_ID
	BEN_ABA_IN_PERIOD_OF_SERVICE_ID
	BEN_ABA_IO_ABSENCE_CASE_ID
	BEN_ABA_IN_ABSENCE_CASE_ID
	BEN_ABA_IO_SINGLE_DAY_FLAG
	BEN_ABA_IN_SINGLE_DAY_FLAG
	BEN_ABA_IO_START_DATE_DURATION
	BEN_ABA_IN_START_DATE_DURATION
	BEN_ABA_IO_END_DATE_DURATION
	BEN_ABA_IN_END_DATE_DURATION
	BEN_ABA_IO_ABSENCE_PATTERN_CD
	BEN_ABA_IN_ABSENCE_PATTERN_CD
	BEN_ABA_IO_ENTERPRISE_ID
	BEN_ABA_IN_ENTERPRISE_ID
	BEN_ABA_IO_LEGISLATIVE_DATA_GROUP_ID
	BEN_ABA_IN_LEGISLATIVE_DATA_GROUP_ID
	BEN_ABA_IO_LEGAL_ENTITY_ID
	BEN_ABA_IN_LEGAL_ENTITY_ID
	BEN_ABA_IO_PERSON_ID
	BEN_ABA_IN_PERSON_ID
	BEN_ABA_IO_ABSENCE_TYPE_ID

	BEN_ABA_IN_ABSENCE_TYPE_ID
	BEN_ABA_IN_ABSENCE_TYPE
	BEN_ABA_IO_ABSENCE_TYPE
	BEN_ABA_IO_CHILD_EVENT_TYPE_CD
	BEN_ABA_IN_CHILD_EVENT_TYPE_CD
	BEN_ABA_IO_ABSENCE_TYPE_REASON_ID
	BEN_ABA_IN_ABSENCE_TYPE_REASON_ID
	BEN_ABA_IO_ABSENCE_ENTRY_BASIC_FLAG
	BEN_ABA_IN_ABSENCE_ENTRY_BASIC_FLAG
	BEN_ABA_IO_ABSENCE_STATUS_CD
	BEN_ABA_IN_ABSENCE_STATUS_CD
	BEN_ABA_IO_APPROVAL_STATUS_CD
	BEN_ABA_IN_APPROVAL_STATUS_CD
	BEN_ABA_IO_START_DATE
	BEN_ABA_IN_START_DATE
	BEN_ABA_IO_START_DATETIME
	BEN_ABA_IN_START_DATETIME
	BEN_ABA_IO_END_DATETIME
	BEN_ABA_IN_END_DATETIME
	BEN_ABA_IO_END_DATE
	BEN_ABA_IN_END_DATE
	BEN_ABA_IO_START_TIME
	BEN_ABA_IN_START_TIME
	BEN_ABA_IO_END_TIME
	BEN_ABA_IN_END_TIME
	BEN_ABA_IO_PLANNED_END_DATE
	BEN_ABA_IN_PLANNED_END_DATE
	BEN_ABA_IO_OPEN_ENDED_FLAG
	BEN_ABA_IN_OPEN_ENDED_FLAG
	BEN_ABA_IO_CONFIRMED_DATE
	BEN_ABA_IN_CONFIRMED_DATE
	BEN_ABA_IO_SUBMITTED_DATE
	BEN_ABA_IN_SUBMITTED_DATE
	BEN_ABA_IO_CONDITION_START_DATE
	BEN_ABA_IN_CONDITION_START_DATE
	BEN_ABA_IO_DURATION
	BEN_ABA_IN_DURATION
	BEN_ABA_IO_UOM
	BEN_ABA_IN_UOM
	BEN_ABA_IO_COMMENTS
	BEN_ABA_IN_COMMENTS

	BEN_ABA_IO_CREATED_BY
	BEN_ABA_IN_CREATED_BY
	BEN_ABA_IO_PROJECT_ID
	BEN_ABA_IN_PROJECT_ID
	BEN_ABA_IO_CREATION_DATE
	BEN_ABA_IN_CREATION_DATE
	BEN_ABA_IO_LAST_UPDATED_BY
	BEN_ABA_IN_LAST_UPDATED_BY
	BEN_ABA_IO_LAST_UPDATE_DATE
	BEN_ABA_IN_LAST_UPDATE_DATE
	BEN_ABA_IN_LAST_UPDATED_BY
	BEN_ABA_IO_LAST_UPDATE_DATE
	BEN_ABA_IN_LAST_UPDATE_DATE
	BEN_ABA_IO_LAST_UPDATE_LOGIN
	BEN_ABA_IN_LAST_UPDATE_LOGIN
	BEN_ABA_IO_OBJECT_VERSION_NUMBER
	BEN_ABA_IN_OBJECT_VERSION_NUMBER
	BEN_ABA_IO_ATTRIBUTE_CATEGORY
	BEN_ABA_IN_ATTRIBUTE_CATEGORY
	BEN_ABA_IO_ATTRIBUTE1 .. 30
	BEN_ABA_IN_ATTRIBUTE1 .. 30
	BEN_ABA_IO_ATTRIBUTE_NUMBER1 .. 12
	BEN_ABA_IN_ATTRIBUTE_NUMBER1 .. 12
	BEN_ABA_IO_ATTRIBUTE_DATE1.. 10
	BEN_ABA_IN_ATTRIBUTE_DATE1..10
	BEN_ABA_IO_INFORMATION_CATEGORY
	BEN_ABA_IN_INFORMATION_CATEGORY
	BEN_ABA_IO_INFORMATION1..30
	BEN_ABA_IN_INFORMATION1..30
	BEN_ABA_IO_INFORMATION_NUMBER1 .. 20
	BEN_ABA_IN_INFORMATION_NUMBER1 .. 20
	BEN_ABA_IO_INFORMATION_DATE1 .. 15
	BEN_ABA_IN_INFORMATION_DATE1 .. 15
	BEN_ABA_IO_SPL_CONDITION
	BEN_ABA_IN_SPL_CONDITION
	BEN_ABA_IO_INITIAL_REPORT_BY_ID
	BEN_ABA_IN_INITIAL_REPORT_BY_ID
	BEN_ABA_IO_NOTIFICATION_DATE
	BEN_ABA_IN_NOTIFICATION_DATE
	BEN_ABA_IO_LATE_NOTIFY_FLAG
	BEN_ABA_IN_LATE_NOTIFY_FLAG

	BEN_ABA_IO_INITIAL_TIMELY_NOTIFY_FLAG
	BEN_ABA_IN_INITIAL_TIMELY_NOTIFY_FLAG
	BEN_ABA_IO_CERTIFICATION_AUTH_FLAG
	BEN_ABA_IN_CERTIFICATION_AUTH_FLAG
	BEN_ABA_IO_AUTH_STATUS_UPDATE_DATE
	BEN_ABA_IN_AUTH_STATUS_UPDATE_DATE
	BEN_ABA_IO_TIMELINESS_OVERRIDE_DATE
	BEN_ABA_IN_TIMELINESS_OVERRIDE_DATE
	BEN_ABA_IO_DISEASE_CODE
	BEN_ABA_IN_DISEASE_CODE
	BEN_ABA_IO_PERIOD_OF_INCAP_TO_WORK_FLAG
	BEN_ABA_IN_PERIOD_OF_INCAP_TO_WORK_FLAG
	BEN_ABA_IO_ESTABLISHMENT_DATE
	BEN_ABA_IN_ESTABLISHMENT_DATE
Person Selection	BEN_IV_PERSON_ID
	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
Rate End Date	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
Rate Lower Limit	
	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
Rate Upper Limit	
	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
To Be Sent Date	VALUE
	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
Variable Rate Add On Calculation	BEN_IV_RT_STRT_DT
	BEN_IV_CVG_STRT_DT
	BEN_IV_PERSON_ID
	BEN_IV_EFFECTIVE_DATE

Benefit Function - BEN_FN_GET_CHAR_VALUE

BEN_FN_GET_CHAR_VALUE

In this session, we will cover the usages of Benefit Function,

BEN_FN_GET_CHAR_VALUE.

Since we have already discussed about the basic functionality of the Fast Formula Functions, Contexts and Parameters in one of [our previous session](#), we are not going to discuss in detail about that. If you do not understand the basic functionality of Fast Formula Function, we recommend you to read [our previous session](#) before starting here.

As of today, the following information is collected from different sources. We always recommend you to contact the Oracle Support to get the accurate and updated information. If you think any of the information is not up to date or accurate, please let us know, we will correct the mistakes.

Since this function, **BEN_FN_GET_CHAR_VALUE** is one of the important and complicated functions of Benefit module; we would like to explain the functionality here in detail.

This function, **BEN_FN_GET_CHAR_VALUE** helps to extract a data from a column and it supports 3 Context and 6 Parameters to extract the value.

The contexts are:

- 1) *HR Assignment*
- 2) *Effective Date*
- 3) *Business Group ID.*

Since these Contexts are passed internally and customer has no control over them, we don't want to discuss here the details. The only thing that the user needs to understand is, this function cannot be used unless the Fast Formula Type supports all these Contexts.

For example, none of the Compensation Formula Types supports Business Group Id as context and therefore, this Function cannot be used in Compensation Fast Formula types.

In case, if you want to use this function in one of the Compensation Fast Formula, you need to define a (child) Fast Formula in BEN type and call the (child) Fast Formula from Compensation Fast Formula by passing Business Group Id as context.

The parameters of BEN_FN_GET_CHAR_VALUE:

- 1) *TABLE NAME*
- 2) *COLUMN NAME*
- 3) *PLAN NAME*
- 4) *OPTION NAME*
- 5) *KEY NAME*
- 6) *KEY VALUE*

these Parameters names are provided here just for reference. Since the user does not use this Parameter name anywhere in the formula, please remember them as Parameter 1 to Parameter 6.

1) Parameter 1: The first parameter is a required parameter and is reserved for the Table Name. Please use the Table name in upper case.

2) Parameter 2: This parameter is a required parameter. This is reserved for Column Name. In case you do not need to pass the Column Name (For example, the Table, 'BEN_CVRD_DPNT_CTFN_PRVDD' does not need a Column Name as it is returning 'Y' or 'N') you need to pass either a 'NULL' value or any dummy value.

3) Parameter 3: This is an optional parameter. This parameter is mostly used for Plan Name but in some cases, the parameter can be used for other values.

In some cases, you do not need this value.

In some cases, you need to pass a hard coded value. For example,

BEN_FN_GET_CHAR_VALUE ('BEN_PRTT_RT_VAL', 'AGG_RT_VAL', 'ACTIVITY_BASE_RATE').

In some other cases, you do not need a value for this parameter but need to pass value for supporting subsequent parameters. For example you need to pass values for parameter 1, 2, 5 and 6. In this case you can pass 'NULL' as a value to this and 4th parameter. *BEN_FN_GET_CHAR_VALUE ('PAY_TIME_PERIODS', 'PAYROLL_TYPE', 'NULL', 'NULL', 'START_DATE', '2017/01/01').*

4) Parameter 4: This is an optional parameter. This parameter is used for Option Name. It is not necessary this parameter is always used for Option Name. In some cases, you do not need this value, in some cases, you need to pass a hard coded value. In some other cases, you need to pass 'NA' to support any option name. In some other cases, you do not need a value for this parameter but need to pass value for subsequent parameters. For example you need to pass values for parameter 1, 2, 5 and 6. In this case you can pass 'NULL' as a value to this and 3rd parameter.

5) Parameter 5: This is an optional parameter. This is an extra parameter. This parameter is generally passed as name parameter along with 6th parameter as name and value pair.

6) Parameter 6: This is an optional parameter. This is usually passed along with 5th parameter. This parameter generally passed as value parameter of name and value pair.

When there is no value found, the function returns 'NO_DATA_FOUND'. There are some exceptions to that where the function is able to location the Plan Name and Option Name but there is no data for the Assignment or for the 5th and 6th parameter, then the function returns 'N' as return value.

Table Name	Column Name	Description
PER_JOB_EXTRA_INFO_F	JEI_ATTRIBUTE1 .. 30	Data from Job Extra information table is extracted for information Type 'JOB_KFF' for the current assignment and the effective date. Only required parameters are Table Name as the first parameter and Column Name as the second parameter
	JEI_INFORMATION1 .. 30	
PER_ALL_ASSIGNMENTS_F	All ATTRIBUTE Columns	To extract value from this table, all you need to pass Table Name as the first parameter and Column Name as the second parameter
	ACTION_CODE	
	EXPENSE_CHECK_ADDRESS	
	INTERNAL_BUILDING	
	INTERNAL_FLOOR	
	INTERNAL_LOCATION	
	INTERNAL_MAILSTOP	
	INTERNAL_OFFICE_NUMBER	

	PRIMARY_ASSIGNMENT_FLAG	
	PRIMARY_FLAG	
	PRIMARY_WORK_RELATION_FLAG	
	PRIMARY_WORK_TERMS_FLAG	
	REASON_CODE	
	RETIREMENT_AGE	
	RETIREMENT_DATE	
	SYSTEM_PERSON_TYPE	
BEN_PRTT_ENRT_RSLT	ADMIN_CATEGORY_CD	The data is extracted for an assignment, effective date, Plan Name and Option Name. Plan Name and Option Name are required columns. In case, if you do not want to pass the Option Name you can use 'NA' to avoid the Option Name validation. If you are passing wrong Plan Name or Option Name then the function will return 'NO_DATA_FOUND' If there is no data found in Enrollment Result for the Assignment, Effective Date, Plan Name and Option Name this function will return 'N' The parameters are: 1) Table Name 2) Column Name 3) Plan Name 4) Option Name
	BNFT_AMT	
	BNFT_NNMNTRY_UOM	
	BNFT_TYP_CD	
	COMP_LVL_CD	
	ENROLLED	
	ENRT_CVG_STRT_DT	
	ENRT_CVG_THRU_DT	
	MPTD_INCM_CALC_CD	
	PRTT_ENRT_RSLT_ID	
	RPLCS_SSPNDD_RSLT_ID	
	SS_CATEGORY_CD	
	SSPNDD_FLAG	
	SVNGS_PLN_FLAG	
	UOM	

	ORGNL_ENRT_DT	
	ERLST_DEENRT_DT	
	ENRT_OVRID_THRU_DT	
	INTERIM_FLAG	
	ENRT_OVRIDN_FLAG	
BEN_ELIG_CVRD_DPNT	CVG_STRT_DT	All the conditions for table BEN_PRTT_ENRT_RSLT' also applied to this table. Along with the parameter Table Name, Column Name, Plan Name, optional (NA) Option Name, you need to pass the dependent full name as the 5th parameter. The dependent name is validated in LIKE condition; even partial name can be passed. Please note the partial name may not guarantee information of correct dependent.
	CVG_THRU_DT	
	DPNT_PERSON_ID	
	ENROLLED	
	RLNSHP_CD	
BEN_CVRD_DPNT_CTFN_PRVDD		Along with Table Name, Plan Name, Option Name and dependent name, you need to pass certification name as 6th parameter. The function either return 'Y' or 'N'. If there is any issue with your parameters like Plan Name, you get 'NO_DATA_FOUND' as result. Since there is no Column Name required and it is a 2nd parameter, you can pass a dummy value or a

		NULL' value to the Column Came.
CMP_SALARY_COMPONENTS	COMPONENT_APPROVED	For This table, along with Table and Column Name, you need to pass 5th parameter as COMPONENT_REASON_CODE' and 6th parameter as Component Reason code. Please note it is a code not the name. So you need to pass Table Name, Column Name (3rd and 4th parameter), any dummy value or NULL' value .
	COMPONENT_REASON_CODE	
	CHANGE_AMOUNT	
	CHANGE_PERCENTAGE	
PER_CONTACT_RELSHIPS_F	DAUGHTER_COUNT	This is very similar to table CMP_SALARY_COMPONENTS'. For this table you need to pass 5th parameter as LEGISLATION_CODE' and a Legislation Code for 6th parameter. In summary, Table Name, Column Name, any dummy value or NULL' for 3rd and 4th parameter, LEGISLATION_CODE' and a Legislation Code for 5th and 6th parameter.
	OTHERS_COUNT	
	SON_COUNT	
	SPOUSE_COUNT	
	DP_COUNT	
	DP_MALE_COUNT	
	DP_FEMALE_COUNT	
	ADOPTED_CHILD_COUNT	
	STEP_CHILD_COUNT	
	FOSTER_CHILD_COUNT	

BEN_PER_LE_HABITS_COV_F	COORD_MED_CVG_END_DT	All you need to pass is Table Name and Column Name to get a value from this table.
	COORD_MED_CVG_STRT_DT	
	COORD_MED_EXT_ER	
	COORD_MED_INSR_CRR_IDENT	
	COORD_MED_INSR_CRR_NAM	
	COORD_MED_PL_NAME	
	COORD_MED_PLN_NO	
	COORD_NO_CVG_FLAG	
	CVRD_IN_ANTHR_PL	
	DISABILITY_STATUS	
	DPDNT_ADOPTION_DATE	
	DPDNT_VLNTRY_SVCE_FLAG	
	ON_MILITARY_SERVICE	
	RECEIPT_OF_DEATH_CERT_DATE	
	REGISTERED_DISABLED_FLAG	
	STUDENT_STATUS	
	TOBACCO_TYPE_USAGE	
PER_JOB_LEG_F	LEGISLATION_CODE	This table needs all the parameters along with the Table and Column Name. The 3rd parameter must be 'JOB_ID' and 4th parameter can be any value or a 'NULL' value, the 5th parameter must be INFORMATION_CATEGORY' and the 6th parameter must be Information Category.
	INFORMATION_CATEGORY	
	PER_JOB_LEG_F .. 30	
	ATTRIBUTE_CATEGORY	
	ATTRIBUTE1 .. 30	
	INFORMATION_NUMBER1 .. 20	

	INFORMATION_DATE1 .. 15	
	ATTRIBUTE_NUMBER1 .. 20	
	ATTRIBUTE_DATE1 .. 15	
CMP_SALARY	SALARY_BASIS_CODE	All you need to pass is Table name and Column Name to get a value from this table.
	SALARY_BASIS_ID	
	SALARY_ANNUALIZATION_FACTOR	
	GRADE_RATE_ID	
	COMPONENT_USAGE	
	NAME	
	DATE_FROM	
	DATE_TO	
	SALARY_AMOUNT	
	ACTION_ID	
	ACTION_REASON_ID	
PER_JOB_LEG_F	LEGISLATION_CODE	This table expects all the parameters along with Table name and Column Name.
	INFORMATION_CATEGORY	
	INFORMATION1 .. 30	The 3rd parameter must be 'JOB_ID', 4th parameter is the job id, please note it is id not the name. 5th parameter is 'INFORMATION_CATEGORY' and the sixth parameter is information Category.
	ATTRIBUTE1 .. 30	
	INFORMATION_NUMBER1 .. 20	
	INFORMATION_DATE1 .. 15	

	ATTRIBUTE_NUMBER1 .. 20	
	ATTRIBUTE_DATE1 .. 15	
BEN_BENEFIT_RELATIONS_F	BENEFIT_RELATION_ID	This table needs all the parameter along with Table and Column Name. The 3rd parameter and 4th parameter can be any dummy value or 'NULL' Value. It is not significant. The 5th parameter must be either 'BENEFIT_RELATION_NAME' or 'BENEFIT_REL_SYSTEM_CD'. If the 5th parameter is 'BENEFIT_RELATION_NAME' then the 6th parameter must be Benefit Relationship name. If the 5th parameter is 'BENEFIT_REL_SYSTEM_CD' then the 6th parameter must be Benefit Relation System code.
	EFFECTIVE_START_DATE	
	EFFECTIVE_END_DATE	
	BENEFIT_RELATION_NAME	
	BENEFIT_REL_SYSTEM_CD	
	PRIMARY_REL	
	STATUS	
	ORIGINAL_SOURCE	
	UPDATED_SOURCE	
	LEGAL_ENTITY_ID	
	PERSON_ID	
	REL_PRMRY_ASG_ID	
	BRN_ATTRIBUTE_CATEGORY	
	BRN_ATTRIBUTE1 .. 30	
	BRN_ATTRIBUTE_NUMBER1 .. 20	
	BRN_ATTRIBUTE_DATE1 .. 15	
BEN_PER_IN_LER	PER_IN_LER_ID	This table needs all the parameter along with Table and Column Name.
	BENEFIT_RELATION_ID	
	LF_EVT_OCRD_DT	

	PROCD_DT	The 3rd parameter and 4th parameter can be any dummy value or 'NULL' value. It is not significant. The 5th parameter must be BENEFIT_RELATION_NAME'. The 6th parameter must be Benefit Relationship name.
	STRTD_DT	
	VOIDD_DT	
	PERSON_ID	
	BCKT_DT	
	CLSD_DT	
	NTFN_DT	
	LEGAL_ENTITY_ID	
	BENEFIT_REL_SYSTEM_CD	
	PRIMARY_REL	
	REL_PRMRY_ASG_ID	
	NAME	
	LER_ID	
PAY_REL_GROUPS_DN	PAYROLL_ASSIGNMENT_ID	This table needs all the parameter along with Table and Column Name. The 3rd parameter and 4th parameter is not significant. The 5th parameter must be either GROUP_TYPE' or no value. The 6th parameter can be a group type code. If the 5 th parameter is not passed, the function uses 'A' as default group type, there is no need for 6 th parameter.
	PAYROLL_TERM_ID	
	PAYROLL_RELATIONSHIP_ID	
	TERM_ID	
	LEGAL_EMPLOYER_ID	
	ASSIGNMENT_ID	

PAY_ELEMENT_ENTRIES_F	AGGREGATE	This Table and Column fetches the sum of element entry values. The 3rd and 4th parameters are required and the 5th and 6th parameters are optional. If the 3rd parameter is Element Name then the 4th parameter is input value name. 5th parameter, From date and 6th parameter, End Date are optional. The dates must be in YYYY/MM/DD format. If the dates are not passed, effective date will be used as the default date.
BEN_PRTT_RT_VAL	AGG_RT_VAL AGG_CMCD_RT_VAL	Beside the Table and column name you need to pass the third parameter as either a plan name or 'ACTIVITY_BASE_RATE' If the third parameter is a plan name, you can pass the fourth parameter as option name or 'NA' for any Option. You can pass optional From Date and To Date as 5th and 6th parameters. The dates must be in YYYY/MM/DD format. If you are not passing any dates, the effective date will be used as the default date. If the 3rd parameter is 'ACTIVITY_BASE_RATE', then you need to pass the option name as 4th parameter and optional From Date and To Date as the last 2 parameters

PAY_DIR_CARDS	TRU	Table name and column names are the only parameters need for this table.
	CARD_ID	
PAY_TIME_PERIODS	PAYROLL_ID	Along with Table and Column Name you need to pass the 5th parameter as 'START_DATE' and the 6th parameter is a date value in YYYY/MM/DD format for the Start Date. Since you may not pass the 5th parameter without 3rd and 4th, you can pass any dummy value.
	PAYROLL_TYPE	
	PERIOD_NUM	
	PERIOD_NAME	
	START_DATE	
	END_DATE	
	CUT_OFF_DATE	
	DEFAULT_PAYDATE	
	TOTAL_TP_IN_CALENDAR_YEAR	

How to debug a Fast Formula

How to Debug the Fast Formula

There are many ways the Fast Formula can be debugged. The different teams that use Fast formula provided different method to debug the formula. We try to go through the most commonly used methods to debug the Fast Formula.

Some of these methods may be obsolete or not available in your version. Please confirm with your support/document about the usage of these methods.

ESS_LOG_WRITE :

Syntax: l_var = ESS_LOG_WRITE('Test Fast Formula')

This is the Most commonly used method to debug the fast formula. The output of this method is written in the ESS Log. The limitation of this method is that the method is not useful unless the Fast Formula is processed by the ESS. In other word if the Fast Formula is executed from the UI, this method cannot create any logs.

How to see these logs:

Every team provides an option see the ESS progress (Monitor Page). The navigation path may differ for each team. We take the Compensation page as example to show how the ESS logs can be viewed.

In Compensation Home Page, chose the Run Batch Process.

In the Run Batch Process page, Click on Monitor Process button.

In the monitor page, we turn Hierarchy flag on, which is our preferred way of seeing the process but it is up to your preference. The Fast Formula are processed by Child Process in Compensation, it may be different in other process. Select the child process. You can see the details in the end of the screen; you may need to go to the end, in case if you have too many processes. In the End you see the attachment as ESS_L_99999.

In some versions, you need to select the process and click the View Log button to see the log file.

HR_TRACE/HRTRACE:

Syntax: l_var = HR_TRACE ('Test Fast Formula') or l_var = HRTRACE ('Test Fast Formula')

HR_TRACE is the method name and HTRACE is the alias for the method. You can use either the name to execute this method. This method works only if you have setup a PLSQL trace on. This will create an output File as you define in your debug setup. This debug setup also enables the debug for entire PL/SQL process. This method returns 'TRUE' if the log is written otherwise 'FALSE'. The limitation of this method is, you need to enable the PL/SQL debugger and needs to have DB structure access to retrieve the output file or you need to know someone who can do that for you. If you do not know how to turn on the debugger please talk to your support to provide the document.

PAY_INTERNAL_LOG_WRITE:

Syntax: l_var = PAY_INTERNAL_LOG_WRITE(:Test Fast Formula').

This method is provided by Payroll, it is an alternative to ESS_LOG_WRITE. This is available only for payroll process. The payroll document describes the method as *'If your formulas are executed from the Payroll Run you can set the F flag for Payroll debug and make PAY_INTERNAL_LOG_WRITE formula function calls to output the messages to the ESS log file.'*

DEBUG:

Syntax: l_var = debug(Test Fast Formula').

This is a clone of HR_TRACE.

US_FORMULA_LOGGING/CA_FORMULA_LOGGING:

Syntax: l_var = XX_FORMULA_LOGGING ('CODE', 'Test Fast Formula').

These methods accepts two parameters, code and message. The code is expected to be more than three characters.

For Simple User, this is same as HR_TRACE,

ADD_RLOG:

Syntax: l_num = ADD_RLOG (ff_id,rule_id, 'Test Fast Formula').

This method is provided by Oracle Time and Labor (OTL) Team. This method accepts three parameters, Formula Id, Rule ID and the message. The logs are written in to one of their table. The method returns the number 1.

You can get the ff_id and rule_id from the context HWM_FFS_ID, and HWM_RULE_ID

ADD_LOG

Syntax: l_num = ADD_LOG (ff_id, 'Test Fast Formula').

This is a subset of ADD_RLOG method. The Rule_id is internally determined by this method. The functionality is same as ADD_RLOG.

How to See OTL Logs:

From Navigator, select 'Workforce Management' > 'Time Management'

From Time Management, select 'Analyze Rule Processing Details'

On search screen, select Rule set name, click search

On search results, click on Time card Processing Id to view detail

On Analyze Rule Processing Details page you can look at the Rule processing log.

-----*****-----

Common Issues faced by Compensation Fast Formula Developers

Common Issues

We found that many Fast Formula users face the same issues again and again. We thought that we can put together some guidelines that could help them in solving their issues.

Issue	Description
Working with multi Assignments	When an employee transferred in a period or terminated and rehired in the same period. The Fast Formula needs to deal with two different assignment of the person.

Working with multi Assignments

As we described earlier in our previous blogs, the Fast Formula is either executed for a person (TCS, Primary assignment is used internally) or for an Assignments (CWB, GSP). Some time in real business cases, the Fast Formula needs to deals with more than one assignment. For example, when a person is transferred or a person is terminated and hired in the same period where the Fast Formula is executed, the Fast Formula needs to works with both these assignment to collect the information. Since the Fast Formula is executed with an Assignment Id on an effective date, the Fast Formula developer needs to create their own logic to switch between the assignments.

Considering these issues, Compensation started passing an array of Assignment Id, Term Id, Payroll Assignment Id, Payroll Term Id and Payroll Relationship Id as input values to Fast Formula from Future Release (Please check with Oracle about the availability).

The input values are:

- *CMP_IVR_ASSIGNMENT_ID*
- *CMP_IVR_TERM_ID*
- *CMP_IVR_PAY_ASSIGNMENT_ID*
- *CMP_IVR_PAY_TERM_ID*
- *CMP_IVR_PAY_RELATION_ID*

In case, the Compensation is not able to find related information for the Assignment, it will pass -1 in other arrays to make sure all these arrays are passed with equal numbers of rows.

R11: Since we do not have this array input values support in R11, we also provide an example that uses the existing array DBIs to get the same result.

For both the examples, we use the same hypothetical example to show the logic.

The formula is developed using TCS Item Fast Formula but the same Fast Formula can be used in any Compensation Fast Formula types with minor changes.

These Fast formulas go through all the primary employee assignments to get their salary.

```
/*  
Name: TCS_RXX_MULTI_ASG  
Type : Total Compensation Item  
*/
```

```
/*Declare DBI*/  
DEFAULT FOR CMP_ASSIGNMENT_SALARY_AMOUNT IS 0  
DEFAULT FOR PER_ASG_ASSIGNMENT_TYPE IS 'N'  
DEFAULT FOR PER_ASG_PRIMARY_FLAG IS 'N'  
DEFAULT FOR PER_ASG_REL_DATE_START IS '1900/01/01 00:00:00'(date)
```

```
/*OUTPUT VALUES */  
DEFAULT FOR COMPENSATION_DATES IS EMPTY_TEXT_NUMBER
```

DEFAULT FOR VALUES is EMPTY_TEXT_NUMBER
DEFAULT FOR ASSIGNMENTS is EMPTY_TEXT_NUMBER
DEFAULT FOR LEGALEMPLOYERS is EMPTY_TEXT_NUMBER

/* Input Values */
DEFAULT for CMP_IVR_ASSIGNMENT_ID is EMPTY_NUMBER_NUMBER
INPUTS ARE CMP_IV_PERSON_ID , CMP_IV_PERIOD_START_DATE (DATE),
CMP_IV_PERIOD_END_DATE (DATE), CMP_IVR_ASSIGNMENT_ID (NUMBER_NUMBER) ,
CMP_IVR_TERM_ID (NUMBER_NUMBER), CMP_IVR_PAY_ASSIGNMENT_ID (NUMBER_NUMBER),
CMP_IVR_PAY_TERM_ID (NUMBER_NUMBER), CMP_IVR_PAY_RELATION_ID (NUMBER_NUMBER)

L_data= ESS_LOG_WRITE('ENTERING TCS_R12_MULTI_ASG ')
L_counter = 1
index = CMP_IVR_ASSIGNMENT_ID.FIRST(-1)
L_data= ESS_LOG_WRITE('first index ' + TO_CHAR(index))

L_PERSON_ID = GET_CONTEXT (PERSON_ID, -1)
L_ASG_ID = GET_CONTEXT (HR_ASSIGNMENT_ID, -1)
L_TOT_COUNT = CMP_IVR_ASSIGNMENT_ID.COUNT
L_data= ESS_LOG_WRITE(' NO OF ASSIGNMENT ' + TO_CHAR(L_TOT_COUNT))
L_data= ESS_LOG_WRITE(' PERSON_ID ' + TO_CHAR(L_PERSON_ID))
L_data= ESS_LOG_WRITE(' ASG_ID ' + TO_CHAR(L_ASG_ID))
L_data= ESS_LOG_WRITE(' Start Date ' + TO_CHAR(CMP_IV_PERIOD_START_DATE))
L_data= ESS_LOG_WRITE(' End Date ' + TO_CHAR(CMP_IV_PERIOD_END_DATE))

/* Looping through the assignment array */
WHILE (CMP_IVR_ASSIGNMENT_ID.EXISTS(index))

```

LOOP (
  I_data= ESS_LOG_WRITE( ' Assignment in Array ' + TO_CHAR( CMP_IVR_ASSIGNMENT_ID[index]) )
  /* Though this formul ahs nothing to do with payroll data, we are just printing the data
     This will help the user who wantsto user payroll data
  */
  I_data= ESS_LOG_WRITE( 'TERM ID   ' + TO_CHAR(CMP_IVR_TERM_ID[index]) )
  I_data= ESS_LOG_WRITE( 'Pay Asg ID   ' + TO_CHAR(CMP_IVR_PAY_ASSIGNMENT_ID[index]) )
  I_data= ESS_LOG_WRITE( 'Pay Asg Rel ID' + TO_CHAR(CMP_IVR_PAY_RELATION_ID[index]) )

  /*Changing the context of assinment so that we can get asg related data from DBI
     Please note the change context works only within the bracket. Outside of the bracket
     the original context is in effect
  */
  CHANGE_CONTEXTS(HR_ASSIGNMENT_ID= CMP_IVR_ASSIGNMENT_ID[index] )
  (
    /* Validate the primary flag and assignment type */
    IF ( PER_ASG_ASSIGNMENT_TYPE = 'E' AND PER_ASG_PRIMARY_FLAG = 'Y' ) THEN
    (
      I_data= ESS_LOG_WRITE( 'Assignment is Primary Employee' )
      I_date_hire = PER_ASG_REL_DATE_START
      /*if salary is not 0*/
      I_data= ESS_LOG_WRITE( 'Salary Amount ' || TO_CHAR(CMP_ASSIGNMENT_SALARY_AMOUNT) )

      IF CMP_ASSIGNMENT_SALARY_AMOUNT WAS NOT DEFAULTED THEN
      (
        COMPENSATION_DATES[I_counter]= TO_CHAR( CMP_IV_PERIOD_END_DATE, 'YYYY/MM/DD')
        ASSIGNMENTS[I_counter] = TO_CHAR(CMP_IVR_ASSIGNMENT_ID[index])
        VALUES[I_counter] = TO_CHAR(CMP_ASSIGNMENT_SALARY_AMOUNT)
        I_counter = I_counter + 1
      )
    )
  )
  /* Moving to next assingment*/
  index = CMP_IVR_ASSIGNMENT_ID.NEXT(index,-1)
)

```

```

I_data= ESS_LOG_WRITE( 'EXITINGTCS_R12_MULTI_ASG ' )
RETURN COMPENSATION_DATES , VALUES,ASSIGNMENTS

```

/*

Name: TCS_R11_MULTI_ASG

Type : Total Compensation Item

*/

/*Declare DBI*/

DEFAULT FOR CMP_ASSIGNMENT_SALARY_AMOUNT IS 0

DEFAULT FOR PER_ASG_ASSIGNMENT_TYPE IS 'N'

DEFAULT FOR PER_ASG_PRIMARY_FLAG IS 'N'

DEFAULT FOR PER_ASG_REL_DATE_START IS '1900/01/01 00:00:00'(date)

DEFAULT FOR PER_ASG_PAYROLL_INFO_PAYROLL_ASSIGNMENT_ID IS -1

DEFAULT_DATA_VALUE FOR PER_HIST_ASG_EFFECTIVE_START_DATE IS '1900/01/01 00:00:00'(date)

DEFAULT_DATA_VALUE FOR PER_HIST_ASG_EFFECTIVE_END_DATE IS '1900/01/01 00:00:00'(date)

DEFAULT_DATA_VALUE FOR PER_HIST_ASG_ASSIGNMENT_ID IS -1

/* Input Values */

INPUTS ARE CMP_IV_PERSON_ID , CMP_IV_PERIOD_START_DATE (DATE), CMP_IV_PERIOD_END_DATE (DATE)

COMPENSATION_DATES = ''

VALUES = ''

ASSIGNMENTS = ''


```
L_data= ESS_LOG_WRITE( 'ENTERING TCS_R11_MULTI_ASG ' )
```

```
L_counter = 1  
index = PER_HIST_ASG_ASSIGNMENT_ID.FIRST(-1)  
L_data= ESS_LOG_WRITE( 'first index ' + TO_CHAR(index) )
```

```
L_PERSON_ID = GET_CONTEXT(PERSON_ID, -1)  
L_ASG_ID = GET_CONTEXT(HR_ASSIGNMENT_ID, -1)  
L_TOT_COUNT = PER_HIST_ASG_ASSIGNMENT_ID.COUNT  
L_data= ESS_LOG_WRITE( ' NO OF ASSIGNMENT ' + TO_CHAR(L_TOT_COUNT) )  
L_data= ESS_LOG_WRITE( ' PERSON_ID ' + TO_CHAR(L_PERSON_ID) )  
L_data= ESS_LOG_WRITE( ' ASG_ID ' + TO_CHAR(L_ASG_ID) )  
L_data= ESS_LOG_WRITE( ' Start Date ' + TO_CHAR(CMP_IV_PERIOD_START_DATE) )  
L_data= ESS_LOG_WRITE( ' End Date ' + TO_CHAR(CMP_IV_PERIOD_END_DATE) )
```

```
/* Looping through the assignment array */  
WHILE ( PER_HIST_ASG_ASSIGNMENT_ID.EXISTS(index))  
LOOP (  
    /* the history has all the date tracked rows, we take the row that falls within  
    our date  
    */  
    L_data= ESS_LOG_WRITE( ' Assignment of asg array ' + TO_CHAR(PER_HIST_ASG_ASSIGNMENT_ID[index]) )  
    L_data= ESS_LOG_WRITE( ' Start Date of asg array ' + TO_CHAR(PER_HIST_ASG_EFFECTIVE_START_DATE[index]) )  
    L_data= ESS_LOG_WRITE( ' End Date of asg array ' + TO_CHAR(PER_HIST_ASG_EFFECTIVE_END_DATE[index]) )  
    IF ( PER_HIST_ASG_EFFECTIVE_START_DATE[index] <= CMP_IV_PERIOD_END_DATE AND  
        PER_HIST_ASG_EFFECTIVE_END_DATE[index] >= CMP_IV_PERIOD_START_DATE) THEN
```

```

(
  l_date = GREATEST(CMP_IV_PERIOD_START_DATE, PER_HIST_ASG_EFFECTIVE_START_DATE[index])

CHANGE_CONTEXTS(HR_ASSIGNMENT_ID= PER_HIST_ASG_ASSIGNMENT_ID[index], EFFECTIVE_DATE = l_date )
(
  /* Validate the primary flag and assignment type */
  IF ( PER_ASG_ASSIGNMENT_TYPE = 'E' AND PER_ASG_PRIMARY_FLAG = 'Y' ) THEN
  (
    l_data= ESS_LOG_WRITE( 'Assignment is Primary Employee' )
    l_date_hire = PER_ASG_REL_DATE_START
    /*if salary is not 0*/
    l_data= ESS_LOG_WRITE( 'Salary Amount ' || TO_CHAR(CMP_ASSIGNMENT_SALARY_AMOUNT) )
    l_data = ESS_LOG_WRITE(' PAY ASG ' || TO_CHAR(PER_ASG_PAYROLL_INFO_PAYROLL_ASSIGNMENT_ID) )
    IF CMP_ASSIGNMENT_SALARY_AMOUNT WAS NOT DEFAULTTED THEN
    (
      COMPENSATION_DATES= COMPENSATION_DATES||TO_CHAR( l_date, 'YYYY/MM/DD')|| ';'
      ASSIGNMENTS = ASSIGNMENTS||TO_CHAR(PER_HIST_ASG_ASSIGNMENT_ID[index]) || ';'
      VALUES = VALUES ||TO_CHAR(CMP_ASSIGNMENT_SALARY_AMOUNT) || ';'
      l_counter = l_counter + 1
      l_data= ESS_LOG_WRITE( 'date ' || COMPENSATION_DATES)
      l_data= ESS_LOG_WRITE( 'asg' || ASSIGNMENTS )
      l_data= ESS_LOG_WRITE( 'value' || VALUES )
    )
  )
)
)
index = PER_HIST_ASG_ASSIGNMENT_ID.NEXT(index, -1)
)
COMPENSATION_DATES = LTRIM(COMPENSATION_DATES)
VALUES           = LTRIM(VALUES)
ASSIGNMENTS      = ITRIM(ASSIGNMENTS)

l_data= ESS_LOG_WRITE( 'EXITING TCS_R11_MULTI_ASG ' )
RETURN COMPENSATION_DATES , VALUES,ASSIGNMENTS

```

Please note that these formulas are not fully tested. Use these codes as a sample to understand the logic.

Please also check the availability of the features with Oracle.

Fast Formula – Introduction to Compensation Workbench

Fast Formula – Introduction to Compensation Workbench (CWB), Salary (CMP), Total Compensation (TCS), and Grade Step Progression (GSP).

In [section 1](#), we went over the basics of the Fast Formula engine and its components and in [section 2](#), we showed the steps to create a simple Fast Formula. If you are new to Fast Formula, please check these sections before proceeding further on this section.

In this section, we will introduce you to a real life Fast Formula of product Compensation. Formulas are like User Defined Functions. To develop any Functions, the developer needs to understand three basic things.

1. What is available? (Global variables, IN parameters or any other cached data)
2. What is expected? (Return values, OUT parameters , cached data or global variables)
3. How to go from available to expected? In other words, business logic and technical know-how.

We will now discuss about what is available such as Formula Types, Context, Input values in Fast Formula for products like CMP, TCS and GSP and what is expected such as Return variables and values.

Formula Types:

Compensation (CMP/TCS/GSP) has provided 10 Formula Types (may be more or less depending on the release)

Formula Type	Description	Usage
Compensation Currency Selection	Compensation Currency Selection Rule	This Formula Type is used for determining Currency at component Level. Compensation-Manage Plan->Configure Compensation Component -> Edit Component
Compensation Default Access Level	Compensation Default Access Level Rule	By this type, customer can define customized access level in hierarchy and reviewer.

		Compensation-Manage Plan->Configure Hierarchy
Compensation Default and Override	Compensation Default and Override Rule	This Formula Type is attached in CWB attribute values to provide the value for the attribute Compensation->Manage Plan ->Configure Work Sheet-Configure tab -> detail table-> properties
Compensation End Date	Compensation End Date Rule	This is created for Variable compensation. I believe it is not implemented yet.
Compensation Hierarchy Determination	Compensation Hierarchy Determination	Compensation-Manage Plan->Configure Hierarchy
Compensation Person Selection	Compensation Person Selection Rule	This Formula type is used by Work Bench and Total Compensation ESS process to Filter the people. Compensation->Generate Compensation. Compensation->Run Batch Process.
Compensation Start Date	Compensation Start Date Rule	This is created for Variable compensation. I believe it is not implemented yet.
Salary Progression Date Determination Rule	Salary Progression Date Determination Rule to determine the date	This formula type is attached in Assignment or Salary update date for the GSP process. Compensation -> Manage Progressions Ladder
Salary Progression Rate Calculation Rule	Salary Progression Rate Calculation Rule to determine the Salary	This formula type is attached in New rate calculation for GSP Compensation -> Manage Progressions Ladder
Total Compensation Item	Total Compensation Item Rule	This formula type is attached in Total Compensation Item. Allows providing value for Total Compensation Item. Compensation->Manage Compensation Item

Context:

Compensation (CMP/TCS/GSP) has provided 16 contexts. As of now, all Formula Types of CMP/TCS/GSP support the same set of context. I found this very interesting and this has made it easy to deal with the different Formula Types of the same product.

Context	Type	Description
DATE_EARNED	Date	Effective date of the process, if person terminated, termination date
EFFECTIVE_DATE	Date	Effective date of the process, if person terminated, termination date
HR_ASSIGNMENT_ID	Number	Assignment Id, (primary Assignment for TCS)
PERSON_ID	Number	Person Id

HR_TERM_ID	Number	Term id of the assignment (primary Assignment's Term ID for TCS)
GRADE_ID	Number	Grade of the Assignment (primary Assignment's Grade ID for TCS)
GRADE_RATE_ID	Number	Grade Rate of the Assignment (primary Assignment's Grade Rate ID for TCS)
JOB_ID	Number	Job of the Assignment (primary Assignment's Job ID for TCS)
LEGISLATIVE_DATA_GROUP_ID	Number	LDG of the Assignment (primary Assignment's LDG ID for TCS)
ORGANIZATION_ID	Number	Organization of the Assignment (primary Assignment's Organization ID for TCS)
PAYROLL_ASSIGNMENT_ID	Number	Payroll Assignment of the HR assignment (primary Assignment's Payroll Assignment ID for TCS)
PAYROLL_RELATIONSHIP_ID	Number	Payroll Relation of the HR Assignment (primary Assignment's Payroll Relation ID for TCS)
PAYROLL_TERM_ID	Number	Payroll Term of the HR Assignment (primary Assignment's Payroll Term ID for TCS)
END_DATE	Date	Plan/Statement period start
START_DATE	Date	Plan/Statement period end date
COMPENSATION_RECORD_TYPE	String	This is not passed by the product. User can defined the contest in the formula for using External Data

Note: The Fast Formula engine is designed in such way that the context is set only when the Fast Formula uses the context. When I say “used”, it may be when you are directly calling a function, GET_CONTEXT, NEED_CONTEXT, CHANGE_CONTEXTS or CONTEXT_IS_SET or indirectly when using a DBI.

In our example, in the previous section, we neither used a DBI nor a context function and therefore no Context was passed to the Formula. We will discuss this in detail in the next sections.

Input Values: Input values are the information that are passed by the product to Formula engine in addition to Context. Like Context, the Input values are passed only if the input values are used in the Formula. Unlike Context, the Compensation passes different set of Input values to the formula.

The list of the formula type and the context are as follows (may be more or less depending on the release).

Formula Type Name	Input values	Type	Format
Compensation Currency Selection			

	CMP_IV_PLAN_ID	TEXT	
	CMP_IV_PERIOD_ID	TEXT	
	CMP_IV_COMPONENT_ID	TEXT	
	CMP_IV_PLAN_START_DATE	TEXT	
	CMP_IV_PLAN_END_DATE	TEXT	
	CMP_IV_PLAN_ELIG_DATE	TEXT	
	CMP_IV_PERFORMANCE_EFF_DATE	TEXT	
	CMP_IV_PROMOTION_EFF_DATE	TEXT	
	CMP_IV_XCHG_RATE_DATE	TEXT	
	CMP_IV_PLAN_EXTRACTION_DATE	TEXT	
Compensation Default and Override			

	CMP_IV_PLAN_ID	TEXT	
	CMP_IV_PERIOD_ID	TEXT	
	CMP_IV_PLAN_START_DATE	TEXT	
	CMP_IV_PLAN_END_DATE	TEXT	
	CMP_IV_PLAN_ELIG_DATE	TEXT	
	CMP_IV_PERFORMANCE_EFF_DATE	TEXT	
	CMP_IV_PROMOTION_EFF_DATE	TEXT	
	CMP_IV_XCHG_RATE_DATE	TEXT	
	CMP_IV_XCHG_RATE_DATE	TEXT	
	CMP_IV_ITEM_NAME	TEXT	
	CMP_IV_COMPONENT_ID	TEXT	

	CMP_IV_PLAN_EXTRACTION_DATE	TEXT	
Compensation Hierarchy Determination			
	CMP_IV_PLAN_ID	TEXT	
	CMP_IV_PERIOD_ID	TEXT	
	CMP_IV_PLAN_START_DATE	TEXT	
	CMP_IV_PLAN_END_DATE	TEXT	
	CMP_IV_PLAN_ELIG_DATE	TEXT	
	CMP_IV_PERFORMANCE_EFF_DATE	TEXT	
	CMP_IV_PROMOTION_EFF_DATE	TEXT	
	CMP_IV_XCHG_RATE_DATE	TEXT	

	CMP_IV_ASSIGNMENT_ID	TEXT	
	CMP_IV_PLAN_EXTRACTION_DATE	TEXT	
Compensation Person Selection			
	CMP_IV_PLAN_START_DATE	TEXT	YYYY/MM/DD
	CMP_IV_PLAN_END_DATE	TEXT	YYYY/MM/DD
	CMP_IV_PLAN_ELIG_DATE	TEXT	YYYY/MM/DD
	CMP_IV_PLAN_EXTRACTION_DATE	TEXT	YYYY/MM/DD
	CMP_IV_PERFORMANCE_EFF_DATE	TEXT	YYYY/MM/DD
	CMP_IV_PROMOTION_EFF_DATE	TEXT	YYYY/MM/DD

	CMP_IV_XCHG_RATE_DATE	TEXT	YYYY/MM/DD
	CMP_IV_PERIOD_ID	TEXT	
Total Compensation Item			
	CMP_IV_PERSON_ID	Number	
	CMP_IV_PERIOD_START_DATE	Date	
	CMP_IV_PERIOD_END_DATE	Date	
	CMP_IV_STMT_ID	Number	
	CMP_IV_PERIOD_ID	Number	
	CMP_IV_ITEM_NAME	TEXT	
Compensation Person Selection (TCS)			
	CMP_IV_PERSON_ID	TEXT	

	CMP_IV_EFFECTIVE_DATE	TEXT	YYYY/MM/DD
	CMP_IV_PERIOD_START_DATE	TEXT	YYYY/MM/DD
	CMP_IV_PERIOD_END_DATE	TEXT	YYYY/MM/DD
	CMP_IV_STMT_ID	TEXT	
	CMP_IV_PERIOD_ID	TEXT	
The following are array input values for TCS Item Formula. Since the TCS item Formula is executed for a person and only primary assignment is passed as context, these array input values are introduced in the latest releases to pass all the assignment of a person and its related information to the fast formula as input values. Please check with the TCS document for the availability of these input values.	CMP_IVR_ASSIGNMENT_ID	NUMBER_NUMBER	
	CMP_IVR_TERM_ID	NUMBER_NUMBER	-1 in case of null
	CMP_IVR_PAY_ASSIGNMENT_ID	NUMBER_NUMBER	-1 in case of null
	CMP_IVR_PAY_TERM_ID	NUMBER_NUMBER	-1 in case of null
	CMP_IVR_PAY_RELATION_ID	NUMBER_NUMBER	-1 in case of null

Compensation Default Access Level	CMP_IV_PLAN_START_DATE	Text	
	CMP_IV_PLAN_END_DATE	Text	
	CMP_IV_PLAN_ID	Text	
	CMP_IV_PERIOD_ID	Text	
	CMP_IV_PLAN_ELIG_DATE	Text	
	CMP_IV_PLAN_EXTRACTION_DATE	Text	
	CMP_IV_PERFORMANCE_EFF_DATE	Text	
	CMP_IV_PROMOTION_EFF_DATE	Text	
	CMP_IV_XCHG_RATE_DATE	Text	
	CMP_IV_HIERARCHY_TYPE	Text	

Salary Progression Date Determination Rule	CMP_IV_GRADE_LADDER_ID	Number	
	LEGISLATIVE_DATA_GROUP_ID	Number	
	CMP_IV_ASSIGNMENT_ID	Number	
	CMP_IV_PERSON_ID	Number	
	CMP_IV_GRADE_ID	Number	
	CMP_IV_GRADE_RATE_ID	Number	
	CMP_IV_GRADE_STEP_ID	Number	
Salary Progression Rate Calculation Rule	CMP_IV_GRADE_LADDER_ID	Number	
	LEGISLATIVE_DATA_GROUP_ID	Number	

	CMP_IV_ASSIGNMENT_ID	Number	
	CMP_IV_PERSON_ID	Number	
	CMP_IV_GRADE_ID	Number	
	CMP_IV_GRADE_RATE_ID	Number	
	CMP_IV_GRADE_STEP_ID	Number	

In case, the date format is not defined for the date input value, you need to use the default (Database) date format. In case you do not know the default date format, you can write the data in your log file to find the date format.

Return Value/Variable:

The following list is the expected return values or variables from the CWB/TCS/GSP Formula Types.

Formula Type Name	OUTPUT
Compensation Currency Selection	No restriction, Customer should make sure what they return is a currency code

Compensation Default and Override	In previous release, the expected variables are L_DATA_TYPE, L_DEFAULT_VALUE L_DATA_TYPE determine the type of the value such as CHAR, NUMBER, DATE L_DEFAULT_VALUE holds the return value. In the recent releases, you do not need to return the data type. It is determined from the attribute where the Formula is attached. Therefore, there is no restriction on the variable name. The Date value must be returned in YYYY/MM/DD format.
Compensation Hierarchy Determination	There are three options 1) Return Person id and Assignment id in variable L_PERSON_ID and L_ASSIGNMENT_ID 2) Pass Person Number in L_PERSON_NUMBER 3) Pass assignment number in L_ASSIGNMENT_NUMBER In case you want to pass assignment id or person id as null, use -999
Compensation Person Selection	The Return value must be either 'Y' or 'N'
Total Compensation Item	TCS Item formula allows to return multiple rows by delimiting values with ',' The Required values are date and values. The date and values should be returned in variable, COMPENSATION_DATES, VALUES. The date should be returned in YYYY/MM/DD format. Other optional values are assignment id, legal employer id should be returned in variable, ASSIGNMENTS, LEGALEMPLOYERS. The latest release supports returning array instead of delimited values. The array should use the same variables name as above.

Compensation Default Access Level	This formula supports the values. UPDATABLE NOACCESS READONLY
Salary Progression Date Determination Rule	Return date value in YYYY/MM/DD format
Salary Progression Rate Calculation Rule	Supports only numeric value

This concludes the introduction to the Compensation Product Fast Formulas. We will start with a real life example in our next section. **If you have any questions, please let add them in the comments.**

Grade Steps Progressions (GSP) Fast Formula Samples

We found that there is not much information available on GSP Formula Types. So, we have put together a good set of sample codes with a hypothetical use case that could help the GSP users.

The information provided in this blog may not be the latest or may be obsoleted. Please consult with your Product Support to get the latest information. The code provided here is to give an example.

We explained the contexts of GSP Formula Types in our [previous blog](#) and therefore we are not going to repeat the same again. All the CMP/TCS and GSP formulas types support the same Contexts.

Formula Type: Compensation Person Selection.

The type of formula used in ESS parameters is person Selection Formula.

This formula is executed while selecting the person for the process. The count of processed people is calculated after executing this formula and individual persons are processed after the person selection Formula is executed for all the people who were selected by other ESS parameters.

Input value for Person Selection Formula for GSP process.

Input Value	Type	Format
CMP_IV_GRADE_LADDER_ID	Number	

Since the Person Selection is common for the entire compensation product, you can use any example provided for CWB, TCS or GSP.

Formula Type: Participation and Rate Eligibility.

The Formula type belongs to Oracle Benefit and used in Oracle Benefit's Eligibility. GSP uses the Benefit Eligibility Engine to determine the eligibility of a Grade Ladder, grades and steps for a person. Therefore, you need to create eligibility profile to limit /Filter the Grade Ladder, Grades or Steps.

Again, it is very common for CWB, TCS and GSP. Though it is common type, it uses different input values and business logic to validate a Ladder, Grade or Steps.

The most important aspects the eligibility formula is the Return value and Return variable. This type of formula supports the value 'Y' or 'N' and the return variable must be **ELIGIBLE**. If you return any other variable or in any other case format the Formula set the person/grade/step as ineligible.

The contexts Supported by the GSP eligibility Formulas

Context	Type
DATE_EARNED	Date
EFFECTIVE_DATE	Date
HR_ASSIGNMENT_ID	Number
JOB_ID	Number
LEGAL_EMPLOYER_ID	Number
LEGISLATIVE_DATA_GROUP_ID	Number
PAYROLL_ASSIGNMENT_ID	Number
PAYROLL_RELATIONSHIP_ID	Number
PAYROLL_TERM_ID	Number
PERSON_ID	Number
HR_TERM_ID	Number

Eligibility of Grade Ladder.

You can create eligibility profile at Ladder level. This formula is executed once for an assignment. If the Assignment is not eligible for the Ladder, the process does not evaluate the grades and steps. This is the First formula which gets executed at the person Level processing.

Input Values	Types
CMP_IV_GRADE_LADDER_ID	Number
CMP_IV_GRADE_LADDER_NAME	Text

Eligibility of Grade.

The formula is attached with the Grade eligibility profile and is executed for every person. The grade is not eligible then the Steps are not processed. The process is intelligent enough to process the grades that are above the current grade of the person and it continues the process till it finds the first ineligible grade.

Input Values	Types
CMP_IV_GRADE_LADDER_ID	Number
CMP_IV_GRADE_LADDER_NAME	Text
CMP_IV_GRADE_ID	Number
CMP_IV_GRADE_NAME	Text

Eligibility Step.

Steps are processed very similar to the grades.

Input Values	Types
CMP_IV_GRADE_LADDER_ID	Number
CMP_IV_GRADE_LADDER_NAME	Text
CMP_IV_GRADE_ID	Number
CMP_IV_GRADE_NAME	Text
CMP_IV_STEP_ID	Number
CMP_IV_STEP_NAME	Text

Use Case: To demonstrate the Eligibility formula, we used a hypothetical Use Case, in this Use Case, the Ladder has two Grades and each Grade has four Steps.

The Eligibility is attached at Step level. No eligibility is defined at Grade Level or Ladder Level.

The person will move to next step for every process only when the person worked more than a year in the Organization. In case, if the person has worked more than three years in the Organization, then he is eligible for third step (Skip one step). The person is eligible for only one step in a process.

Create an eligibility profile with formula. (Formula is created before the eligibility profile)

Create Steps rule with eligibility profile

Formula text of Eligibility.

/*

Formula Type: Participation and Rate Eligibility

**/*

/ DBI Used in the FF */*

DEFAULT FOR PER_PER_LATEST_REHIRE_DATE IS '1900/01/01 00:00:00'(date)

DEFAULT FOR PER_GRADE_STEP_NAME IS '-1'

/ Input Values */*

*INPUTS ARE CMP_IV_GRADE_LADDER_ID(NUMBER), CMP_IV_GRADE_ID (NUMBER),
CMP_IV_STEP_ID(NUMBER),*

*CMP_IV_GRADE_LADDER_NAME (TEXT), CMP_IV_GRADE_NAME (TEXT), CMP_IV_STEP_NAME
(TEXT)*

*L_temp = ESS_LOG_WRITE('***Entering Tilak_Lakshmi_Eligibility_Elig_FF***')*

/ Get context Values */*

L_PERSON_ID = GET_CONTEXT(PERSON_ID, -1)

L_ASG_ID = GET_CONTEXT(HR_ASSIGNMENT_ID, -1)

L_EFF_DATE = get_context(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date))

/ Local Variables */*

L_1_ANIV = ADD_DAYS(PER_PER_LATEST_REHIRE_DATE, 365)

L_3_ANIV = ADD_DAYS(PER_PER_LATEST_REHIRE_DATE, 1095)

L_ELIG = 'N'

ELIGIBLE = 'N'

L_temp = ESS_LOG_WRITE('Person ID : ' + to_char(L_PERSON_ID))

```
L_temp = ESS_LOG_WRITE('Asg ID : ' + to_char(L_ASG_ID) )
```

```
L_temp = ESS_LOG_WRITE('Effective date : ' + to_char(L_EFF_DATE) )
```

```
L_temp = ESS_LOG_WRITE('First Anniversary : ' + to_char(L_1_ANIV) )
```

```
L_temp = ESS_LOG_WRITE('Third Anniversary : ' + to_char(L_3_ANIV) )
```

```
L_temp = ESS_LOG_WRITE('Current Step : ' + PER_GRADE_STEP_NAME )
```

```
L_temp = ESS_LOG_WRITE('Validating STEP : ' + CMP_IV_STEP_NAME )
```

```
/* Get the value from WSA */
```

```
L_ELIG_ASG = WSA_GET('WSA_GSP_ELIG_PERSON', -1)
```

```
L_temp = ESS_LOG_WRITE('Elig ASG ID : ' + TO_CHAR( L_ELIG_ASG ) )
```

```
/* IF the asg already eligible */
```

```
IF (L_ELIG_ASG = L_ASG_ID ) THEN
```

```
(
```



```
L_temp = ESS_LOG_WRITE('Assignment is already eligible for a Step' )
```

```
L_ELIG = 'Y'
```

```
)
```

```
/* not eligible yet */
```

```
IF (L_ELIG = 'N') THEN
```

```
(
```

```
/* Three year anniversary passed */
```

```
IF (L_3_ANIV <= L_EFF_DATE ) THEN
```

```
(
```

```
L_temp = ESS_LOG_WRITE('Assignment has worked more then 3 years' )
```

```
IF (PER_GRADE_STEP_NAME = '1' AND CMP_IV_STEP_NAME = '3') THEN
```

```
(
```

```
ELIGIBLE = 'Y'
```

```
)ELSE (
```

```
    IF (PER_GRADE_STEP_NAME = '2' AND CMP_IV_STEP_NAME = '4') THEN
```

```
    (
```

```
        ELIGIBLE = 'Y'
```

```
    )
```

```
)
```

```
) ELSE
```

```
    (
```

```
        /* One year anniversary */
```

```
        L_temp = ESS_LOG_WRITE('Assignment has worked more then 1 year')
```

```
        IF (L_1_ANIV <= L_EFF_DATE ) THEN
```

```
        (
```

```
            IF (PER_GRADE_STEP_NAME = '1' AND CMP_IV_STEP_NAME = '2') THEN
```

```
            (
```

ELIGIBLE = 'Y'

)ELSE (

IF (PER_GRADE_STEP_NAME = '2' AND CMP_IV_STEP_NAME = '3') THEN

(

ELIGIBLE = 'Y'

) ELSE (

IF (PER_GRADE_STEP_NAME = '3' AND CMP_IV_STEP_NAME = '4') THEN

(

ELIGIBLE = 'Y'

)

)

)

)

)

)

/ if eligible store the value */*

IF (ELIGIBLE = 'Y') THEN

(

WSA_SET('WSA_GSP_ELIG_PERSON', L_ASG_ID)

)

L_temp = ESS_LOG_WRITE('Leaving Tilak_Laskshmi_Eligibility_Elig_FF : ' + ELIGIBLE)

RETURN ELIGIBLE

Explanation of Formula.

/ DBI Used in the FF */*

DEFAULT FOR PER_PER_LATEST_REHIRE_DATE IS '1900/01/01 00:00:00'(date)

*DEFAULT FOR PER_GRADE_STEP_NAME IS '-1'*Initialized the DBI that is going to be used. The hiring date is used for calculating the Anniversary and Step name is used for comparing the steps to determine the eligible step.

/ Input Values */*

*INPUTS ARE CMP_IV_GRADE_LADDER_ID(NUMBER), CMP_IV_GRADE_ID (NUMBER),
CMP_IV_STEP_ID(NUMBER),*

*CMP_IV_GRADE_LADDER_NAME (TEXT), CMP_IV_GRADE_NAME (TEXT),CMP_IV_STEP_NAME
(TEXT)*

Initialized the Input values. Though we do not use all the input values, we initialized all the Input values for our example.

/ Get context Values */*

L_PERSON_ID = GET_CONTEXT(PERSON_ID, -1)

L_ASG_ID = GET_CONTEXT(HR_ASSIGNMENT_ID, -1)

L_EFF_DATE = get_context(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date))

The variables are initialized from the context. We use effective date to calculate and compare the years of experience. The Assignment Id is used to find the previous eligibility.

/ Local Variables */*

L_1_ANIV = ADD_DAYS(PER_PER_LATEST_REHIRE_DATE, 365)

L_3_ANIV = ADD_DAYS(PER_PER_LATEST_REHIRE_DATE, 1095)

L_ELIG = 'N'

ELIGIBLE = 'N'

Required local variables are initialized or calculated. First year and Third year anniversary date are calculated from effective Date. L_elig is defaulted to 'N' and used to determine if the assignment is already eligible for a step and ELIGIBLE for returning the result (please note that this type of formula supports only the return variable 'ELIGIBLE').

/ Get the value from WSA */*

L_ELIG_ASG = WSA_GET('WSA_GSP_ELIG_PERSON', -1)

L_temp = ESS_LOG_WRITE('Elig ASG ID : ' + TO_CHAR(L_ELIG_ASG))

/ IF the asg already eligible */*

IF (L_ELIG_ASG = L_ASG_ID) THEN

(

L_temp = ESS_LOG_WRITE('Assignment is already eligible for a Step')

L_ELIG = 'Y'

)

/ not eligible yet */*

IF (L_ELIG = 'N') THEN

To validate if a person is already eligible, we check the variable/value from WSA (to understand more about WSA please read out blog, [Communicating between Fast Formulas](#)), if the WSA variable matches with the assignment id, we set the l_elig variable to 'Y'. Since the person is already eligible for a step, we do not validate further, we return the default value of ELIGIBLE that is 'N'.

```
/* Three year anniversary passed */
```

```
IF (L_3_ANIV <= L_EFF_DATE ) THEN
```

```
(
```

```
L_temp = ESS_LOG_WRITE('Assignment has worked more then 3 years' )
```

```
IF (PER_GRADE_STEP_NAME = '1' AND CMP_IV_STEP_NAME = '3') THEN
```

```
(
```

```
ELIGIBLE = 'Y'
```

```
)ELSE (
```

```
IF (PER_GRADE_STEP_NAME = '2' AND CMP_IV_STEP_NAME = '4') THEN
```


(

ELIGIBLE = 'Y'

)

)

If the person has worked for more than 3 years, we check the person's current step (DBI PER_GRADE_STEP_NAME) is '1' and the Formula validates step (Input value, CMP_IV_STEP_NAME) is '3' (third step from the current step) or current step is 2. And the validated step is 4, then we set the ELIGIBLE variable to 'Y' (otherwise the default value, 'N').

IF (L_1_ANIV <= L_EFF_DATE) THEN

(

IF (PER_GRADE_STEP_NAME = '1' AND CMP_IV_STEP_NAME = '2') THEN

(

ELIGIBLE = 'Y'

```
)ELSE (
```

```
IF (PER_GRADE_STEP_NAME = '2' AND CMP_IV_STEP_NAME = '3') THEN
```

```
(
```

```
ELIGIBLE = 'Y'
```

```
) ELSE (
```

```
IF (PER_GRADE_STEP_NAME = '3' AND CMP_IV_STEP_NAME = '4') THEN
```

```
(
```

```
ELIGIBLE = 'Y'
```

```
)
```

```
)
```

```
)
```

If the person worked more than 1 year and less than 3 years, we check the current step (DBI PER_GRADE_STEP_NAME) and the validated step (Input value, CMP_IV_STEP_NAME) is the next step (Example: Current step is 1 and next step is 2) then we set the ELIGIBLE variable to 'Y' (otherwise the default value, 'N').

/ if eligible store the value */*

IF (ELIGIBLE = 'Y') THEN

(

WSA_SET('WSA_GSP_ELIG_PERSON', L_ASG_ID)

)

If the person is eligible then we set the WSA variable to the assignment id so that we do not need to validate other steps.

RETURN ELIGIBLE

In the End, We return the ELIGIBLE value (Y/N).

I don't think logs need any explanation, please comment if you need any detail on the log .

Log of 1 year experienced Person, eligible for Step 2 .

****Entering Tilak_Lakshmi_Eligibility_Elig_FF****

Person ID : 300100167151081

Asg ID : 300100167151144

Effective date : 2015-07-23T00:00:00.000Z

First Anniversary : 2015-07-23T00:00:00.000Z

Third Anniversary : 2017-07-22T00:00:00.000Z

Current Step : 1

Validating STEP : 2

Elig ASG ID : -1

Assignment has worked more then 1 year

Leaving Tilak_Laskshmi_Eligibility_Elig_FF : Y

******Entering Tilak_Lakshmi_Eligibility_Elig_FF******

Person ID : 300100167151081

Asg ID : 300100167151144

Effective date : 2015-07-23T00:00:00.000Z

First Anniversary : 2015-07-23T00:00:00.000Z

Third Anniversary : 2017-07-22T00:00:00.000Z

Current Step : 1

Validating STEP : 3

Elig ASG ID : 300100167151144

Assignment is already eligible for a Step

Leaving Tilak_Laskshmi_Eligibility_Elig_FF : N

Ineligible assignment_id 300100167151144: Reason ERL

****Entering Tilak_Lakshmi_Eligibility_Elig_FF****

Person ID : 300100167151081

Asg ID : 300100167151144

Effective date : 2015-07-23T00:00:00.000Z

First Anniversary : 2015-07-23T00:00:00.000Z

Third Anniversary : 2017-07-22T00:00:00.000Z

Current Step : 1

Validating STEP : 4

Elig ASG ID : 300100167151144

Assignment is already eligible for a Step

Leaving Tilak_Lakshmi_Eligibility_Elig_FF : N

Log of 3 year experienced Person, not eligible for step 2 but 3.

Processing Person : FFTEST, Tilak-Lakshmi

****Entering Tilak_Lakshmi_Eligibility_Elig_FF****

Person ID : 300100167099668

Asg ID : 300100167099683

Effective date : 2015-07-23T00:00:00.000Z

First Anniversary : 2002-07-23T00:00:00.000Z

Third Anniversary : 2004-07-22T00:00:00.000Z

Current Step : 1

Validating STEP : 2

Elig ASG ID : 300100167151144

Assignment has worked more then 3 years

Leaving Tilak_Laskshmi_Eligibility_Elig_FF : N

Ineligible assignment_id 300100167099683: Reason ERL

******Entering Tilak_Lakshmi_Eligibility_Elig_FF******

Person ID : 300100167099668

Asg ID : 300100167099683

Effective date : 2015-07-23T00:00:00.000Z

First Anniversary : 2002-07-23T00:00:00.000Z

Third Anniversary : 2004-07-22T00:00:00.000Z

Current Step : 1

Validating STEP : 3

Elig ASG ID : 300100167151144

Assignment has worked more then 3 years

Leaving Tilak_Laskshmi_Eligibility_Elig_FF : Y

******Entering Tilak_Lakshmi_Eligibility_Elig_FF******

Person ID : 300100167099668

Asg ID : 300100167099683

Effective date : 2015-07-23T00:00:00.000Z

First Anniversary : 2002-07-23T00:00:00.000Z

Third Anniversary : 2004-07-22T00:00:00.000Z

Current Step : 1

Validating STEP : 4

Elig ASG ID : 300100167099683

Assignment is already eligible for a Step

Leaving Tilak_Laskshmi_Eligibility_Elig_FF : N

This is end of the Eligibility formula. We will cover the Rate and Date formulas in our next blog.

As usual, if you have any questions or suggestions, please feel free to comment.

You Tube Tutorial:

GSP Person Selection Formula 1 - Oracle Fusion HCM Fast Formula Tutorial-8

```
/*
Name : GSP_PERSON_SELECTION
Type. : Compensation Person Selection
Purpose : to filter the person who are in probation period or already in step 5 or more
*****/
/* DBI */
DEFAULT FOR PER_GRADE_STEP_SEQUENCE IS -1
DEFAULT FOR PER_ASG_DATE_PROBATION_END IS '1900/01/01 00:00:00' (date)
/* input */
INPUTS ARE CMP_IV_GRADE_LADDER_ID (NUMBER)
l_data = SET_LOG(' Entering GSP_PERSON_SELECTION')
RETVAL = 'Y'
grade = GET_CONTEXT(GRADE_ID , -1)
eDate = GET_CONTEXT(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date) )
l_data = SET_LOG(' Grade ID ' || TO_CHAR(grade) )
l_data = SET_LOG(' EFF DATE ' || TO_CHAR(eDate) )
l_data = SET_LOG(' STEP SEQ ' || TO_CHAR(PER_GRADE_STEP_SEQUENCE) )
l_data = SET_LOG(' PROB DATE ' || TO_CHAR(PER_ASG_DATE_PROBATION_END) )
/* step is < 5 */
IF (PER_GRADE_STEP_SEQUENCE >= 5 ) THEN (
    RETVAL = 'N'
    l_data = SET_LOG(' Failed in step ' )
)
/* probation validation */
IF (RETVAL = 'Y' ) THEN (
    IF ( PER_ASG_DATE_PROBATION_END >= eDate ) THEN (
        RETVAL = 'N'
        l_data = SET_LOG(' Failed in prob ' )
    )
)
)
```

```
l_data = SET_LOG(' Leaving GSP_PERSON_SELECTION ' || RETVAL )  
RETURN RETVAL
```

Grade Steps Progressions (GSP) Fast Formula Samples - 2

In our previous blog, we explained the Eligibility Formula for GSP. In this blog, we will continue our discussion with the Salary progression rate calculation Rule (GSP Rate), Salary progression, Date determination and Rule (GSP Date).

Formula Type: Salary progression rate calculation Rule (GSP Rate).

These Formulas allow you to customize the rate for all the Grades and Steps in a Grade Ladder. The Formula is defined at a Grade Ladder level and therefore the FF is executed for every Grade/Step in the Ladder and the value from the Formula takes precedence to the defined rate at Grade/Step level.

Note that the Formulas are not returning the salary but the Grade Step Rate. The GSP process is using the defined Annual Factors and FTE (Full Time Equivalent) to calculate the Salary from the value returned by the Fast Formula. Do not assume that the value returned from the Formula is used as it is.

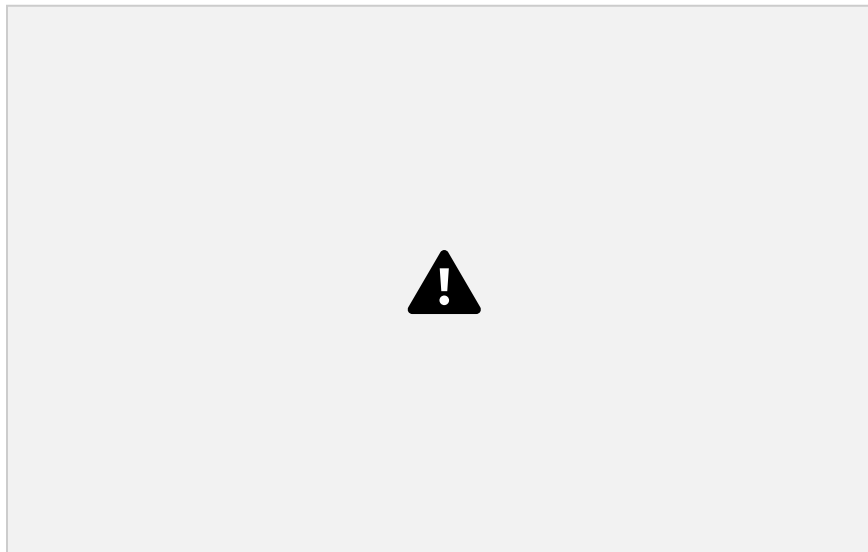
The Formulas uses the same context as any other Compensation Formula. [Please see our previous blog on Compensation Formula Types and Contexts.](#)

Input Values:

Input Values	Types
CMP_IV_GRADE_LADDER_ID	Number

CMP_IV_GRADE_LADDER_NAME	Text
CMP_IV_GRADE_ID	Number
CMP_IV_GRADE_NAME	Text
CMP_IV_STEP_ID	Number
CMP_IV_STEP_NAME	Text

Grade Ladder Setup with Rate and Date Formula.



Formula use case For Calculating Rate:

As per our hypothetical use case, the Rate is calculated from a person's current salary.

- Get the Current Salary and Salary Frequencies from the Salary setup using DBI.
- Get the Salary frequency from the Grade Ladder DBI, if the Ladder's frequency is not the same as Salary setup (you do not need to get it from DBI as it is defined at ladder level, you can hard code the values), convert the salary to Grade Ladder frequency.
- For each step, we add 2% of the original salary and the percentage increased with every setup. For example if the current step is 1 and Rate is calculated for Step 2, then we add 2% of salary, if Rate is calculated for Step 3 then we add 4% salary and so on.

- As per our [eligibility Rule](#), the person is either eligible for next step or the following step after that, therefore we use 2% or 4% hike.
- If there is any FTE defined, we need to convert the rate with FTE because it is again applied in the process after the formula returns the value.

Formula Type: Salary progression rate calculation Rule

/******

Formula Type : Salary Progression Rate Calculation Rule

*****/

/* DBI FOR Ladder Factor and Freq */

DEFAULT_DATA_VALUE FOR PER_ASG_GRADE_RATES_ANNUALIZATION_FACTOR IS 1

DEFAULT_DATA_VALUE FOR PER_ASG_GRADE_RATES_RATE_FREQUENCY IS ''

DEFAULT_DATA_VALUE FOR PER_ASG_GRADE_RATES_NAME IS ''

DEFAULT_DATA_VALUE FOR PER_GRADE_RATES_ANNUALIZATION_FACTOR IS 1

DEFAULT_DATA_VALUE FOR PER_GRADE_RATES_RATE_FREQUENCY IS ''

DEFAULT FOR PER_ASG_GRADE_LADDER_NAME IS ''

/* DBI FOR Salary Information */

DEFAULT FOR CMP_ASSIGNMENT_SALARY_FTE_VALUE IS 1

DEFAULT FOR CMP_ASSIGNMENT_SALARY_AMOUNT IS -1

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR IS -1

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS -1

DEFAULT FOR CMP_ASSIGNMENT_SALARY_BASIS_CODE IS ''

DEFAULT FOR PER_GRADE_STEP_NAME IS '-1'

/* Input Values */

INPUTS ARE CMP_IV_GRADE_LADDER_ID(NUMBER), CMP_IV_GRADE_ID (NUMBER),

CMP_IV_STEP_ID(NUMBER),

CMP_IV_GRADE_LADDER_NAME (TEXT), CMP_IV_GRADE_NAME (TEXT), CMP_IV_STEP_NAME (TEXT)

DEFAULT FOR CMP_IV_STEP_NAME IS '-1'

L_temp = ESS_LOG_WRITE('*** Entering Tilak_Lakshmi_GSP_RATE')

```
L_temp = ESS_LOG_WRITE('Ladder name ' + PER_ASG_GRADE_LADDER_NAME )
```

```
L_Grade_annual_factor = 12
```

```
L_Grade_Freq = 'MONTHLY'
```

```
/* find the current asg frequency
```

```
  You do not need this, since the FF written for a Ladder
```

```
*/
```

```
index = PER_ASG_GRADE_RATES_ANNUALIZATION_FACTOR.FIRST(-1)
```

```
WHILE (PER_ASG_GRADE_RATES_ANNUALIZATION_FACTOR.EXISTS(index))
```

```
LOOP (
```

```
  L_temp = ESS_LOG_WRITE('Grade Rate Name ' + PER_ASG_GRADE_RATES_NAME[index] )
```

```
  IF (PER_ASG_GRADE_RATES_NAME[index] = PER_ASG_GRADE_LADDER_NAME) THEN (
```

```
    L_Grade_annual_factor = PER_ASG_GRADE_RATES_ANNUALIZATION_FACTOR[index]
```

```
    L_Grade_Freq      = PER_ASG_GRADE_RATES_RATE_FREQUENCY[index]
```

```
    L_temp= ESS_LOG_WRITE( ' Grade Annualization Factor ' + to_char(L_Grade_annual_factor) )
```

```
    L_temp= ESS_LOG_WRITE( ' Grade Annualization Frequency ' + L_Grade_Freq )
```

```
    EXIT
```

```
  )
```

```
  index = PER_ASG_GRADE_RATES_ANNUALIZATION_FACTOR.NEXT(index ,-1)
```

```
)
```

```
L_temp= ESS_LOG_WRITE( ' CMP_ASSIGNMENT_SALARY_FTE_VALUE ' +
```

```
to_char(CMP_ASSIGNMENT_SALARY_FTE_VALUE) )
```

```
L_temp= ESS_LOG_WRITE( ' CMP_ASSIGNMENT_SALARY_AMOUNT ' +
```

```
to_char(CMP_ASSIGNMENT_SALARY_AMOUNT) )
```

```
L_temp= ESS_LOG_WRITE( ' CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR ' +
```

```
to_char(CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR) )
```

```
L_temp= ESS_LOG_WRITE( ' CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT ' +
```

```
to_char(CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT) )
```

```
L_temp= ESS_LOG_WRITE( ' CMP_ASSIGNMENT_SALARY_BASIS_CODE ' +
```

```
CMP_ASSIGNMENT_SALARY_BASIS_CODE )
```

```
L_temp= ESS_LOG_WRITE( ' PER_GRADE_STEP_NAME ' + PER_GRADE_STEP_NAME )
```

```
L_temp= ESS_LOG_WRITE( ' Eligible STEP_NAME ' + CMP_IV_STEP_NAME )
```

```

L_salary = CMP_ASSIGNMENT_SALARY_AMOUNT
/* if the freq of the grade and salary is not the same */
IF (L_Grade_Freq != CMP_ASSIGNMENT_SALARY_BASIS_CODE) THEN (
    L_salary = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
    /* Convert the annual Salary to Grade rate */
    IF (L_Grade_annual_factor != 1) THEN (
        L_salary = L_salary / L_Grade_annual_factor
    )
)

L_temp= ESS_LOG_WRITE( ' Grade Rate salary ' + to_char(L_salary) )

/* Fte*/
IF (CMP_ASSIGNMENT_SALARY_FTE_VALUE WAS NOT DEFAULTTED) THEN (
    L_salary = L_salary / CMP_ASSIGNMENT_SALARY_FTE_VALUE
    L_temp= ESS_LOG_WRITE( ' Grade Rate salary ' + to_char(L_salary) )
)
L_temp= ESS_LOG_WRITE( ' FTE Salary ' + to_char(L_salary) )

/* add % to salary */
IF ( (TO_NUMBER(CMP_IV_STEP_NAME) - to_number(PER_GRADE_STEP_NAME)) > 1 ) THEN (
    L_salary =L_salary * 1.4
)ELSE (
    L_salary =L_salary * 1.2
)
L_temp= ESS_LOG_WRITE( ' Adjusted Salary ' + to_char(L_salary) )

L_temp = ESS_LOG_WRITE('Exiting Tilak_Lakshmi_GSP_RATE: ' + TO_CHAR(L_salary) )
Return L_salary

```

Explanation of Formula.

```

/* DBI FOR Ladder Factor and Freq */
DEFAULT_DATA_VALUE FOR PER_ASG_GRADE_RATES_ANNUALIZATION_FACTOR IS 1
DEFAULT_DATA_VALUE FOR PER_ASG_GRADE_RATES_RATE_FREQUENCY is ''
DEFAULT_DATA_VALUE FOR PER_ASG_GRADE_RATES_NAME is ''

```



```

DEFAULT_DATA_VALUE FOR PER_GRADE_RATES_ANNUALIZATION_FACTOR IS 1
DEFAULT_DATA_VALUE FOR PER_GRADE_RATES_RATE_FREQUENCY IS ''
DEFAULT FOR      PER_ASG_GRADE_LADDER_NAME IS ''
/* DBI FOR Salary Information */
DEFAULT FOR CMP_ASSIGNMENT_SALARY_FTE_VALUE IS 1
DEFAULT FOR CMP_ASSIGNMENT_SALARY_AMOUNT IS -1
DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR IS -1
DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS -1
DEFAULT FOR CMP_ASSIGNMENT_SALARY_BASIS_CODE IS ''
DEFAULT FOR PER_GRADE_STEP_NAME IS '-1'

```

Declaration and initialization of Formula. We have declared more DBI than what we actually used in the Formula.

```

/* Input Values */
INPUTS ARE CMP_IV_GRADE_LADDER_ID(NUMBER), CMP_IV_GRADE_ID (NUMBER),
CMP_IV_STEP_ID(NUMBER),
      CMP_IV_GRADE_LADDER_NAME (TEXT), CMP_IV_GRADE_NAME (TEXT), CMP_IV_STEP_NAME
(TEXT)

DEFAULT FOR CMP_IV_STEP_NAME IS '-1'

```

Declaration of Input values. The Step name alone initialized with default value because if the Ladder does not uses the Step, we do not get any errors from using Step Name Input values.

```

L_temp = ESS_LOG_WRITE('*** Entering Tilak_Lakshmi_GSP_RATE' )
L_temp = ESS_LOG_WRITE('Ladder name ' + PER_ASG_GRADE_LADDER_NAME )

```

```

L_Grade_annual_factor = 12
L_Grade_Freq = 'MONTHLY'

```

```

/* find the current asg frequency
You do not need this, since the FF written for a Ladder
*/
index = PER_ASG_GRADE_RATES_ANNUALIZATION_FACTOR.FIRST(-1)
WHILE (PER_ASG_GRADE_RATES_ANNUALIZATION_FACTOR.EXISTS(index))

```

LOOP (

L_temp = ESS_LOG_WRITE('Grade Rate Name ' + PER_ASG_GRADE_RATES_NAME[index])

IF (PER_ASG_GRADE_RATES_NAME[index] =PER_ASG_GRADE_LADDER_NAME) THEN (

L_Grade_annual_factor = PER_ASG_GRADE_RATES_ANNUALIZATION_FACTOR[index]

L_Grade_Freq = PER_ASG_GRADE_RATES_RATE_FREQUENCY[index]

L_temp= ESS_LOG_WRITE(' Grade Annualization Factor ' + to_char(*L_Grade_annual_factor*))

L_temp= ESS_LOG_WRITE(' Grade Annualization Frequency ' + *L_Grade_Freq*)

EXIT

)

index = PER_ASG_GRADE_RATES_ANNUALIZATION_FACTOR.NEXT(*index* ,-1)

)

Annualization Factor is defaulted to 12 and Frequency is defaulted to Monthly. As this Formula is created for a Ladder, we know in advance what is the Frequency and Annualization Factor.

Therefore, we do not need to find the information from the DBI, we can use the hard coded values.

The code that finds the Frequency and Annualization factor do not work for this formula. This is just an example. The reason why it is not working is explained in the 'Limitation of Formula' section.

L_temp= ESS_LOG_WRITE(' CMP_ASSIGNMENT_SALARY_FTE_VALUE ' +
to_char(CMP_ASSIGNMENT_SALARY_FTE_VALUE))

L_temp= ESS_LOG_WRITE(' CMP_ASSIGNMENT_SALARY_AMOUNT ' +
to_char(CMP_ASSIGNMENT_SALARY_AMOUNT))

L_temp= ESS_LOG_WRITE(' CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR ' +
to_char(CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR))

L_temp= ESS_LOG_WRITE(' CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT ' +
to_char(CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT))

L_temp= ESS_LOG_WRITE(' CMP_ASSIGNMENT_SALARY_BASIS_CODE ' +
CMP_ASSIGNMENT_SALARY_BASIS_CODE)

L_temp= ESS_LOG_WRITE(' PER_GRADE_STEP_NAME ' + PER_GRADE_STEP_NAME)

L_temp= ESS_LOG_WRITE(' Eligible STEP_NAME ' + CMP_IV_STEP_NAME)

L_salary = CMP_ASSIGNMENT_SALARY_AMOUNT

/* if the freq of the grade and salary is not the same */

```

IF (L_Grade_Freq != CMP_ASSIGNMENT_SALARY_BASIS_CODE) THEN (
  L_salary = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
  /* Convert the annual Salary to Grade rate */
  IF (L_Grade_annual_factor != 1) THEN (
    L_salary = L_salary / L_Grade_annual_factor
  )
)

```

Converting the salary to the Grade Rate value.

```

L_temp= ESS_LOG_WRITE( ' Grade Rate salary ' + to_char(L_salary) )

```

```

/* Fte*/
IF (CMP_ASSIGNMENT_SALARY_FTE_VALUE WAS NOT DEFAULTTED) THEN (
  L_salary = L_salary / CMP_ASSIGNMENT_SALARY_FTE_VALUE
  L_temp= ESS_LOG_WRITE( ' Grade Rate salary ' + to_char(L_salary) )
)
L_temp= ESS_LOG_WRITE( ' FTE Salary ' + to_char(L_salary) )

```

Applying FTE to the Grade Rate Value.

```

/* add % to salary */
IF ( (TO_NUMBER(CMP_IV_STEP_NAME) - to_number(PER_GRADE_STEP_NAME)) > 1 ) THEN (
  L_salary =L_salary * 1.4
)ELSE (
  L_salary =L_salary * 1.2
)
L_temp= ESS_LOG_WRITE( ' Adjusted Salary ' + to_char(L_salary) )

```

Giving raise a per the experience.

```

L_temp = ESS_LOG_WRITE('Exiting Tilak_Lakshmi_GSP_RATE: ' + TO_CHAR(L_salary) )
Return L_salary

```

Returning the Final Value.

Limitation of the Formula:

- 1) No DBI at Steps level to find the rate is available.

2) The sample code is using the Grade level DBI to find the Rate and there is no rate defined at the Grade level.

Log

```
*** Entering Tilak_Lakshmi_GSP_RATE
Ladder name Tilak-Lakshmi Test Ladder
Grade Rate Name GSP Midwest Customer Support Grades
CMP_ASSIGNMENT_SALARY_FTE_VALUE .5
CMP_ASSIGNMENT_SALARY_AMOUNT 1001
CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR 12
CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT 12012
CMP_ASSIGNMENT_SALARY_BASIS_CODE MONTHLY
PER_GRADE_STEP_NAME 1
Eligible STEP_NAME 2
Grade Rate salary 1001
Grade Rate salary 2002
FTE Salary 2002
Adjusted Salary 2402.4
Exiting Tilak_Lakshmi_GSP_RATE: 2402.4
```

Formula use case For Calculating Date:

Honestly, we were not able to find a good use case for calculating the date. We are not sure if the following use case has any practical purpose but it may help you to understand the basics of the Data Calculation Fast Formula. We decided to determine the date from the Frequency of the salary. If the Salary Frequency is Annual, the Date will be the first of next year. If it is Monthly then the Date will be the first of next month.

Otherwise, the Effective date will be returned.

Formula Type: Salary progression rate calculation Rule

```
/*****
```

Formula Type: Salary Progression Date Determination Rule

```
*****/
```

```

DEFAULT FOR PER_ASG_GRADE_LADDER_NAME IS ''
/* DBI FOR Salary Information */
DEFAULT FOR CMP_ASSIGNMENT_SALARY_BASIS_CODE IS ''

/* Input Values */
INPUTS ARE CMP_IV_GRADE_LADDER_ID(NUMBER), CMP_IV_GRADE_ID (NUMBER),
CMP_IV_STEP_ID(NUMBER),
        CMP_IV_GRADE_LADDER_NAME (TEXT), CMP_IV_GRADE_NAME (TEXT), CMP_IV_STEP_NAME (TEXT)

L_temp = ESS_LOG_WRITE('*** Entering Tilak_Lakshmi_GSP_DATE' )
L_temp = ESS_LOG_WRITE('Ladder name ' + PER_ASG_GRADE_LADDER_NAME )

L_date = GET_CONTEXT(EFFECTIVE_DATE, '2000/01/10' (date))
L_date_c = to_char(L_date, 'YYYY/MM/DD')

L_temp = ESS_LOG_WRITE('effective Date ' + L_date_c )
L_temp = ESS_LOG_WRITE('Salary Frequency ' + CMP_ASSIGNMENT_SALARY_BASIS_CODE )

/*Calculate the Date */
IF (CMP_ASSIGNMENT_SALARY_BASIS_CODE = 'ANNUALLY') THEN (

        L_DATE = TRUNC(ADD_MONTHS(L_DATE, 12), 'YYYY')
)ELSE (
        IF (CMP_ASSIGNMENT_SALARY_BASIS_CODE = 'MONTHLY') THEN (

                L_DATE = TRUNC(ADD_MONTHS(L_DATE, 12), 'MM')
        )
)
L_date_c = to_char(L_date, 'YYYY/MM/DD')

L_temp = ESS_LOG_WRITE('Exiting Tilak_Lakshmi_GSP_DATE: '+L_date_c )
Return L_date_c

```

The Formula is pretty self-explanatory. If the Salary Frequency is Annually, 12 months is added to effective date and First date of the year is calculated from the date. If the Frequency is Monthly, 1 month is added to the effective date and First date of the month is calculated from the Date. The result is converted into 'YYYY/MM/DD' date format and the value is returned.

Log

```
*** Entering Tilak_Lakshmi_GSP_DATE
Ladder name Tilak-Lakshmi Test Ladder
effective Date 2016/07/30
Salary Frequency MONTHLY
Exiting Tilak_Lakshmi_GSP_DATE: 2016/08/01
```

This completes the Rate and Date formulas. As usual, if you have any questions or suggestions, please feel free to comment.

You Tube Tutorial:

GSP Person Selection Formula 2 - Oracle Fusion HCM Fast Formula Tutorial-9

```
/*
Name: GSP_PERSON_SELECTION2
Type: Compensation Person Selection
Purpose: Skip the person who is already in ceiling step

***** /
/* DBI */
DEFAULT_DATA_VALUE FOR PER_GRD_GRADE_HISTORY_CEILING_STEP_ID IS -1
```

```

DEFAULT_DATA_VALUE FOR PER_GRD_GRADE_HISTORY_EFFECTIVE_START_DATE IS '1900/01/01 00:00:00' (date)
DEFAULT_DATA_VALUE FOR PER_GRD_GRADE_HISTORY_EFFECTIVE_END_DATE IS '1900/01/01 00:00:00' (date)
DEFAULT FOR PER_GRADE_STEP_NAME IS ''
/* input Values */
INPUTS ARE CMP_IV_GRADE_LADDER_ID (NUMBER)
l_data = set_log('Entering GSP_PERSON_SELECTION2' )
grade = GET_CONTEXT(GRADE_ID, -1)
eDate = GET_CONTEXT(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date) )
l_data = set_log('Ladder ' || TO_CHAR(CMP_IV_GRADE_LADDER_ID) )
l_data = set_log('Grade ' || TO_CHAR(grade) )
l_data = set_log('EFF DATE ' || TO_CHAR(eDate) )
l_data = set_log('Person Grade Step Name ' || PER_GRADE_STEP_NAME )
/* Find the ceiling id of eff date */
RETVAL = 'Y'
l_step_id = -1
index = PER_GRD_GRADE_HISTORY_CEILING_STEP_ID.FIRST( -1)
WHILE (PER_GRD_GRADE_HISTORY_CEILING_STEP_ID.EXISTS(index))
LOOP (
  lstDate = PER_GRD_GRADE_HISTORY_EFFECTIVE_START_DATE[index]
  lenDate = PER_GRD_GRADE_HISTORY_EFFECTIVE_END_DATE[index]
  l_data = set_log('CEILING Dates ' || TO_CHAR(lstDate) || ' / ' || TO_CHAR(lenDate))
  if (edate >= lstDate AND edate <= lenDate ) THEN (
    l_step_id = PER_GRD_GRADE_HISTORY_CEILING_STEP_ID[index]
    l_data = set_log('Step ID ' || TO_CHAR(l_step_id) )
    exit
  )
  index = PER_GRD_GRADE_HISTORY_CEILING_STEP_ID.NEXT(index, -1 )
)
/* eof loop */
IF ( l_step_id != -1 ) THEN (
  lparam = '|=STEP_ID=' || TO_CHAR(l_step_id)
  l_data = set_log('Person Grade Step Name ' || lparam )
  l_step_name = GET_VALUE_SET('GSP_GET_STEP_NAME' , lparam)
  l_data = set_log('Ceil Step Name ' || l_step_name )
  IF ( PER_GRADE_STEP_NAME = l_step_name )THEN (
    RETVAL = 'N'
  )
)
l_data = set_log('Leaving GSP_PERSON_SELECTION2 ' || RETVAL )
RETURN RETVAL

```

Grade Steps Progressions (GSP) Fast Formula and Seniority Dates.

In our previous blogs, we discussed about the [Eligibility, Rate and Date](#) Fast Formulas for GSP. In this blog, we will continue discussing with GSP Eligibility and Seniority Dates.

Seniority Dates: Seniority date is the date on which the person status/attribute changes started. In other words, it is the latest effective start date of a person's attributes like Position, Job, Grade or Steps etc. **It could be more complicated with adjustment but for the purpose of GSP, we are not getting into the details.**

This Seniority date helps to find the length of time the person remained or worked on a status without further changes. For example, if we need to find the number of days a person has worked in a Step or Grade, we need to know the start date of the Step or Grade changes. This is what we call as Seniority Date.

Many times, we see that the eligibility of a Step or Grade depends on the number of days, months or hours the person has worked on the current Step or Grade. For example, a person is eligible for the next step only after 6 months of previous step changes.

In this blog, we discuss the different possible approaches that are available to meet the requirements and the strengths and the limitations of each of these approaches.

Fast Formula: If the requirement is simple like if a person is eligible for a Step or if the person spends 6 or more months on current Step, we can write a formula where we can change the context of EFFECTIVE_DATE to prior to 6 months and get the Step Name. If the step name is same as the current Step, then we can make the Person eligible for the next step.

/*

Formula Type: Participation and Rate Eligibility

*/

/* DBI Used in the FF */

DEFAULT FOR PER_GRADE_STEP_NAME IS '-1'

L_temp = ESS_LOG_WRITE('***Entering Tilak_Lakshmi_Eligibility_Elig_FF***')


```

/* Get context Values */
L_PERSON_ID = GET_CONTEXT(PERSON_ID, -1)
L_ASG_ID   = GET_CONTEXT(HR_ASSIGNMENT_ID, -1)
L_EFF_DATE = get_context(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date))

/* Local Variables */

ELIGIBLE  = 'N'
l_current_step = PER_GRADE_STEP_NAME
l_Prev_step = ''
l_Prev_date = ADD_MONTHS(L_EFF_DATE, -6)

L_temp = ESS_LOG_WRITE('Person ID : ' + to_char(L_PERSON_ID) )
L_temp = ESS_LOG_WRITE('Asg ID : ' + to_char(L_ASG_ID) )
L_temp = ESS_LOG_WRITE('Effective date : ' + to_char(L_EFF_DATE) )
L_temp = ESS_LOG_WRITE('current Step : ' + l_current_step )
L_temp = ESS_LOG_WRITE('Effective date - 6 : ' + to_char(l_Prev_date) )
/* Change the context to 6 month past*/
CHANGE_CONTEXTS(EFFECTIVE_DATE=l_Prev_date)(

    l_Prev_step = PER_GRADE_STEP_NAME
    L_temp = ESS_LOG_WRITE('Prev Step : ' + l_Prev_step )
)

IF (l_current_step = l_Prev_step) THEN (
    ELIGIBLE = 'Y'
)
L_temp = ESS_LOG_WRITE('Leaving Tilak_Laskshmi_Eligibility_Elig_FF : ' + ELIGIBLE)
RETURN ELIGIBLE

```

If the requirement is not that simple, we can make use of the Oracle HCM Seniority Module to calculate the seniority date.

HCM Seniority Date: HCM Seniority date calculation is supported for different attributes of a person to calculate the seniority date. As of now the following attributes are supported. Please check with your support to see the attributes that are supported.

- Business Unit
- Bargaining Unit
- Collective Agreement
- Country
- Department
- Enterprise
- Grade
- Grade Step
- Job
- Legal Employer
- Location
- Position
- Union

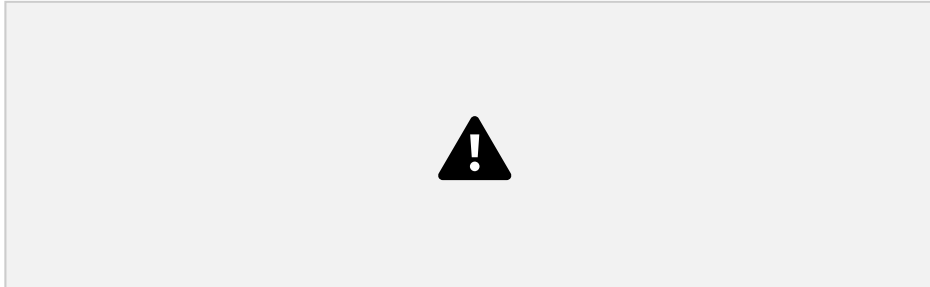
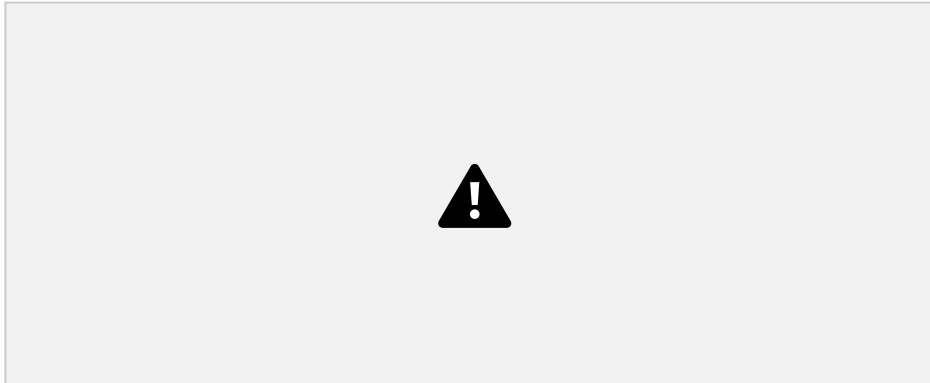
Since the GSP is related to Steps and Grades, we will discuss the setup and usage of Steps seniority date for our example. The setup and usage of the seniority date are the same for all attributes so that you can apply the same steps to all the Seniority dates.

Definition of Seniority Date: Seniority date definition is based on the lookup code from the lookup type, *ORA_PER_SENIORITY_ITEMS*. We need to create our custom lookup code as a first step of Seniority Date definition. For our example, we create a lookup code, *GSP_TL_STEP_CODE* as follows.



Once the Lookup Code definition is done, we need to attach the seniority date attribute to the Lookup Code as follows.

Setup and Maintenance > Configure Seniority Dates



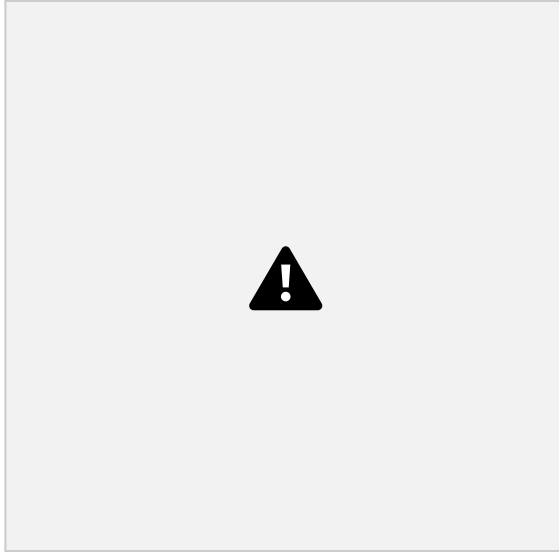
Once the Seniority date setup is done on the basis of Step, we need to generate Seniority date for the person. *(Again for the purpose of GSP, we are using simplest possible setups. If you need to understand the Seniority setup details please refer the setup documents.)*

Person Management -> Manage Employment -> (Menu)->Manage Seniority Dates.

If this is the first time, there may not be any data for the seniority. Run the 'Recalculate Seniority' to generate the date for your seniority setup. This will trigger the ESS process on the background to generate the data. Once the process is completed, you can see the seniority date for your setup.



If you want to process the Seniority date for more than one person, you can use the ESS process, ***Calculate Seniority Dates*** provided by the Seniority module.



Seniority dates and Fast Formula.

The Seniority Date module has provided enough DBIs to access the Seniority data. Our sample Formula helps you to understand the usage of Seniority Date DBIs with GSP Formulas. For our use case, we used the similar requirement from our [eligibility blog](#). The Eligibility Example was based on the experience/Seniority date of the Organization/Length of Service. For this blog, we change the logic to validate the seniority date in a Step. If a person has worked three or more years in the current Step, the person will be eligible for the third Step. If a person has worked one to three years on the current Step, the person will be eligible for the next step; otherwise, the person is not eligible.

/*

Formula Type: Participation and Rate Eligibility

*/

/ DBI Used in the FF */*

DEFAULT FOR PER_PER_LATEST_REHIRE_DATE IS '1900/01/01 00:00:00'(date)

DEFAULT FOR PER_GRADE_STEP_NAME IS '-1'

/ Seniority Date */*

/ Seniority Date */*

DEFAULT_DATA_VALUE FOR PER_SENDR_F_V3_LEVEL_OBJECT_ID is -1

DEFAULT_DATA_VALUE FOR PER_SENDR_F_V3_SENIORITY_DATE is '1901/01/01' (date)

DEFAULT_DATA_VALUE FOR PER_SENDR_F_V3_SENIORITY_DATE_CODE is ''

```

/* Input Values */
INPUTS ARE CMP_IV_GRADE_LADDER_ID(NUMBER), CMP_IV_GRADE_ID (NUMBER),
CMP_IV_STEP_ID(NUMBER),
        CMP_IV_GRADE_LADDER_NAME (TEXT), CMP_IV_GRADE_NAME (TEXT),CMP_IV_STEP_NAME (TEXT)

```

```

L_temp = ESS_LOG_WRITE('***Entering Tilak_Lakshmi_Eligibility_Elig_FF***')

```

```

/* Get context Values */
L_PERSON_ID = GET_CONTEXT(PERSON_ID, -1)
L_ASG_ID   = GET_CONTEXT(HR_ASSIGNMENT_ID, -1)
L_EFF_DATE = get_context(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date))
L_1_ANIV   = L_EFF_DATE
L_3_ANIV   = L_EFF_DATE

```

```

/* Local Variables */
L_ELIG     = 'N'
ELIGIBLE   = 'N'

```

```

L_temp = ESS_LOG_WRITE('Person ID : ' + to_char(L_PERSON_ID) )
L_temp = ESS_LOG_WRITE('Asg ID : ' + to_char(L_ASG_ID) )
L_temp = ESS_LOG_WRITE('Effective date : ' + to_char(L_EFF_DATE) )

```

```

L_temp = ESS_LOG_WRITE('Current Step : ' + PER_GRADE_STEP_NAME )
L_temp = ESS_LOG_WRITE('Validating STEP : ' + CMP_IV_STEP_NAME )

```

```

/* Get the value from WSA */
L_ELIG_ASG = WSA_GET('WSA_GSP_ELIG_PERSON', -1)
L_temp = ESS_LOG_WRITE('Elig ASG ID : ' + TO_CHAR( L_ELIG_ASG ) )

```

```

/* Calculate the Seniority Date */

```

```

l_index = PER_SENDR_F_V3_SENIORITY_DATE.FIRST(-1)
L_temp = ESS_LOG_WRITE('First index of Seniority date : ' + TO_CHAR( l_index ) )
IF (l_index <> -1) THEN (

```

```

WHILE PER_SENDR_F_V3_SENIORITY_DATE.EXISTS(l_index)
LOOP (
    /* When the assignment is the same and lookup code is GSP_TL_STEP_CODE get the date */
    L_temp = ESS_LOG_WRITE('Seniority Object ID : ' + TO_CHAR(
PER_SENDR_F_V3_LEVEL_OBJECT_ID[l_index] ))
    L_temp = ESS_LOG_WRITE('Seniority Date Code : ' + PER_SENDR_F_V3_SENIORITY_DATE_CODE[l_index] )
    IF (PER_SENDR_F_V3_LEVEL_OBJECT_ID[l_index] = L_ASG_ID AND
PER_SENDR_F_V3_SENIORITY_DATE_CODE[l_index]='GSP_TL_STEP_CODE' ) THEN (

        L_1_ANIV = ADD_DAYS(PER_SENDR_F_V3_SENIORITY_DATE[l_index], 365)
        L_3_ANIV = ADD_DAYS(PER_SENDR_F_V3_SENIORITY_DATE[l_index], 1095 )
        L_temp = ESS_LOG_WRITE('Seniority Date : ' + TO_CHAR( PER_SENDR_F_V3_SENIORITY_DATE[l_index] ))
        L_temp = ESS_LOG_WRITE('First Anniversary : ' + to_char(L_1_ANIV) )
        L_temp = ESS_LOG_WRITE('Third Anniversary : ' + to_char(L_3_ANIV) )
        EXIT
    )
    l_index = PER_SENDR_F_V3_SENIORITY_DATE.NEXT(l_index, -1)
)
)

/* IF the asg already eligible */
IF (L_ELIG_ASG = L_ASG_ID ) THEN
(
    L_temp = ESS_LOG_WRITE('Assignment is already eligible for a Step')
    L_ELIG = 'Y'
)

/* not eligible yet */
IF (L_ELIG = 'N') THEN
(
    /* Three year anniversary passed */
    IF (L_3_ANIV <= L_EFF_DATE ) THEN
    (
        L_temp = ESS_LOG_WRITE('Assignment has worked more then 3 years')
        IF (PER_GRADE_STEP_NAME = '1' AND CMP_IV_STEP_NAME = '3') THEN
        (
            ELIGIBLE = 'Y'
        )ELSE (

```

```

    IF (PER_GRADE_STEP_NAME = '2' AND CMP_IV_STEP_NAME = '4') THEN
    (
        ELIGIBLE  = 'Y'
    )ELSE (
        IF (PER_GRADE_STEP_NAME = '3' AND CMP_IV_STEP_NAME = '5') THEN
        (
            ELIGIBLE  = 'Y'
        )
    )
    )
) ELSE
(
    /* One year anniversary */
    L_temp = ESS_LOG_WRITE('Assignment has worked more then 1 year')
    IF (L_1_ANIV <= L_EFF_DATE ) THEN
    (
        IF (PER_GRADE_STEP_NAME = '1' AND CMP_IV_STEP_NAME = '2') THEN
        (
            ELIGIBLE  = 'Y'
        )ELSE (
            IF (PER_GRADE_STEP_NAME = '2' AND CMP_IV_STEP_NAME = '3') THEN
            (
                ELIGIBLE  = 'Y'
            ) ELSE (
                IF (PER_GRADE_STEP_NAME = '3' AND CMP_IV_STEP_NAME = '5') THEN
                (
                    ELIGIBLE  = 'Y'
                )
            )
        )
    )
)

)

/* if eligible store the value */
IF (ELIGIBLE = 'Y') THEN
(

```



```
WSA_SET('WSA_GSP_ELIG_PERSON', L_ASG_ID)
```

```
)
```

```
L_temp = ESS_LOG_WRITE('Leaving Tilak_Laskshmi_Eligibility_Elig_FF : ' + ELIGIBLE)
```

```
RETURN ELIGIBLE
```

Explanation of Fast Formula.

Since the common logic is explained in the [eligibility blog](#), we are not going to repeat the same here. We will go through the changes that are related to Seniority DBIs.

```
/*  
Formula Type: Participation and Rate Eligibility  
  
*/  
/* DBI Used in the FF */  
DEFAULT FOR PER_PER_LATEST_REHIRE_DATE IS '1900/01/01  
00:00:00'(date)  
DEFAULT FOR PER_GRADE_STEP_NAME IS '-1'  
/* Seniority Date */  
/* Seniority Date */  
DEFAULT_DATA_VALUE FOR PER_SENDT_F_V3_LEVEL_OBJECT_ID is -1  
DEFAULT_DATA_VALUE FOR PER_SENDT_F_V3_SENIORITY_DATE is  
'1901/01/01'(date)  
DEFAULT_DATA_VALUE FOR PER_SENDT_F_V3_SENIORITY_DATE_CODE is ''
```

For our Example we use only three DBIs from Seniority Date. These Seniority DBIs use Person_ID and Effective_Date as Contexts.

Level Object Id indicates the level where the seniority is defined. In our case it is Assignment.

If there is only one assignment Id for a person, you may not need this DBI. If you have more than one then you need to validate this DBI to identify the Assignment because the data are for a person. If there is only one assignment Id for a person, you may not need this DBI. If you have more than one, then you need to validate this DBI to identify the Assignment because the data are for a person. The DBI, PER_SENDT_F_V3_SENIORITY_DATE_CODE represents the lookup code. There might be more than one seniority date setup so this DBI helps validating the intended date code to make sure you are getting the correct seniority date.

Another DBI you can consider is PER_SENDR_F_V3_SENIORITY_FIELD_KEY. This stores the Step_id, in case if you want to validate the Step id with the Current Step Id of the person.

```

/* Calculate the Seniority Date */

L_index = PER_SENDR_F_V3_SENIORITY_DATE.FIRST(-1)
L_temp = ESS_LOG_WRITE('First index of Seniority date : ' + TO_CHAR( L_index ) )
IF (L_index <> -1) THEN (

  WHILE PER_SENDR_F_V3_SENIORITY_DATE.EXISTS(L_index)
  LOOP (
    /* When the assignment is the same and lookup code is GSP_TL_STEP_CODE get the date */
    L_temp = ESS_LOG_WRITE('Seniority Object ID : ' + TO_CHAR(
PER_SENDR_F_V3_LEVEL_OBJECT_ID[L_index] ) )
    L_temp = ESS_LOG_WRITE('Seniority Date Code : ' +
PER_SENDR_F_V3_SENIORITY_DATE_CODE[L_index] )
    IF (PER_SENDR_F_V3_LEVEL_OBJECT_ID[L_index] = L_ASG_ID AND
PER_SENDR_F_V3_SENIORITY_DATE_CODE[L_index]='GSP_TL_STEP_CODE' ) THEN (

      L_1_ANIV = ADD_DAYS(PER_SENDR_F_V3_SENIORITY_DATE[L_index], 365)
      L_3_ANIV = ADD_DAYS(PER_SENDR_F_V3_SENIORITY_DATE[L_index], 1095 )
      L_temp = ESS_LOG_WRITE('Seniority Date : ' + TO_CHAR(
PER_SENDR_F_V3_SENIORITY_DATE[L_index] ) )
      L_temp = ESS_LOG_WRITE('First Anniversary : ' + to_char(L_1_ANIV) )
      L_temp = ESS_LOG_WRITE('Third Anniversary : ' + to_char(L_3_ANIV) )
      EXIT
    )
    L_index = PER_SENDR_F_V3_SENIORITY_DATE.NEXT(L_index, -1)
  )
)
)

```

Since the DBI are array based, we make sure the Array has values; once the array length is determined, we loop through the array. Since all these DBI are coming from same Route, since they share the same Index, we do not need to validate the index again.

The value of the array validated against the Assignment and Seniority Lookup codes, if they match we get the seniority Date from the DBI. These DBI are extracted for an effective date, we do not expect more than one value for a Seniority date setup. Once we get the seniority date, we calculate the 3 year and 1 year anniversary date from the seniority date and these anniversaries are validate later to determine the eligibility of the step.

Fast Formula Log.

Entering Tilak_Lakshmi_Eligibility_Elig_FF

Person ID : 300100167151081

Asg ID : 300100167151144

Effective date : 2018-08-07T00:00:00.000Z

Current Step : 1

Validating STEP : 2

Elig ASG ID : -1

First index of Seniority date : 1

Seniority Object ID : 300100167151144

Seniority Date Code : GSP_TL_STEP_CODE

Seniority Date : 2014-07-23T00:00:00.000Z

First Anniversary : 2015-07-23T00:00:00.000Z

Third Anniversary : 2017-07-22T00:00:00.000Z

Assignment has worked more then 3 years

Leaving Tilak_Laskshmi_Eligibility_Elig_FF : N

Entering Tilak_Lakshmi_Eligibility_Elig_FF

Person ID : 300100167151081

Asg ID : 300100167151144

Effective date : 2018-08-07T00:00:00.000Z

Current Step : 1

Validating STEP : 3

Elig ASG ID : -1

First index of Seniority date : 1

Seniority Object ID : 300100167151144

Seniority Date Code : GSP_TL_STEP_CODE

Seniority Date : 2014-07-23T00:00:00.000Z

First Anniversary : 2015-07-23T00:00:00.000Z

Third Anniversary : 2017-07-22T00:00:00.000Z

Assignment has worked more then 3 years

Leaving Tilak_Laskshmi_Eligibility_Elig_FF : Y

Entering Tilak_Lakshmi_Eligibility_Elig_FF

Person ID : 300100167151081

Asg ID : 300100167151144

Effective date : 2018-08-07T00:00:00.000Z

Current Step : 1

Validating STEP : 4

Elig ASG ID : 300100167151144

First index of Seniority date : 1

Seniority Object ID : 300100167151144

Seniority Date Code : GSP_TL_STEP_CODE

Seniority Date : 2014-07-23T00:00:00.000Z

First Anniversary : 2015-07-23T00:00:00.000Z

Third Anniversary : 2017-07-22T00:00:00.000Z

Assignment is already eligible for a Step

Leaving Tilak_Laskshmi_Eligibility_Elig_FF : N

*** Entering Tilak_Lakshmi_GSP_DATE

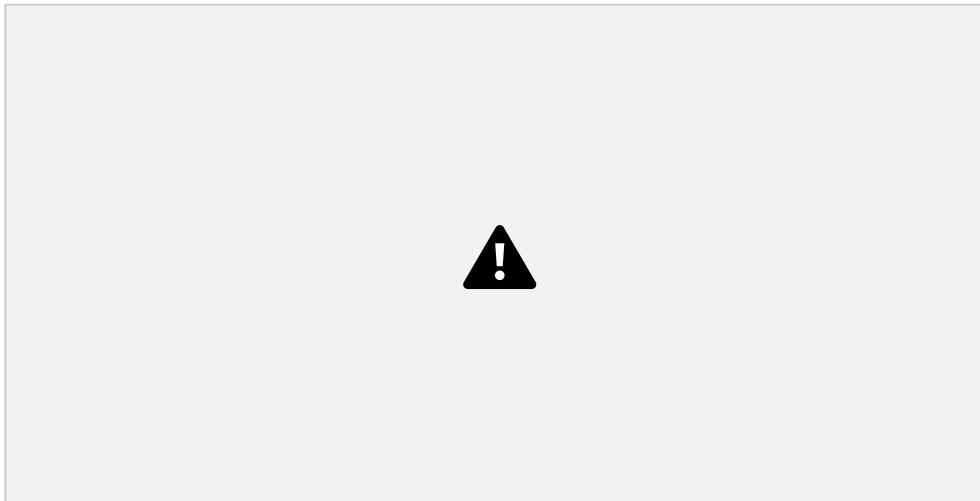
Ladder name Tilak-Lakshmi Test Ladder
effective Date 2018/08/07
Salary Frequency MONTHLY
Exiting Tilak_Lakshmi_GSP_DATE: 2018/09/01

The Logs are self explanatory, we are are not getting into the details of the logs .

Derived Factors and Seniority Dates.

In case you do not want to use the fast formula to validate the seniority dates/length of the status, the Eligibility module has provide an option to use the seniority module in their Derived Factor. Since we are limiting the blogs scope to Fast Formula, we do not want to get into the details of the Derived Factors setup but the image below is self-explanatory to understand the setup.

Derived Factor Setup:



Eligibility with Derived Factors.

Benefit Data in Compensation Formula

Compensation and Benefit Data

In this blog, we try to explain how the Benefit data can be used in Compensation by using Fast Formula. In our example, we use Total Compensation Item Formula, but you can use any Compensation or Grade Steps Formula as these CMP and GSP Fast Formula Types use the same Context.

In our example, we get Benefit Amount and Benefit Coverage Start Date of a plan using Benefit function, BEN_FN_GET_CHAR_VALUE and these values are used as Total Compensation Transaction Date and Total Compensation Transaction Values.

Can we use the Benefit Function, BEN_FN_GET_CHAR_VALUE in Compensation Fast Formula?

No, we cannot use the Benefit Function, BEN_FN_GET_CHAR_VALUE in any Compensation Fast Formula. The Function, BEN_FN_GET_CHAR_VALUE uses the context, HR_ASSIGNMENT_ID, EFFECTIVE_DATE and BUSINESS_GROUP_ID. Unfortunately, Compensation Fast Formula Types do not support the context, BUSINESS_GROUP_ID therefore we cannot use any Benefit DBI or Function in Compensation. Most of the BEN DBI and Functions uses the context, BUSINESS_GROUP_ID.

How do we use Benefit Function or DBI in Compensation Fast Formula?

The only possible way is to use the [parent and child Fast Formula](#) concept where you can use the Benefit Formula as a child Formula. The Benefit Formula may use different Context so that understanding of the Benefit Formula type is crucial before start using it. Once you know the DBI or Function and their context, the Benefit Formula can be selected as per your requirement (need of the Contexts). The value for the context can be passed from the parent (Compensation) Formula and you need to make sure the contexts value that are passed to the child Formulas are available in Compensation before calling the Child (Benefit) Formula (for example, Business Group Id).

How we implemented the COMP and BEN Formula:

We too implemented our Formula using [parent and child Formula](#) concept. As we said we are planning to use Benefit Function, BEN_FN_GET_CHAR_VALUE. This Benefit Function uses the contexts,

HR_ASSIGNMENT_ID, EFFECTIVE_DATE and BUSINESS_GROUP_ID, and different parameters as per the expected values.

Context: Compensation Fast Formula types supports HR_ASSIGNMENT_ID, EFFECTIVE_DATE and BUSINESS_GROUP_ID is not a dynamic value and it is the same to the entire Organization so that we decided to hardcode the value to pass as context to the child Formula.

Parameters: For our requirement we need the parameters, Table name, Column name, Plan name and Option name. Since we know the above value, we are hard coding the parameters values too.

Table name => BEN_PRTT_ENRT_RSLT

Column name => BNFT_AMT/ENRT_CVG_STRT_DT

Plan Name => BEN_AUTO_DEP_DESG_PLAN

Option Name => AUTODESGEMP

Child Formula (Benefit):

/*****

Formula Type : Age Calculation

Formula Name : CMP_BEN_TEST_1

Returns: Coverage Start Date and Benefit Amount

*****/

L_val = SET_LOG('Entering into FF CMP_BEN_TEST_1')

RET_DATE = 'N'

RET_VAL = 'N'

RET_DATE_CHAR = 'N'

L_ASG_ID = GET_CONTEXT(HR_ASSIGNMENT_ID, -1)

L_EFF_DATE = get_context(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date))

L_BG_ID = GET_CONTEXT(BUSINESS_GROUP_ID, -1)

L_val = SET_LOG('Assignment ID ' || to_char(L_ASG_ID))

L_val = SET_LOG('Eff Date ' || to_char(L_EFF_DATE))

```

L_val = SET_LOG('BG Date ' || to_char(L_BG_ID) )

RET_DATE_CHAR = BEN_FN_GET_CHAR_VALUE ('BEN_PRTT_ENRT_RSLT', 'ENRT_CVG_STRT_DT',
'BEN_AUTO_DEP_DESG_PLAN', 'AUTODESGEMP')

RET_VAL = BEN_FN_GET_CHAR_VALUE ('BEN_PRTT_ENRT_RSLT', 'BNFT_AMT', 'BEN_AUTO_DEP_DESG_PLAN',
'AUTODESGEMP')

L_val = SET_LOG('cvg date val ' || RET_DATE_CHAR || '/' || RET_VAL)
/* Date expected to be in yyyy-mm-dd format string
And we change that to yyyy/mm/dd format
*/
IF (LENGTH(RET_DATE_CHAR) = 10) THEN (
RET_DATE = TO_CHAR( TO_DATE( RET_DATE_CHAR), 'YYYY/MM/DD')
)

L_val = SET_LOG('Exiting into FF CMP_BEN_TEST_1 ' || RET_DATE || '/' || RET_VAL)
RETURN RET_DATE , RET_VAL

```

Parent Formula (Total Compensation)

```

/*****
Formula Type : Total Compensation Item
Formula Name : CMP_BEN_TEST
Returns: Coverage Start Date and Benefit Amount
*****/

L_val = SET_LOG('Entering into FF CMP_BEN_TEST')
COMPENSATION_DATES = '2000/01/01'
VALUES = '-1'

L_ASG_ID = GET_CONTEXT(HR_ASSIGNMENT_ID, -1)
L_EFF_DATE = get_context(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date))

L_val = SET_LOG('Assignment ID ' || to_char(L_ASG_ID) )
L_val = SET_LOG('Eff Date ' || to_char(L_EFF_DATE) )

IF (IS_EXECUTABLE('CMP_BEN_TEST_1')) THEN
(
SET_INPUT('BUSINESS_GROUP_ID', 100010025071996)
EXECUTE('CMP_BEN_TEST_1')

```

```

COMPENSATION_DATES = GET_OUTPUT('RET_DATE','2000/01/02')
VALUES = GET_OUTPUT('RET_VAL','-2')

L_DATA = SET_LOG( ' RETURN VALUE OF CHILD '|| COMPENSATION_DATES || ' / '|| VALUES )
)
L_val = SET_LOG('Exiting into FF CMP_BEN_TEST ' || COMPENSATION_DATES || ' / '|| VALUES)

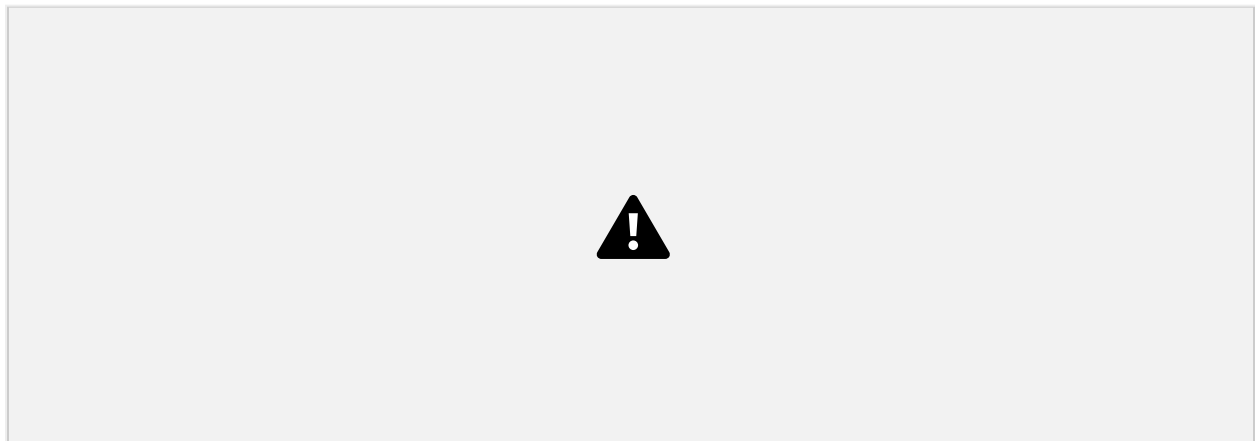
RETURN COMPENSATION_DATES, VALUES

```

Since the above formulas are simple and at this point, we don't think that the Formulas needed any explanation.

We use SET_LOG instead of ESS_LOG_WRITE. This set_log is compensation function this is an alternate to ESS_LOG_WRITE. Advantage of SET_LOG is, if you use the Formula validator/tester in Compensation, this will be useful.

ESS Monitor from Compensation:



The Ess Log provides the output values (Date: 2018/10/12 Value : 10000)

```

called init
Processing person range 238912
Person Type validation is passed
RT processing item : 300100181283432

```


Entering into FF CMP_BEN_TEST
Assignment ID 300100171603642
Eff Date 2019-12-31T00:00:00.000Z
Entering into FF CMP_BEN_TEST_1
Assignment ID 300100171603642
Eff Date 2019-12-31T00:00:00.000Z
BG Date 100010025071996
cvg date val 12-10-2018 /10000

Exiting into FF CMP_BEN_TEST_1 2018/10/12 /10000
RETURN VALUE OF CHILD 2018/10/12 / 10000
Exiting into FF CMP_BEN_TEST 2018/10/12 /10000
Creating employer statement : 100000011593283

Person Status: C

Grade Steps Progression: Incremental salary Formula Sample

The Grade/Step Progression functionality supports updating the worker's salary with the rate associated with the Grade/Step the worker is progressed to. GSP does not support incrementing the worker's current salary with the grade/step rate amount defined.

Requirement: Consider the following scenario:

- Worker on Step 1 and current salary Rate is \$5000.
- After a Length of Service of 1 year on Step 1, the worker must be progressed to Step 2. The rate defined at Step 2 is 300. The worker's current salary should be incremented by \$300.00 i.e.
 $5000 + 300 = 5300$.
- After a Length of Service of 2 year on Step 2, the worker must be progressed to Step 3. The salary needs to be incremented by \$500.00 i.e $5300 + 500 = 5800$.

This allows the initial salary given to each worker can be different (in this case 5000). The rate defined at the step level is the increment amount and not the total salary rate. This requirement cannot be achieved directly by defining grade or step rates but can be achieved using a fast formula to calculate the increment amounts. The following are 2 possible solutions to accommodate the above requirement using fast formula.

Possible Solution 1:

In the fast Formula, find the current Grade / Step of the worker and the salary rate. For each possible grade or step, define the increment amount within the fast formula. Apply the increment amount to the current rate and calculate the new salary rate.

Find the current rate (say rate)

```
If CMP_IV_GRADE_NAME = 'GRADE1' and CMP_IV_STEP_NAME = '2' then (  
    value = 300  
)Else(  
    If CMP_IV_GRADE_NAME = 'GRADE1' and CMP_IV_STEP_NAME = '3' then (  
        value = 500  
    )  
)
```

Then you can add the calculated value to the current grade rate

value = rate (Current Salary) + value

Possible Solution 2:

The incremental rate value is defined for each step in the ladder. Find the step rate in the formula and return the sum of worker's current salary rate and step rate.

In this blog, we are going to discuss the possible solution 2.

Fast Formula:

```
/******  
  
FF Name: TILAK_GSP_SALARY_INCREMENT  
Type: Salary Progression Rate Calculation Rule  
Purpose: Get the current Salary from the Assignment and add the Rate from the GSP Steps To determine the incremented salary  
Author: Tilak  
  
*****/* DBI for the Fast Formula */  
DEFAULT FOR PER_GRADE_STEP_NAME IS '-1'  
DEFAULT_DATA_VALUE FOR PER_GSP_STEP_RATES_STEP_NAME is '-1'  
DEFAULT_DATA_VALUE FOR PER_GSP_STEP_RATES_VALUE_VALUE IS '-1'  
DEFAULT_DATA_VALUE FOR PER_GSP_STEP_RATES_STEP_ID IS -1  
DEFAULT FOR CMP_ASSIGNMENT_SALARY_AMOUNT is 0  
  
/* Input Values */  
  
INPUTS ARE CMP_IV_GRADE_LADDER_ID(NUMBER), CMP_IV_GRADE_ID (NUMBER),  
CMP_IV_GRADE_STEP_ID(NUMBER), GRADE_STEP_ID(NUMBER) ,  
CMP_IV_GRADE_LADDER_NAME (TEXT), CMP_IV_GRADE_NAME (TEXT),CMP_IV_STEP_NAME (TEXT)  
  
DEFAULT FOR CMP_IV_STEP_NAME IS '-1'  
DEFAULT FOR CMP_IV_GRADE_NAME IS '-1'  
DEFAULT FOR CMP_IV_STEP_NAME IS '-1'  
DEFAULT FOR CMP_IV_GRADE_STEP_ID IS -1  
DEFAULT FOR GRADE_STEP_ID IS -1  
  
L_temp = SET_LOG ('*** Entering TILAK_GSP_SALARY_INCREMENT '  
L_salary = 0  
L_eff_date = GET_CONTEXT(EFFECTIVE_DATE, '2000/01/10' (date))
```

```

l_grade_id = GET_CONTEXT(GRADE_ID , -1 )
l_asg_id = GET_CONTEXT(HR_ASSIGNMENT_ID , -1 )

/* print the contexts */
L_temp = SET_LOG ('Context Effective Date ' + to_char(l_eff_date) )
L_temp = SET_LOG ('Context Assignment ' + to_char(l_asg_id) )
L_temp = SET_LOG ('Context Grade ' + to_char(l_grade_id) )

/* print input values */
L_temp = SET_LOG ('IV Grade ' + to_char(CMP_IV_GRADE_ID) )
L_temp = SET_LOG ('IV Grade ' + CMP_IV_GRADE_NAME )
L_temp = SET_LOG ('IV Step ' + to_char(CMP_IV_GRADE_STEP_ID) )
L_temp = SET_LOG ('IV Step ' + CMP_IV_STEP_NAME )
L_temp = SET_LOG ('GRADE_STEP_ID ' + TO_CHAR(GRADE_STEP_ID) )
L_Current_rate = 0

/* Find the current salary from DBI */
L_Current_rate = CMP_ASSIGNMENT_SALARY_AMOUNT
L_temp = SET_LOG ('current Salary Value in loop ' + TO_CHAR(L_Current_rate) )

l_rate_value = '0'

/* get the step rate value */

l_index = PER_GSP_STEP_RATES_VALUE_VALUE.FIRST(-1)
WHILE (PER_GSP_STEP_RATES_VALUE_VALUE.EXISTS(l_index))
LOOP(

/* You can have additional validation on dates */
L_temp = SET_LOG ('Rate Value in loop ' + PER_GSP_STEP_RATES_VALUE_VALUE[l_index] )
L_temp = SET_LOG ('Rate Value step id loop ' + to_char(PER_GSP_STEP_RATES_STEP_ID[l_index]) )

L_temp = SET_LOG ('Rate Value step name loop ' + PER_GSP_STEP_RATES_STEP_NAME[l_index] )

IF (PER_GSP_STEP_RATES_STEP_ID[l_index] = CMP_IV_GRADE_STEP_ID ) THEN ( L_temp = SET_LOG ('selected Rate
Value in loop ' + PER_GSP_STEP_RATES_VALUE_VALUE[l_index] )
l_rate_value = PER_GSP_STEP_RATES_VALUE_VALUE[l_index]
EXIT
)
l_index = PER_GSP_STEP_RATES_VALUE_VALUE.NEXT(l_index ,-1)

)
l_salary = L_Current_rate + to_number(l_rate_value)
L_temp = SET_LOG ('Exiting TILAK_GSP_SALARY_INCREMENT ***** ' + to_char(l_salary) )
RETURN l_salary

```

Partial Log from the Process:

Start Date : 2020-10-01

Start of the range process rangeid: 100000000844440 ,in the thread 24752

Effective date: 01-10-2020 , Assignment Id: 300000007387311

Processing Person : TilakAnalyst1119, Lakshmi

Grades	Step	Defaulted Eligible
--------	------	--------------------

Effective date: 01-11-2020 , Assignment Id: 300000007387311

*** Entering TILAK_GSP_SALARY_INCREMENT

Context Effective Date 2020-11-01T00:00:00.000Z

Context Assignment 300000007387311

Context Grade 300000006552581

IV Grade 300000006552581

IV Grade ANALYST Grade Step IC2\

IV Step 300000006552585

IV Step IC2.2

GRADE_STEP_ID 300000006552584

current Salary Value in loop 2101.11

Rate Value in loop 2101.11

Rate Value step id loop 300000006552584

Rate Value step name loop IC2.1

Rate Value in loop 2202.12

Rate Value step id loop 300000006552585

Rate Value step name loop IC2.2

selected Rate Value in loop 2202.12

Exiting TILAK_GSP_SALARY_INCREMENT ***** 4303.23

Rate from Formula:4303.23

Value / Factor / Annualized rate:4303.23 / 12 / 51638.76

salary/fte /ann fte / proposed salary 25213.32 / 2101.11 / 25213.32 / 51638.76

Proposed salary Fte 4303.23

Formatted proposed Salary 2 / / 4303.23

*** Entering TILAK_GSP_SALARY_INCREMENT

Context Effective Date 2020-11-01T00:00:00.000Z

Context Assignment 300000007387311

Context Grade 300000006552581

IV Grade 300000006552581

IV Grade ANALYST Grade Step IC2

IV Step 300000006552588

IV Step IC2.3

GRADE_STEP_ID 300000006552584

current Salary Value in loop 2101.11

Rate Value in loop 2101.11

Rate Value step id loop 300000006552584

Rate Value step name loop IC2.1

Rate Value in loop 2202.12

Rate Value step id loop 300000006552585

Rate Value step name loop IC2.2

Rate Value in loop 2303.13

Rate Value step id loop 300000006552588

Rate Value step name loop IC2.3

selected Rate Value in loop 2303.13

Exiting TILAK_GSP_SALARY_INCREMENT ***** 4404.24

Rate from Formula:4404.24

Explanation of the Fast Formula:

The following three DBI's are array DBI's based on context **GRADE_ID** and **EFFECTIVE_DATE**.

In other words, the information for the DBI is extracted for a Grade Id and a Date. By default, the Formula uses the person current grade and the effective date of the process.

DEFAULT_DATA_VALUE FOR PER_GSP_STEP_RATES_STEP_NAME is '-1'

DEFAULT_DATA_VALUE FOR PER_GSP_STEP_RATES_VALUE_VALUE IS '-1'

DEFAULT_DATA_VALUE FOR PER_GSP_STEP_RATES_STEP_ID IS -1

If you are using grades and grade rates, you can use the following DBI's instead of Step based DBI's.

PER_GRADE_RATES_NAME

PER_GRADE_RATE_VALUE_VALUE

The following DBI is used to get the employees current Salary. It is based on context

HR_ASSIGNMENT_ID and **EFFECTIVE_DATE**.

DEFAULT FOR CMP_ASSIGNMENT_SALARY_AMOUNT is 0

These following lines are used to declare the Input values. You can find the information about the Grade, steps for which the Formula is executed.

/ Input Values */*

*INPUTS ARE CMP_IV_GRADE_LADDER_ID(NUMBER), CMP_IV_GRADE_ID (NUMBER),
CMP_IV_GRADE_STEP_ID(NUMBER), GRADE_STEP_ID(NUMBER) ,
CMP_IV_GRADE_LADDER_NAME (TEXT), CMP_IV_GRADE_NAME (TEXT),CMP_IV_STEP_NAME
(TEXT)*

The following lines provide a default value to the Input values so that the formula does not error in case the values are not passed from the Fast formula.

DEFAULT FOR CMP_IV_STEP_NAME IS '-1'

DEFAULT FOR CMP_IV_GRADE_NAME IS '-1'

DEFAULT FOR CMP_IV_STEP_NAME IS '-1'

DEFAULT FOR CMP_IV_GRADE_STEP_ID IS -1

DEFAULT FOR GRADE_STEP_ID IS -1

The following lines are used to declaring variables for the current rate and eligible step rate.

L_salary = 0

L_Current_rate = 0

The following line is to find the person current salary value.

L_Current_rate = CMP_ASSIGNMENT_SALARY_AMOUNT

The following loop is to find the rate of all of steps in current grade. Since the DBI gets all the step rates, we need to validate the step that we are looking for.

If you are hard coding the value for Steps or Grade, you can implement your logic here instead of querying the value from the eligible Grades and Steps.

```

l_rate_value = '0'
/* get the step rate value */
l_index = PER_GSP_STEP_RATES_VALUE_VALUE.FIRST(-1)
WHILE (PER_GSP_STEP_RATES_VALUE_VALUE.EXISTS(l_index))
LOOP(

    /* You can have additional validation on dates */
    L_temp = SET_LOG ('Rate Value in loop ' + PER_GSP_STEP_RATES_VALUE_VALUE[l_index]
    L_temp = SET_LOG ('Rate Value step id loop ' +
to_char(PER_GSP_STEP_RATES_STEP_ID[l_index]) )
    L_temp = SET_LOG ('Rate Value step name loop ' + PER_GSP_STEP_RATES_STEP_NAME[l_index]
)

```

When the DBI step id matches the step id for which formula is executed, we get the value from the DBI.

```

IF (PER_GSP_STEP_RATES_STEP_ID[l_index] = CMP_IV_GRADE_STEP_ID ) THEN (
    L_temp = SET_LOG ('selected Rate Value in loop ' +
PER_GSP_STEP_RATES_VALUE_VALUE[l_index] )
    l_rate_value = PER_GSP_STEP_RATES_VALUE_VALUE[l_index]
    EXIT
)
l_index = PER_GSP_STEP_RATES_VALUE_VALUE.NEXT(l_index ,-1)
)

```

Once we get the current salary and the Step rate, we add them and return the final value.

```

l_salary = L_Current_rate + to_number(l_rate_value)
L_temp = SET_LOG ('Exiting TILAK_GSP_SALARY_INCREMENT ***** ' + to_char(l_salary) )
RETURN l_salary

```

This formula works on the person current grade.

If your requirement is to write a formula that evaluates for a different grade, you need to change the context of the Grade for which formula is executed.

```

/* get the step rate value */

CHANGE_CONTEXTS(GRADE_ID= CMP_IV_GRADE_ID)
(

```

```

l_index = PER_GSP_STEP_RATES_VALUE_VALUE.FIRST(-1)
WHILE (PER_GSP_STEP_RATES_VALUE_VALUE.EXISTS(l_index))

LOOP(

  /* You can have additional validation on dates */
  L_temp = SET_LOG ('Rate Value in loop ' + PER_GSP_STEP_RATES_VALUE_VALUE[l_index] )
  L_temp = SET_LOG ('Rate Value step id loop ' +
to_char(PER_GSP_STEP_RATES_STEP_ID[l_index]) )
  L_temp = SET_LOG ('Rate Value step name loop ' + PER_GSP_STEP_RATES_STEP_NAME[l_index]
)
  IF (PER_GSP_STEP_RATES_STEP_ID[l_index] = CMP_IV_GRADE_STEP_ID ) THEN (
    L_temp = SET_LOG ('selected Rate Value in loop ' +
PER_GSP_STEP_RATES_VALUE_VALUE[l_index] )
    l_rate_value = PER_GSP_STEP_RATES_VALUE_VALUE[l_index]
    EXIT
  )
  l_index = PER_GSP_STEP_RATES_VALUE_VALUE.NEXT(l_index ,-1)
)
)
)

```

Grade Step Progression (GSP) Ladder Level Eligibility Fast Formula

YouTube Tutorial: GSP Ladder Eligible Formula 2 - Oracle Fusion HCM Fast Formula Tutorial-11

Please See the Explanation in the YouTube Tutorial

/*

Name: TLK_GSP_ELIG_CHILD

Type: Compensation Person Selection

Purpose: Skip the person who is already in ceiling step

*****/

/* DBI */

DEFAULT_DATA_VALUE FOR PER_GRD_GRADE_HISTORY_CEILING_STEP_ID IS -1

DEFAULT_DATA_VALUE FOR PER_GRD_GRADE_HISTORY_EFFECTIVE_START_DATE IS

'1900/01/01 00:00:00' (date)

```

DEFAULT_DATA_VALUE FOR PER_GRD_GRADE_HISTORY_EFFECTIVE_END_DATE IS
'1900/01/01 00:00:00' (date)
l_data = set_log('Entering TLK_GSP_ELIG_CHILD' )
grade = GET_CONTEXT(GRADE_ID, -1)
eDate = GET_CONTEXT(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date) )
l_data = set_log('Grade ' || TO_CHAR(grade) )
l_data = set_log('EFF DATE ' || TO_CHAR(eDate) )
STEPID = -1
STEPNAME = ''
/* Find the ceiling id of eff date */
index = PER_GRD_GRADE_HISTORY_CEILING_STEP_ID.FIRST( -1)
WHILE (PER_GRD_GRADE_HISTORY_CEILING_STEP_ID.EXISTS(index))
LOOP (
  lStartDate = PER_GRD_GRADE_HISTORY_EFFECTIVE_START_DATE[index]
  lenDate = PER_GRD_GRADE_HISTORY_EFFECTIVE_END_DATE[index]
  l_data = set_log('CEILING Dates ' || TO_CHAR(lStartDate) || ' / ' || TO_CHAR(lenDate))
  if (eDate >= lStartDate AND eDate <= lenDate ) THEN (
    STEPID = PER_GRD_GRADE_HISTORY_CEILING_STEP_ID[index]
    l_data = set_log('Step ID ' || TO_CHAR(STEPID) )
    exit
  )
  index = PER_GRD_GRADE_HISTORY_CEILING_STEP_ID.NEXT(index, -1 )
)
IF STEPID != -1 THEN (
  lparam = '|=STEP_ID=' || TO_CHAR(STEPID)
  l_data = set_log('Person Grade Step Name ' || lparam )
  STEPNAME = GET_VALUE_SET('GSP_GET_STEP_NAME' , lparam)
  l_data = set_log('Ceil Step Name ' || STEPNAME )
)
l_data = set_log('Leaving GSP_PERSON_SELECTION2 ' || STEPNAME )
RETURN STEPNAME
/*
Name: GSP_PART_ELIG
Type: Participation and Rate Eligibility
Function: Exclude the people who are in step 5 or >
        Exclude the people who are in probation
*****/
DEFAULT FOR PER_GRADE_STEP_NAME IS ''
DEFAULT FOR PER_GRADE_STEP_SEQUENCE IS -1
DEFAULT FOR PER_ASG_DATE_PROBATION_END IS '1900/01/01 00:00:00' (date)
DEFAULT FOR PER_ASG_GRADE_ID is -1
INPUTS ARE CMP_IV_GRADE_LADDER_ID(NUMBER),CMP_IV_GRADE_LADDER_NAME (TEXT),
CMP_IV_PERIOD_START_DATE (DATE), CMP_IV_PERIOD_END_DATE (DATE)
ldata = SET_LOG('Entering GSP_PART_ELIG' )
ELIGIBLE = 'Y'
eDate = GET_CONTEXT(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date) )
asgID = GET_CONTEXT(HR_ASSIGNMENT_ID, -1 )
ldata = SET_LOG('PER_ASG_GRADE_ID ' || TO_CHAR(PER_ASG_GRADE_ID) )

```



```

ldata = SET_LOG('EFFECTIVE_DATE ' || TO_CHAR(eDate) )
ldata = SET_LOG('Asg ID ' || TO_CHAR(asgID) )
ldata = SET_LOG('PER_ASG_DATE_PROBATION_DATE_END
'||TO_CHAR(PER_ASG_DATE_PROBATION_END))
ldata = SET_LOG('PER_GRADE_STEP_SEQUENCE ' || TO_CHAR(PER_GRADE_STEP_SEQUENCE)
)
ldata = SET_LOG('PER_GRADE_STEP_NAME ' || PER_GRADE_STEP_NAME )
ldata = SET_LOG('CMP_IV_GRADE_LADDER_ID ' || TO_CHAR(CMP_IV_GRADE_LADDER_ID ) )
ldata = SET_LOG('CMP_IV_GRADE_LADDER_NAME ' || CMP_IV_GRADE_LADDER_NAME )
ldata = SET_LOG('CMP_IV_PERIOD_START_DATE ' || TO_CHAR(CMP_IV_PERIOD_START_DATE ) )
ldata = SET_LOG('CMP_IV_PERIOD_END_DATE ' || TO_CHAR(CMP_IV_PERIOD_END_DATE ) )
IF PER_GRADE_STEP_SEQUENCE >= 5 THEN (
  ELIGIBLE = 'N'
) ELSE (
  IF PER_ASG_DATE_PROBATION_END >= eDate THEN (
    ELIGIBLE = 'N'
  )ELSE (
    IF PER_ASG_GRADE_ID WAS NOT DEFAULTED THEN (
      IF IS_EXECUTABLE('GSP_CHILD_ELIG') THEN (
        SET_INPUT('GRADE_ID' , PER_ASG_GRADE_ID)
        EXECUTE('GSP_CHILD_ELIG')
        ceil_step_name = GET_OUTPUT('STEPNAME', '')
        ldata = SET_LOG('ceil_step_name ' || ceil_step_name )
        if ceil_step_name != '' AND ceil_step_name = PER_GRADE_STEP_NAME THEN (
          ELIGIBLE = 'N'
        )
      )
    )
  )
)
ldata = SET_LOG('Leaving GSP_PART_ELIG : '|| ELIGIBLE)
Return ELIGIBLE

```

Grade Steps Progression's Grade and Steps Level Eligibility Fast Formula

The Details of this Formula available in Youtube Tutorial:

GSP Grade|Steps Eligibility Formula - Oracle Fusion HCM Fast Formula Tutorial-12

Child Fast Formula

/*Formula Name: GSP_PERSON_CEILING_STEP

Type : Compensation Person Selection

Purpose: Check of the person is in the ceiling step

*/

```
DEFAULT FOR PER_GRADE_STEP_NAME IS ''
DEFAULT FOR PER_GRADE_STEP_SEQUENCE IS -1
DEFAULT_DATA_VALUE FOR PER_GRD_GRADE_HISTORY_CEILING_STEP_ID IS -1
DEFAULT_DATA_VALUE FOR PER_GSP_STEP_RATES_STEP_NAME IS ''
DEFAULT_DATA_VALUE FOR PER_GSP_STEP_RATES_STEP_SEQUENCE is -1
DEFAULT_DATA_VALUE FOR PER_GSP_STEP_RATES_STEP_ID IS -1
l_data = set_log('Entering GSP_PERSON_CEILING_STEP' )
grade = GET_CONTEXT(GRADE_ID, -1)
eDate = GET_CONTEXT(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date) )
l_data = set_log('Grade ' || TO_CHAR(grade) )
l_data = set_log('EFF DATE ' || TO_CHAR(eDate) )
l_data = set_log('CURR STEP NAME ' || PER_GRADE_STEP_NAME )
l_data = set_log('CURR STEP SEQ ' || TO_CHAR(PER_GRADE_STEP_SEQUENCE) )
STEPID = -1
STEPNAME = ''
STEPSEQ = -1
CEILSTEPID = -1
INCEILSTEP = 'N'
indx = PER_GRD_GRADE_HISTORY_CEILING_STEP_ID.FIRST(-1)
if indx != -1 THEN (
    CEILSTEPID = PER_GRD_GRADE_HISTORY_CEILING_STEP_ID[indx]
)
IF CEILSTEPID != -1 THEN (
    indx = PER_GSP_STEP_RATES_STEP_ID.FIRST(-1)
    WHILE PER_GSP_STEP_RATES_STEP_ID.EXISTS(indx)
    LOOP(
        STEPID = PER_GSP_STEP_RATES_STEP_ID[INDX]
        STEPNAME = PER_GSP_STEP_RATES_STEP_NAME[INDX]
        STEPSEQ = PER_GSP_STEP_RATES_STEP_SEQUENCE[INDX]
        l_data = set_log('STEP ' || TO_CHAR(STEPID) )
        l_data = set_log('SEQ ' || TO_CHAR(STEPSEQ) )
        l_data = set_log('NAME ' || STEPNAME )
        IF CEILSTEPID = STEPID THEN (
            l_data = set_log('FOUND CEILING ' || STEPNAME )
            EXIT
        )
    )
    indx = PER_GSP_STEP_RATES_STEP_ID.NEXT(indx, -1)
)
)ELSE (
    indx = PER_GSP_STEP_RATES_STEP_ID.LAST(-1)
    if indx != -1 THEN (
        STEPID = PER_GSP_STEP_RATES_STEP_ID[INDX]
        STEPNAME = PER_GSP_STEP_RATES_STEP_NAME[INDX]
        STEPSEQ = PER_GSP_STEP_RATES_STEP_SEQUENCE[INDX]
        l_data = set_log('LAST STEP ' || TO_CHAR(STEPID) )
        l_data = set_log('LAST SEQ ' || TO_CHAR(STEPSEQ) )
        l_data = set_log('LAST NAME ' || STEPNAME )
    )
)
)
IF STEPID != -1 THEN (
```

```

IF PER_GRADE_STEP_NAME = STEPNAME THEN (
  INCEILSTEP = 'Y'
  l_data = set_log('YES IN CEILING ' || STEPNAME )
)
)
l_data = set_log('Leaving GSP_PERSON_CEILING_STEP' )
RETURN INCEILSTEP
*****

```

Parent Fast Formula

```

/*
Name: GSP_PERSON_ELIG
Formula Type : Participation and Rate Eligibility
Purpose: Person should have worked in the current step for a year to be
        eligible for next step.
        Person is eligible for only next step in the same Grade
        If the person is already in the ceiling step of current Grade
        then the person is eligible for next grade and the first step
*/
DEFAULT FOR PER_GRADE_STEP_NAME IS ''
DEFAULT FOR PER_GRADE_STEP_SEQUENCE IS -1
DEFAULT FOR PER_ASG_GRADE_ID IS -1
DEFAULT FOR PER_ASG_GRADE_NAME IS ''
DEFAULT FOR PER_GRADE_STEP_EFFECTIVE_START_DATE IS '1900/01/01 00:00:00' (date)
DEFAULT FOR CMP_IV_STEP_ID IS -1
DEFAULT FOR CMP_IV_STEP_NAME IS ''
INPUTS ARE CMP_IV_GRADE_LADDER_ID(NUMBER),CMP_IV_GRADE_LADDER_NAME (TEXT),
        CMP_IV_PERIOD_START_DATE (DATE), CMP_IV_PERIOD_END_DATE (DATE) ,
        CMP_IV_GRADE_ID (NUMBER), CMP_IV_GRADE_NAME (TEXT), CMP_IV_GRADE_SEQ (NUMBER) ,
        CMP_IV_STEP_ID (NUMBER), CMP_IV_STEP_NAME (TEXT)

```

```

ldata = SET_LOG('Entering TLK_GSP_PERSON_ELIG' )
ELIGIBLE = 'Y'
eDate = GET_CONTEXT(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date) )
asgID = GET_CONTEXT(HR_ASSIGNMENT_ID, -1 )
ldata = SET_LOG('PER_ASG_GRADE_ID ' || TO_CHAR(PER_ASG_GRADE_ID) )
ldata = SET_LOG('PER_ASG_GRADE_NAME ' || PER_ASG_GRADE_NAME )
ldata = SET_LOG('PER_GRADE_STEP_SEQUENCE ' || TO_CHAR(PER_GRADE_STEP_SEQUENCE) )
ldata = SET_LOG('PER_GRADE_STEP_NAME ' || PER_GRADE_STEP_NAME )
ldata = SET_LOG('EFFECTIVE_DATE ' || TO_CHAR(eDate) )
ldata = SET_LOG('PER_GRADE_STEP_EFFECTIVE_START_DATE ' ||
TO_CHAR(PER_GRADE_STEP_EFFECTIVE_START_DATE) )
ldata = SET_LOG('Asg ID ' || TO_CHAR(asgID) )
ldata = SET_LOG('CMP_IV_GRADE_LADDER_ID ' || TO_CHAR(CMP_IV_GRADE_LADDER_ID) )
ldata = SET_LOG('CMP_IV_GRADE_LADDER_NAME ' || CMP_IV_GRADE_LADDER_NAME )
ldata = SET_LOG('CMP_IV_GRADE_ID ' || TO_CHAR(CMP_IV_GRADE_ID) )
ldata = SET_LOG('CMP_IV_GRADE_SEQ ' || TO_CHAR(CMP_IV_GRADE_SEQ) )
ldata = SET_LOG('CMP_IV_GRADE_NAME ' || CMP_IV_GRADE_NAME )
ldata = SET_LOG('CMP_IV_STEP_ID ' || TO_CHAR(CMP_IV_STEP_ID) )
ldata = SET_LOG('CMP_IV_STEP_NAME ' || CMP_IV_STEP_NAME )
/* more then a year validation */
IF DAYS_BETWEEN(eDate , PER_GRADE_STEP_EFFECTIVE_START_DATE) < 365 THEN (
  ELIGIBLE = 'N'
  ldata = SET_LOG('inelig due to year validation ' )
)
)

```

```

IF ELIGIBLE = 'Y' THEN (
  inceiling = 'N'
  IF CMP_IV_STEP_ID WAS DEFAULTED THEN (
    ldata = SET_LOG(' In Grade Validation' )
    IF PER_ASG_GRADE_ID WAS NOT DEFAULTED THEN (
      IF IS_EXECUTABLE('GSP_PERSON_CEILING_STEP') THEN(
        SET_INPUT('GRADE_ID', PER_ASG_GRADE_ID )
        EXECUTE('GSP_PERSON_CEILING_STEP')
        inceiling = GET_OUTPUT('INCEILSTEP', 'N')
        ldata = SET_LOG(' inceiling ' ||inceiling )
      )
    )
    /* validate the current and elig grade */
    IF PER_ASG_GRADE_ID != CMP_IV_GRADE_ID THEN (
      ELIGIBLE = 'N'
      IF inceiling = 'Y' THEN (
        IF PER_ASG_GRADE_NAME = 'Analyst 1' and CMP_IV_GRADE_NAME = 'Analyst 2' THEN (
          ELIGIBLE = 'Y'
          ldata = SET_LOG('Next Grade allowed ' || CMP_IV_GRADE_NAME )
        ) ELSE (
          IF PER_ASG_GRADE_NAME = 'Analyst 2' and CMP_IV_GRADE_NAME = 'Analyst 3' THEN (
            ELIGIBLE = 'Y'
            ldata = SET_LOG('Next Grade allowed ' || CMP_IV_GRADE_NAME )
          )
        )
      )
    )
  )
)ELSE (
  ldata = SET_LOG(' In Step Validation' )
  IF CMP_IV_STEP_ID != -1 THEN (
    ELIGIBLE = 'N'
    IF PER_ASG_GRADE_NAME = CMP_IV_GRADE_NAME THEN (
      IF PER_GRADE_STEP_SEQUENCE+1 = TO_NUMBER(CMP_IV_STEP_NAME) THEN (
        ELIGIBLE = 'Y'
        ldata = SET_LOG('Next step allowed ' || CMP_IV_STEP_NAME )
      )
    )ELSE (
      /* moving to new g4ade */
      IF TO_NUMBER(CMP_IV_STEP_NAME) = 1 THEN(
        ELIGIBLE = 'Y'
        ldata = SET_LOG('First step allowed ' || CMP_IV_STEP_NAME )
      )
    )
  )
)
ldata = SET_LOG('Leaving TLK_GSP_PERSON_ELIG '|| ELIGIBLE )
RETURN ELIGIBLE

```

Workforce Compensation Eligibility Formula

/*

FF: CWB_ELIG_FF

Type: Participation and Rate Eligibility

purpose: Make the person eligible if the person worked more than a year

*/

DEFAULT FOR PER_PERSON_ENTERPRISE_HIRE_DATE is '1900/10/01 00:00:00' (date)

DEFAULT FOR PER_PER_FULL_NAME is ''

DEFAULT FOR CMP_IV_COMPONENT_ID IS -1

DEFAULT FOR CMP_IV_PLAN_ELIG_DATE IS ''

DEFAULT FOR CMP_IV_PERFORMANCE_EFF_DATE IS ''

DEFAULT FOR CMP_IV_PROMOTION_EFF_DATE IS ''

DEFAULT FOR CMP_IV_XCHG_RATE_DATE IS ''

INPUTS ARE CMP_IV_PLAN_START_DATE (text), CMP_IV_PLAN_END_DATE (text),

CMP_IV_PLAN_ELIG_DATE(text),CMP_IV_PLAN_EXTRACTION_DATE

(text), CMP_IV_PERFORMANCE_EFF_DATE(text), CMP_IV_PROMOTION_EFF_DATE (text),

CMP_IV_XCHG_RATE_DATE (text), CMP_IV_PLAN_ID (number), CMP_IV_PERIOD_ID(number),

CMP_IV_COMPONENT_ID (number)

l_log= set_log('*****ENTERING CWB_ELIG_FF')

eDate = GET_CONTEXT(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date))

personid = GET_CONTEXT(PERSON_ID, -1)

assignid = GET_CONTEXT(HR_ASSIGNMENT_ID, -1)

l_log= set_log('EFF DATE ' + TO_CHAR(eDate))

l_log= set_log('personid ' + TO_CHAR(personid))

l_log= set_log('assignid ' + TO_CHAR(assignid))

l_log= set_log('PER_PER_FULL_NAME ' + PER_PER_FULL_NAME)

l_log= set_log('CMP_IV_PLAN_ID ' + TO_CHAR(CMP_IV_PLAN_ID))

l_log= set_log('CMP_IV_PERIOD_ID ' + TO_CHAR(CMP_IV_PERIOD_ID))

l_log= set_log('CMP_IV_COMPONENT_ID '+TO_CHAR(CMP_IV_COMPONENT_ID))

l_log= set_log('CMP_IV_PLAN_START_DATE ' + CMP_IV_PLAN_START_DATE)

l_log= set_log('CMP_IV_PLAN_END_DATE ' + CMP_IV_PLAN_END_DATE)

l_log= set_log('CMP_IV_PLAN_ELIG_DATE ' + CMP_IV_PLAN_ELIG_DATE)

```

l_log= set_log('CMP_IV_PLAN_EXTRACTION_DATE ' + CMP_IV_PLAN_EXTRACTION_DATE )
l_log= set_log('CMP_IV_PERFORMANCE_EFF_DATE ' + CMP_IV_PERFORMANCE_EFF_DATE )
l_log= set_log('CMP_IV_PROMOTION_EFF_DATE ' + CMP_IV_PROMOTION_EFF_DATE )
l_log= set_log('CMP_IV_PLAN_START_DATE ' + CMP_IV_PLAN_START_DATE )

ELIGIBLE = 'N'

l_log= set_log('PER_PERSON_ENTERPRISE_HIRE_DATE ' +
TO_CHAR(PER_PERSON_ENTERPRISE_HIRE_DATE) )

IF DAYS_BETWEEN(eDate, PER_PERSON_ENTERPRISE_HIRE_DATE) >= 365 THEN
(
    ELIGIBLE = 'Y'
)

l_log= set_log('*****Leaving CWB_ELIG_FF ' + ELIGIBLE )

RETURN ELIGIBLE

```

Compensation Hierarchy Determination YouTube Tutorial Example

```

/*

FF Type : Compensation Hierarchy Determination
FF Name: CMP_HIERARCHY
Purpose: Pass the persons managers manager as his hierarchy, if the Managers manager not available
        pass manager as hierarchy
*/

DEFAULT FOR PER_ASG_MGR_FULL_NAME is 'XXX'
DEFAULT FOR PER_ASG_MGR_MANAGER_ID is -999
DEFAULT FOR PER_ASG_MGR_MANAGER_ASG_ID is -999
DEFAULT FOR PER_PER_FULL_NAME is ''

INPUTS ARE CMP_IV_PLAN_START_DATE (text), CMP_IV_PLAN_END_DATE (text),
        CMP_IV_PLAN_ELIG_DATE(text),CMP_IV_PLAN_EXTRACTION_DATE(text),
        CMP_IV_PERFORMANCE_EFF_DATE(text), CMP_IV_PROMOTION_EFF_DATE (text),
        CMP_IV_XCHG_RATE_DATE (text), CMP_IV_PLAN_ID (number),
        CMP_IV_PERIOD_ID(number)
l_log= set_log('*****ENTERING CMP_HIERARCHY')
eDate = GET_CONTEXT(EFFECTIVE_DATE, '1900/01/01 00:00:00' (date) )
personid = GET_CONTEXT(      PERSON_ID, -1 )
assignid = GET_CONTEXT( HR_ASSIGNMENT_ID, -1 )

```

```

l_log= set_log('EFF DATE ' + TO_CHAR(eDate) )
l_log= set_log('personid ' + TO_CHAR(personid) )
l_log= set_log('assignid ' + TO_CHAR(assignid) )
l_log= set_log('PER_PER_FULL_NAME ' + PER_PER_FULL_NAME )
l_log= set_log('CMP_IV_PLAN_ID ' + TO_CHAR(CMP_IV_PLAN_ID) )
l_log= set_log('CMP_IV_PERIOD_ID ' + TO_CHAR(CMP_IV_PERIOD_ID) )
l_log= set_log('CMP_IV_PLAN_START_DATE ' + CMP_IV_PLAN_START_DATE )
L_ASSIGNMENT_ID = -999
L_PERSON_ID    = -999
l_name        = ''
if assignid != -1 then (
    L_PERSON_ID    = PER_ASG_MGR_MANAGER_ID
    L_ASSIGNMENT_ID = PER_ASG_MGR_MANAGER_ASG_ID
    l_name        = PER_ASG_MGR_FULL_NAME
    l_log= set_log('Manager Name ' + l_name )
    IF PER_ASG_MGR_MANAGER_ASG_ID WAS NOT DEFAULTED THEN (
        CHANGE_CONTEXTS(HR_ASSIGNMENT_ID = L_ASSIGNMENT_ID)
    )
    assignid1 = GET_CONTEXT( HR_ASSIGNMENT_ID, -1 )
    l_log= set_log('assignid 1 ' + TO_CHAR(assignid1) )
    IF PER_ASG_MGR_MANAGER_ASG_ID WAS NOT DEFAULTED THEN (
        L_PERSON_ID    = PER_ASG_MGR_MANAGER_ID
        L_ASSIGNMENT_ID = PER_ASG_MGR_MANAGER_ASG_ID
        l_name        = PER_ASG_MGR_FULL_NAME
        l_log= set_log('Manager mgr Name ' + l_name )
    )ELSE (
        l_log= set_log('Manager mgr not found ' )
    )
)
)
)
)
l_log= set_log('*****Leaving CMP_HIERARCHY')
RETURN L_ASSIGNMENT_ID, L_PERSON_ID

```

HCM - Multi Facet Advance Search

HCM - Multi Facet Advance Search

Type

```

"types": {

    "facetType": {
        "Default": "boolean",

```

```

        "DisplayAttribute": "string",
        "DisplayName": "string",
        "ElasticAggregationsEnabled": "boolean",
        "ElasticFacetName": "string",
        "EndEndDateolTip": "string",
        "Endpoint": "string",
        "FacetType": "string",
        "Finder": "string",
        "Group": "string",
        "GroupLabel": "string",
        "id": "string",
        "KeyAttribute": "string",
        "labelMapping": [
            {
                "disabled": "boolean",
                "label": "string",
                "value": "string"
            }
        ],
        "QParam": "string",
        "StartEndDateolTip": "string",
        "ToolTip": "string",
        "Type": "string"
    },
    "searchMetaDataType": {
        "facetArray": "facetType[]",
        "keywordSearchFieldsArray": "string[]",
        "securityFilters": "object"
    }
},

```

MetaType Default Value (Static Lookup)

```

{
    "facetArray": [
        {
            "id": "PlanTypeCd",
            "Default": true,
            "DisplayName": "Plan Type",
            "Type": "Facet",
            "FacetType": "StaticList",
            "ElasticAggregationsEnabled": false,

```



```

        "ElasticFacetName": "PlanTypeCd",
        "labelMapping": [
            {
                "label": "Plan Type A",
                "value": "A"
            },
            {
                "label": "Plan Type Q",
                "value": "Q"
            },
            {
                "label": "Plan Type N",
                "value": "N"
            }
        ]
    },
    {
        "id": "StatusCd",
        "Default": true,
        "DisplayName": "Plan Status",
        "Type": "Facet",
        "FacetType": "StaticList",
        "ElasticAggregationsEnabled": false,
        "ElasticFacetName": "StatusCd",
        "labelMapping": [
            {
                "label": "Active",
                "value": "A"
            },
            {
                "label": "In Progress",
                "value": "I"
            }
        ]
    }
]
}

```

Meta Data Default (Check Box and Multi Select)

```

{
    "facetArray": [
        {
            "id": "PlanTypeCd",
            "Default": true,
            "DisplayName": "Plan Type",

```

```

        "Type": "Facet",
        "FacetType": "CheckBox",
        "ElasticAggregationsEnabled": false,
        "ElasticFacetName": "PlanTypeCd",
        "KeyAttribute": "LookupCode",
        "DisplayAttribute": "Meaning",
        "Endpoint":
"oracle_hcm_compensationworkforcesetupUI:commonLookupsLOV/getall_commonLookupsLOV",
        "Finder": "LookupTypeFinder;LookupType=CMP_MD_STATUS",
        "Fields": "LookupCode,Meaning",
        "OrderBy": "Meaning"
    },
    {
        "id": "StatusCd",
        "Default": true,
        "DisplayName": "Plan Status",
        "Type": "Facet",
        "FacetType": "StaticList",
        "ElasticAggregationsEnabled": false,
        "ElasticFacetName": "StatusCd",
        "labelMapping": [
            {
                "label": "Active",
                "value": "A"
            },
            {
                "label": "In Progress",
                "value": "I"
            }
        ]
    }
]
}
}

```

Meta Data Default (Check Box and Multi Select and QParam)

```

{
    "facetArray": [
        {
            "id": "PlanTypeCd",
            "Default": true,
            "DisplayName": "Plan Type",
            "Type": "Facet",
            "FacetType": "CheckBox",
            "ElasticAggregationsEnabled": false,
            "ElasticFacetName": "PlanTypeCd",

```

```

        "KeyAttribute": "LookupCode",
        "DisplayAttribute": "Meaning",
        "Endpoint":
"oracle_hcm_compensationworkforcesetupUI:commonLookupsLOV/getall_commonLookupsLOV",
        "Finder": "LookupTypeFinder;LookupType=ORA_HRX_GB_CSP_SCHCAT",
        "Fields": "LookupCode,Meaning",
        "QParam": "LookupCode IN ('A', 'Q', 'N', 'C' )",
        "OrderBy": "Meaning"
    },
    {
        "id": "StatusCd",
        "Default": true,
        "DisplayName": "Plan Status",
        "Type": "Facet",
        "FacetType": "StaticList",
        "ElasticAggregationsEnabled": false,
        "ElasticFacetName": "StatusCd",
        "labelMapping": [
            {
                "label": "Active",
                "value": "A"
            },
            {
                "label": "In Progress",
                "value": "I"
            }
        ]
    }
]
}

```

Function to Get the Values From Search

(attached in Search Submit)

```
seatchObject <- $event.detail
```

```

PageModule.prototype.getSearchObject = function (searchObject) {

    //Deleting as its not supported in rest input payload
    delete searchObject.securityFilters;

    //The below searchFieldsArray is not necessary if the query is null. This has been
added since teh REST been used in this example is not working fine without this. Hence
added.

    if (!(searchObject && searchObject.query)) {
        searchObject.query = '';
    }
}

```

```

}

if (!(searchObject && searchObject.limit)) {
    searchObject.limit = 25;
}

var c = {};
c.PlanTypeCd = '';
c.StatusCd = '';
c.PlanName = '%' + searchObject.query + '%';

if (searchObject) {
    if (searchObject.filters && searchObject.filters.length > 0) {
        for (var i = 0; i < searchObject.filters.length; ++i) {
            var a = searchObject.filters[i];
            let value = '';
            let name = a.name[0];
            let term = a.terms ;
            if (a.value && a.value.length > 0){
                value = "" + a.value[0] + "";
            }
            if (name == "PlanTypeCd") {
                /* when type has checkbox-multi select */
                if (term && term.length > 0){
                    for (var k = 0; k < term.length; ++k) {
                        let condValue = term[k]
                        value += ( value? ",": "")+ "" + condValue + "";
                    }
                }

                if (value){
                    c.PlanTypeCd = value;
                }
            }
            if (name == "StatusCd") {
                c.StatusCd = value;
            }
        }
    }
}

```

```

    }

    return c;
};

```

Possible Facet Types

Date
 CheckBox
 MultiSelectLOV
 RadioButton
 StaticList
 Text
 Flag

Sample Q Parameter

```

"PlanName LIKE  '" + $page.variables.searchPlanName + "'" + (
$page.variables.searchPlanType? " AND PlanTypeCd IN ( '"+
$page.variables.searchPlanType + "'" : '') + ( $page.variables.SearchPlanStatus? "
AND StatusCd="+$page.variables.SearchPlanStatus : '' )

```

HCM - Multi Facet Advance Search Part 2

MetaType Default Value (Text Box)

```

{
  "facetArray": [
    {
      "id": "PlanTypeCd",
      "Default": true,
      "DisplayName": "Plan Type",

```

```

        "Type": "Facet",
        "FacetType": "StaticList",
        "ElasticAggregationsEnabled": false,
        "ElasticFacetName": "PlanTypeCd",
        "labelMapping": [
            {
                "label": "Plan Type A",
                "value": "A"
            },
            {
                "label": "Plan Type Q",
                "value": "Q"
            },
            {
                "label": "Plan Type N",
                "value": "N"
            }
        ]
    },
    {
        "id": "StatusCd",
        "Default": true,
        "DisplayName": "Plan Status",
        "Type": "Text",
        "FacetType": "Text",
        "ElasticAggregationsEnabled": false,
        "ElasticFacetName": "StatusCd",
        "ToolTip": "Status"
    }
]
}

```

MetaType Default Value (Check Box and Multi Select)

```

{
    "facetArray": [
        {
            "id": "PlanTypeCd",
            "Default": true,
            "DisplayName": "Plan Type",
            "Type": "Facet",
            "FacetType": "CheckBox",
            "ElasticAggregationsEnabled": false,
            "ElasticFacetName": "PlanTypeCd",

```

```

        "KeyAttribute": "LookupCode",
        "DisplayAttribute": "Meaning",
        "Endpoint":
"oracle_hcm_compensationworkforcesetupUI:commonLookupsLOV/getall_commonLookupsLOV",
        "Finder": "LookupTypeFinder;LookupType=CMP_MD_STATUS",
        "Fields": "LookupCode,Meaning",
        "OrderBy": "Meaning"
    },
    {
        "id": "StatusCd",
        "Default": true,
        "DisplayName": "Plan Status",
        "Type": "Facet",
        "FacetType": "StaticList",
        "ElasticAggregationsEnabled": false,
        "ElasticFacetName": "StatusCd",
        "labelMapping": [
            {
                "label": "Active",
                "value": "A"
            },
            {
                "label": "In Progress",
                "value": "I"
            }
        ]
    }
]
}

```

Meta Data Default (LOV Multi Select and QParam)

```

{
    "facetArray": [
        {
            "id": "PlanTypeCd",
            "Default": true,
            "DisplayName": "Plan Type",
            "Type": "Facet",
            "FacetType": "MultiSelectLOV",
            "ElasticAggregationsEnabled": false,
            "ElasticFacetName": "PlanTypeCd",
            "KeyAttribute": "LookupCode",
            "DisplayAttribute": "Meaning",
            "Endpoint":
"oracle_hcm_compensationworkforcesetupUI:commonLookupsLOV/getall_commonLookupsLOV",
            "Finder": "LookupTypeFinder;LookupType=ORA_HRX_GB_CSP_SCHCAT",
            "Fields": "LookupCode,Meaning",
            "QParam": "LookupCode IN ('A', 'Q', 'N', 'C')",
            "OrderBy": "Meaning"
        },
        {
            "id": "StatusCd",
            "Default": true,
            "DisplayName": "Plan Status",
            "Type": "Facet",
            "FacetType": "StaticList",

```

```

        "ElasticAggregationsEnabled": false,
        "ElasticFacetName": "StatusCd",
        "labelMapping": [
            {
                "label": "Active",
                "value": "A"
            },
            {
                "label": "In Progress",
                "value": "I"
            }
        ]
    }
}

```

Possible Facet Types

Date
 DateRange
 CheckBox
 MultiSelectLOV
 RadioButton
 StaticList
 Text
 Flag

```

Date
    id = 'dateId';
    Default = true;
    DisplayName = "Search Dae";
    Type = 'Date';
    ElasticFacetName = 'currentDate';
    ToolTip = "Search Date";

```

```

DateRange
    id = 'dateRangeId';
    Default = true;
    DisplayName = "Date Range ";
    Type = 'DateRange';
    ElasticFacetName = "Date Range";
    StartDateToolTip = "Start Date";
    timePeriod.EndDateToolTip = "End Date";

```

The changed Code Take care of Text Facet

```

PageModule.prototype.getSearchObject = function (searchObject) {

```

```

    //Deleting as its not supported in rest input payload

```



```
delete searchObject.securityFilters;

//The below searchFieldsArray is not necessary if the query is null. This has been
added since teh REST been used in this example is not working fine without this. Hence
added.
```

```
if (!(searchObject && searchObject.query)) {
    searchObject.query = '';
}
```

```
if (!(searchObject && searchObject.limit)) {
    searchObject.limit = 25;
}
```

```
var c = {};
c.PlanTypeCd = '';
c.StatusCd = '';
c.PlanName = '%' + searchObject.query + '%';
```

```
if (searchObject) {
    if (searchObject.filters && searchObject.filters.length > 0) {
        for (var i = 0; i < searchObject.filters.length; ++i) {
            var a = searchObject.filters[i];
            let value = '';
            let name = a.name[0];
            let term = a.terms ;
            /* for a text */
            if (a.term){
                term = a.term ;
            }
            if (a.value && a.value.length > 0){
                value = "" + a.value[0] + "";
            }
            /* when type has checkbox-multi select */
            if (term && term.length > 0){
                for (var k = 0; k < term.length; ++k) {
                    let condValue = term[k]
                    value += ( value? ",": "")+ " "+ condValue +"";
                }
            }
        }
    }
}
```

```
if (name == "PlanTypeCd") {
```

```

        c.PlanTypeCd = value;
    }
    if (name == "StatusCd") {
        c.StatusCd = value;
    }
}

}

}

return c;
};

```

Oracle VBCS Codes

Oracle VBCS Code

Exporting VBCS table to XL Shhet

```

define(['xlsx'], function(XLSX) {
    'use strict';

    class PageModule {
    }

    PageModule.prototype.downloadXLS = function (sdp) {
        var elt = document.getElementById(sdp);
        var wb = XLSX.utils.table_to_book(elt, { sheet: "sheet1" });
        return XLSX.writeFile(wb, 'MySheetName.xlsx');
    }

    return PageModule;
});

```

Calculating 13 or 14 months salary in TCS

Calculating 13 or 14 months salary in TCS

This is a sample formula for calculating 13 or 14 months salary for the Total Compensation Statement.

This is just a sample code, this may not work as it is for your needs, and you may need to change the logic as per your requirements.

Please see the YouTube Tutorial for an explanation of this code.

1) Simple Sample Formula

Assume your salary is monthly, the salary has not changed during the statement period.

The 13th salary is added to the salary of December.

```
/******  
FF Name: TILAK_ARRAY_13MONTH_SAL_1  
Type : Total Compensation Item  
Developer: Tilak  
Purpose: Calculate 13-month salary and adding that in DEC month  
  
*****/  
  
/* Declare DBI */  
  
DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT is 0  
DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR is 0  
DEFAULT FOR PER_ASG_LEGAL_ENTITY_ID is -1  
DEFAULT FOR CMP_ASSIGNMENT_SALARY_DATE_FROM is '1900/01/01' (date)
```

```

DEFAULT FOR COMPENSATION_DATES is EMPTY_TEXT_NUMBER
DEFAULT FOR VALUES is EMPTY_TEXT_NUMBER
DEFAULT FOR ASSIGNMENTS is EMPTY_TEXT_NUMBER
DEFAULT FOR LEGALEMPLOYERS is EMPTY_TEXT_NUMBER

/* Declare IV */

INPUTS ARE CMP_IV_PERIOD_START_DATE(DATE),
           CMP_IV_PERIOD_END_DATE (DATE)

/***** Main *****/
temp = SET_LOG('Entering Formula TILAK_ARRAY_13MONTH_SAL_1 ')
myAnnSalary = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
myAnnFactor = CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR

/*
myAnnFactor = 13

IF CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS DEFAULTED THEN
(
temp = SET_LOG('No Salary Found using 13K for testing ')
myAnnSalary = 13000
)
*/

asgld = get_context(HR_ASSIGNMENT_ID, -1)
leld = PER_ASG_LEGAL_ENTITY_ID

myMonthSalary = myAnnSalary / myAnnFactor
temp = SET_LOG('Annual Salary ' || TO_CHAR(myAnnSalary) )
temp = SET_LOG('Monthly Salary ' || TO_CHAR(myMonthSalary) )
temp = SET_LOG('Assinment ' || TO_CHAR(asgld) )
temp = SET_LOG('LE ' || TO_CHAR(leld) )

/* Assumed Salary is first of Every Month First the first date within the period */
firstDate = TRUNC(CMP_IV_PERIOD_START_DATE, 'MM')
endDate = CMP_IV_PERIOD_END_DATE

temp = SET_LOG('first Date ' || TO_CHAR(firstDate) )
temp = SET_LOG('Last Date ' || TO_CHAR(endDate) )

index = 1
IF (firstDate < CMP_IV_PERIOD_START_DATE) then
(
firstDate = ADD_MONTHS(firstDate, 1)
)

/* Create Row */
WHILE firstDate <= endDate
LOOP(
temp = SET_LOG('Processing Date ' || TO_CHAR(firstDate) )
COMPENSATION_DATES[index] = to_char(firstDate, 'YYYY/MM/DD')
ASSIGNMENTS[index] = to_char(asgld)
LEGALEMPLOYERS[index] = to_char(leld)

```

```

VALUES[index]          = to_char(myMonthSalary)
IF ( TO_CHAR(firstDate, 'MM') = '12' ) then
(
VALUES[index]          = to_char(myMonthSalary * 2)
)

index = index + 1
firstDate = add_months(firstDate, 1)
)

temp =SET_LOG('Leaving Formula TILAK_ARRAY_13MONTH_SAL_1 ')
RETURN COMPENSATION_DATES, VALUES, ASSIGNMENTS, LEGALEMPLOYERS

```

- 2) This Formula assumes that the salary might be changed within the period
Th 13th month salary is added as an entry on December

/*****

FF Name: TILAK_ARRAY_13MONTH_SAL_2
Type : Total Compensation Item
Developer: Tilak
Purpose: Calculate 13-month salary and adding that in DEC month

*****/

/ Declare DBI */*

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT is 0
DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR is 0
DEFAULT FOR PER_ASG_LEGAL_ENTITY_ID is -1
DEFAULT FOR CMP_ASSIGNMENT_SALARY_DATE_FROM is '1900/01/01' (date)

DEFAULT FOR COMPENSATION_DATES is EMPTY_TEXT_NUMBER
DEFAULT FOR VALUES is EMPTY_TEXT_NUMBER
DEFAULT FOR ASSIGNMENTS is EMPTY_TEXT_NUMBER
DEFAULT FOR LEGALEMPLOYERS is EMPTY_TEXT_NUMBER

/ Declare IV */*

INPUTS ARE CMP_IV_PERIOD_START_DATE(DATE),
CMP_IV_PERIOD_END_DATE (DATE)

```

/***** Main *****/
temp = SET_LOG('Entering Formula TILAK_ARRAY_13MONTH_SAL_1 ')
myAnnSalary = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
myAnnFactor = CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR

asgld = get_context(HR_ASSIGNMENT_ID, -1)
leld = PER_ASG_LEGAL_ENTITY_ID

temp = SET_LOG('Assinment ' || TO_CHAR(asgld) )
temp = SET_LOG('LE ' || TO_CHAR(leld) )

/* Assuumed Salary is first of Every Month First the first date within the period */
firstDate = TRUNC(CMP_IV_PERIOD_START_DATE, 'MM')
endDate = CMP_IV_PERIOD_END_DATE

temp = SET_LOG('first Date ' || TO_CHAR(firstDate) )
temp = SET_LOG('Last Date ' || TO_CHAR(endDate) )

index = 1
IF (firstDate < CMP_IV_PERIOD_START_DATE) then
(
  firstDate = ADD_MONTHS(firstDate, 1)
)

/* Create Row */
WHILE firstDate <= endDate
LOOP(
  myMonthSalary = 0
  /* Get the salary on the date */
  CHANGE_CONTEXTS(EFFECTIVE_DATE=firstDate)
  (
    myAnnSalary = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
    myAnnFactor = CMP_ASSIGNMENT_SALARY_ANNUALIZATION_FACTOR
    myMonthSalary = myAnnSalary / myAnnFactor

    temp = SET_LOG('Annual Salary ' || TO_CHAR(myAnnSalary) )
    temp = SET_LOG('Monthly Salary ' || TO_CHAR(myMonthSalary) )

  )

  if (myMonthSalary != 0 ) THEN
  (
    temp = SET_LOG('Processing Date ' || TO_CHAR(firstDate) )

```

```

temp =SET_LOG('Processing Date ' || TO_CHAR(firstDate) )
COMPENSATION_DATES[index] = to_char(firstDate, 'YYYY/MM/DD')
ASSIGNMENTS[index]      = to_char(asgld)
LEGAEMPLOYERS[index]    = to_char(leid)
VALUES[index]           = to_char(myMonthSalary)

/* add one more row for Dec */

IF ( TO_CHAR(firstDate, 'MM') = '12' ) then
(
  index = index + 1
  COMPENSATION_DATES[index] = to_char(firstDate, 'YYYY/MM/DD')
  ASSIGNMENTS[index]      = to_char(asgld)
  LEGALEMPLOYERS[index]    = to_char(leid)
  VALUES[index]           = to_char(myMonthSalary)
)

  index = index + 1
)
firstDate = add_months(firstDate, 1)
)

temp =SET_LOG('Leaving Fromula TILAK_ARRAY_13MONTH_SAL_1 ')
RETURN COMPENSATION_DATES, VALUES, ASSIGNMENTS, LEGALEMPLOYERS

```

TCS Annual Salary Fast Formula

TCS Annual Salary Fast Formula

<https://youtu.be/hNX3MIH-PBw>

/*****

FF Name: TILAK_TCS_TCS_ITEM_ANN_SALARY

Type : Total Compensation Item

Developer: Tilak

Purpose: Get Annual Salary of the Employee

```
*****/

/* Declare DBI */

DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT is -1
DEFAULT FOR PER_ASG_LEGAL_ENTITY_ID is -1
DEFAULT FOR CMP_ASSIGNMENT_SALARY_CURRENCY_CODE is 'x'

/* Declare IV */

INPUTS ARE  CMP_IV_PERIOD_START_DATE(DATE),
CMP_IV_PERIOD_END_DATE (DATE)

/***** Main *****/

temp = SET_LOG('Entering Fromula TILAK_SAL_FF ')
VALUES  = ''
COMPENSATION_DATES = ''
ASSIGNMENTS =  TO_CHAR(get_context(HR_ASSIGNMENT_ID, -1) )
LEGALEMPLOYERS = TO_CHAR(PER_ASG_LEGAL_ENTITY_ID)
COMPENSATION_DATES = TO_CHAR(CMP_IV_PERIOD_END_DATE , 'YYYY/MM/DD')
UNIT      = ''
IF (CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED ) THEN (
    VALUES = TO_CHAR(CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT)
    UNIT = CMP_ASSIGNMENT_SALARY_CURRENCY_CODE
)

temp =SET_LOG('Annual Salary ' || VALUES )
temp =SET_LOG('Assinment ' || ASSIGNMENTS)
temp =SET_LOG('LE ' || LEGALEMPLOYERS )
temp =SET_LOG('Date ' || COMPENSATION_DATES )
temp =SET_LOG('Unit ' || Unit )

temp =SET_LOG('Leaving Fromula TILAK_SAL_FF ')

RETURN COMPENSATION_DATES, VALUES, UNIT, ASSIGNMENTS, LEGALEMPLOYERS
```

TCS Stock Item in Criteria Item That uses Fast Formula

/******

FF Name: Tilak_TCS_ITEM_STOCK

Type : Total Compensation Item

Developer: Tilak

Purpose: Find a stock type for a given employee

*****/

/* Declare DBI */

DEFAULT_DATA_VALUE FOR CMP_ASSIGNMENT_RGE_SALARY_CHANGE_AMOUNT IS 0

DEFAULT FOR PER_ASG_LEGAL_ENTITY_ID is -1

DEFAULT_DATA_VALUE FOR

CMP_STOCK_DATA_RGE_ASG_ASSIGNMENT_ID is -1

DEFAULT_DATA_VALUE FOR CMP_STOCK_DATA_RGE_ASG_GRANT_TYPE is 'XXX'

DEFAULT_DATA_VALUE FOR CMP_STOCK_DATA_RGE_ASG_ORIGINAL_SHARES_GRANTED is -1

DEFAULT FOR COMPENSATION_DATES is ''

DEFAULT FOR VALUES is ''

DEFAULT FOR ASSIGNMENTS is ''

DEFAULT FOR LEGALEMPLOYERS is ''

/* Declare IV */

INPUTS ARE CMP_IV_PERSON_ID , CMP_IV_PERIOD_START_DATE(DATE), CMP_IV_PERIOD_END_DATE
(DATE),

CMP_IVR_ASSIGNMENT_ID(NUMBER_NUMBER) , CMP_IVR_TERM_ID(NUMBER_NUMBER),

CMP_IVR_PAY_ASSIGNMENT_ID (NUMBER_NUMBER),

CMP_IVR_PAY_TERM_ID(NUMBER_NUMBER), CMP_IVR_PAY_RELATION_ID(NUMBER_NUMBER)

/****** Main *****/

temp = SET_LOG('Entering Fromula Tilak_TCS_ITEM_STOCK ')

asgId = get_context(HR_ASSIGNMENT_ID, -1)

leId = PER_ASG_LEGAL_ENTITY_ID

perId = get_context(PERSON_ID, -1)

```

temp =SET_LOG('Assinment ' || TO_CHAR(asgId) )
temp =SET_LOG('LE ' || TO_CHAR(leId) )
temp =SET_LOG('person ' || TO_CHAR(perId) )
stdt = get_context(START_DATE, '1900/01/01' (date))
etdt = get_context(END_DATE, '1900/01/01' (date))

temp =SET_LOG('start date ' || TO_CHAR(stdt) )
temp =SET_LOG('End date ' || TO_CHAR(etdt) )

index = CMP_STOCK_DATA_RGE_ASG_ASSIGNMENT_ID.FIRST(-1)

temp = SET_LOG( ' first index ' + TO_CHAR(index) )

COMPENSATION_DATES = to_CHAR(etdt , 'YYYY/MM/DD')
VALUES          = 'N'
ASSIGNMENTS     = TO_CHAR(asgId)
LEGALEMPLOYERS  = TO_CHAR(leId)

WHILE (CMP_IVR_ASSIGNMENT_ID.EXISTS(index))
LOOP (
  ltype = CMP_STOCK_DATA_RGE_ASG_GRANT_TYPE[index]
  lshare = CMP_STOCK_DATA_RGE_ASG_ORIGINAL_SHARES_GRANTED[index] temp = SET_LOG( 'shares' +
TO_CHAR(lshare) )
temp = SET_LOG( 'TYpr ' + ltype )
  if ( ltype = 'ISO' AND lshare > 0 ) THEN
  (
    VALUES = 'Y'
    EXIT
  )

index = CMP_STOCK_DATA_RGE_ASG_ASSIGNMENT_ID.NEXT(index ,-1)
)

temp = SET_LOG('LEAVING Formula Tilak_TCS_ITEM_STOCK ' + VALUES )

RETURN COMPENSATION_DATES, VALUES, ASSIGNMENTS, LEGALEMPLOYERS

```
